**POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY**
Faculty of Information Technology 1

Course: Information System Analysis and Design

**COURSE ESSAY — VERSION 4.0 (FINAL)**

# ABC ONLINE SHOP: FROM REQUIREMENTS TO CODE

*Requirements, analysis, AI integration, design, implementation and demonstration of an e-commerce information system*

| | |
|---|---|
| **Student:** | Bế Quốc Khánh (KHANHBQ) |
| **Student ID:** | B23DCCE049 |
| **Class:** | 04 |
| **Lecturer:** | PGS.TS. Trần Đình Quế |
| **Version:** | V4 (final) |
| **Submitted:** | Report, Visual Paradigm project, database, source code, demo, defence guide |

Hanoi, November 2026

# INSTRUCTOR'S COMMENTS

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

**Score:** ..... (In words: .....)

Hanoi, date ..... month ..... year 2026

**Instructor**

# ABSTRACT

This essay is the final version (V4) of the course essay. It follows one information system, the **ABC Online Shop**, from requirement determination through requirements analysis, artificial intelligence in e-commerce and system design to a working, demonstrated implementation. The material was built up during the semester. Assignment 01 investigated the shop and its problems, Assignment 02 turned them into a requirements baseline with 32 use cases, Assignment 03 and Test 01 practised the data-flow, interaction, class and data models, and Assignment 06 produced a three-layer multimodal search prototype. The essay joins these pieces into one path from a business problem to a running program and explains the decision taken at each joint.

The main results are: a requirements baseline of 7 business problems, 14 stakeholder needs, 24 functional and 12 non-functional requirements traced to 32 use cases; an analysis model with activity, data-flow, sequence, class, state and entity-relationship diagrams; three AI components (multimodal search, aspect-based review insights and a stock-aware recommender) analysed as ordinary requirements with their own risks and fallbacks; an updated analysis class model and a layered design (typed design classes, six packages, data access objects), with a conceptual, logical and physical data model in which product details depend on the category, from which a database of 23 tables and 28 Python classes were generated; and a working Python application with a web interface, a SQLite database, 45 automated tests and an evaluation script. All 35 diagrams are kept in one Visual Paradigm project (TieuLuan_KHANHBQ.vpp). On the test sets, the search returns the expected item first for 25 of 26 queries (96.2%), the review analyser reaches an aspect F1 of 0.96, and every order-state and stock rule of the requirements is checked by a test.

*Keywords: requirements engineering, use case, data flow diagram, UML, data modeling, DAO, MVC, layered architecture, multimodal search, aspect-based sentiment analysis, recommendation, Python.*

## Abbreviations

*Table 1. Abbreviations used in the essay*

| Abbreviation | Meaning |
|---|---|
| ABSA | Aspect-based sentiment analysis |
| AI | Artificial intelligence |
| BCE | Boundary, control, entity (analysis object stereotypes) |
| BP | Business problem (Assignment 01) |
| DAO | Data access object |
| DFD | Data flow diagram |
| ERD | Entity relationship diagram |
| FR / NFR | Functional / non-functional requirement |
| MoSCoW | Must, Should, Could, Won't (prioritisation) |
| MVC | Model, view, controller |

| Abbreviation | Meaning |
|---|---|
| SN | Stakeholder need |
| TF-IDF | Term frequency, inverse document frequency |
| UC | Use case |
| UML | Unified Modeling Language |
| VP | Visual Paradigm, the modeling tool of all diagrams |
| WSGI | Web Server Gateway Interface (Python web standard) |

# CONTENTS

**ABSTRACT** **2**

Abbreviations . . . . . 2

**INTRODUCTION** **10**

**CHAPTER 1. SYSTEM REQUIREMENTS DETERMINATION** **12**

1.1. Requirements in the Development Life Cycle . . . . . 12

1.2. Investigation Approach . . . . . 12

1.3. Business Context . . . . . 12

1.4. Stakeholder Analysis . . . . . 13

1.5. Business Process . . . . . 14

1.6. Information Analysis . . . . . 16

1.7. Current Problems . . . . . 16

1.8. Scope, Boundary and Context . . . . . 17

1.9. System Objectives . . . . . 18

1.10. Elicitation Questions and Assumptions . . . . . 19

1.11. From Problems to Objectives . . . . . 19

1.12. Chapter Summary . . . . . 20

**CHAPTER 2. REQUIREMENTS ANALYSIS AND SPECIFICATION** **21**

2.1. Stakeholder Needs . . . . . 21

2.2. Functional Requirements . . . . . 21

2.3. Non-functional Requirements . . . . . 23

2.4. Prioritisation and Assumptions . . . . . 23

2.5. Actors and the Use Case Model . . . . . 24

2.6. Use Case Specification: UC08 Place Order . . . . . 28

2.7. Validation and Traceability . . . . . 28

2.8. Requirements for the AI Extensions . . . . . 29

2.9. Further Use Case Specifications . . . . . 29

2.10. Acceptance Criteria . . . . . 30

2.11. Managing Requirement Change . . . . . 31

2.12. From Requirements to Analysis Models . . . . . 31

2.13. Activity Modeling . . . . . 32

2.14. Data Flow Modeling . . . . . 37

2.15. Interaction Modeling . . . . . 38

2.16. Analysis Class Model . . . . . 39

2.17. Order State Machine . . . . . 41
2.18. Business Rules and Decision Logic . . . . . 42
2.19. Class Responsibilities . . . . . 43
2.20. CRUD Matrix . . . . . 43
2.21. Consistency Across Models . . . . . 44
2.22. Chapter Summary . . . . . 44
**CHAPTER 3. AI IN E-COMMERCE 45**
3.1. AI as an Enhancement Layer . . . . . 45
3.2. Opportunity Analysis . . . . . 45
3.3. AI Use Cases . . . . . 46
3.4. Multimodal Search: Principles . . . . . 46
3.5. Retrieval and Ranking . . . . . 47
3.6. Review Insights . . . . . 51
3.7. Recommendation . . . . . 53
3.8. Data Requirements . . . . . 53
3.9. Evaluation . . . . . 54
3.10. Error Analysis . . . . . 55
3.11. Risks, Fallbacks and Human Control . . . . . 56
3.12. Technology Options for a Production Version . . . . . 56
3.13. AI Component Specifications . . . . . 57
3.14. Worked Example: Ranking a Combined Query . . . . . 57
3.15. Worked Example: Analysing One Review . . . . . 58
3.16. Business Evaluation . . . . . 58
3.17. AI Traceability . . . . . 58
3.18. Chapter Summary . . . . . 59
**CHAPTER 4. SYSTEM DESIGN 60**
4.1. Class Diagram in Analysis . . . . . 60
4.2. Data Model and Database . . . . . 65
4.3. Class Diagram in Design . . . . . 76
4.4. Layers, Classes and Relationships . . . . . 80
4.5. Code Generated from the Layered Model . . . . . 87
4.6. Conclusion . . . . . 90
**CHAPTER 5. IMPLEMENTATION AND DEMONSTRATION 92**
5.1. Implementation Strategy and Environment . . . . . 92

5.2. Project Structure . . . . . 92
5.3. The Model . . . . . 93
5.4. Data Access Objects . . . . . 93
5.5. Services: Checkout and Payment . . . . . 94
5.6. Controllers and Views . . . . . 96
5.7. AI Components in Code . . . . . 103
5.8. Running the Application . . . . . 104
5.9. Testing . . . . . 106
5.10. Traceability from Requirements to Code and Tests . . . . . 107
5.11. What Writing the Code Changed in the Models . . . . . 108
5.12. Development History . . . . . 109
5.13. Chapter Summary . . . . . 110
**CHAPTER 6. CONCLUSION AND FUTURE WORK** **111**
6.1. Summary of Deliverables . . . . . 111
6.2. Evaluation Against the Requirements . . . . . 111
6.3. Limitations . . . . . 112
6.4. Lessons and Next Steps . . . . . 112
6.5. Submitted Files . . . . . 112
**REFERENCES** **113**

# LIST OF FIGURES

Figure 1. Stakeholder map: internal and external stakeholders and their interest in the shop - VP diagram "STK01 Stakeholder Map"
Figure 2. High-level business process of ABC Online Shop as a BPMN collaboration: customer, shop lanes and partner pools - VP diagram "BP01 High-Level Business Process"
Figure 3. Problem tree: root causes, the six business problems, the core problem and its effects - VP diagram "PT01 Problem Tree"
Figure 4. System context diagram - VP diagram "CTX01 System Context Diagram"
Figure 5. General use case diagram: all actors and the main use case of each package - VP diagram "UC00 General Use Case Diagram"
Figure 6. Use cases of the Account and Catalogue package - VP diagram "UCD1 Account and Catalogue"
Figure 7. Use cases of the Cart, Order and Payment package - VP diagram "UCD2 Cart, Order and Payment"
Figure 8. Use cases of the Administration package - VP diagram "UCD3 Administration"
Figure 9. Use cases of the Operations and Reporting package - VP diagram "UCD4 Operations and Reporting"
Figure 10. Place order activity with swimlanes - VP diagram "AD01 Place Order (Swimlane)"
Figure 11. Order processing and shipment - VP diagram "AD02 Process and Ship Order (Swimlane)"
Figure 12. Manage inventory - VP diagram "AD03 Manage Inventory"
Figure 13. Level 0 data flow diagram - VP diagram "DFD0 Level 0 Diagram"
Figure 14. Level 1 DFD of process 4.0 - VP diagram "DFD1 Level 1 - 4.0 Place Order and Pay"
Figure 15. Sequence diagram of the main checkout scenario - VP diagram "SD01 Place Order (Analysis)"
Figure 16. Sequence diagram of order tracking - VP diagram "SD02 Track Order (Analysis)"
Figure 17. Analysis class diagram - VP diagram "CL01 Analysis Class Diagram"
Figure 18. Order state machine - VP diagram "ST01 Order State Machine"
Figure 19. AI-enabled use cases - VP diagram "UC_AI AI-Enabled Use Cases"
Figure 20. Three-layer architecture of the search subsystem - VP diagram "CMP01 AI Search Three-Layer Architecture"
Figure 21. Voice search - VP diagram "SD03 Voice Search"
Figure 22. Image search - VP diagram "SD04 Image Search"
Figure 23. Output of the search part of main.py demo: text, voice, image, text + image
Figure 24. Review insight pipeline - VP diagram "AD04 Review Insight Pipeline"
Figure 25. Output of evaluate.py, part 1: multimodal search on 26 test queries
Figure 26. Output of evaluate.py, parts 2 and 3: review insights and recommender
Figure 27. Analysis class diagram of ABC Online Shop, updated with association names, multiplicities and the checkout dependency - VP diagram "CL01 Analysis Class Diagram"
Figure 28. Conceptual data model: entities and relationships with their multiplicities - VP diagram "CDM01 Conceptual Data Model"
Figure 29. Logical data model: relations with attributes, keys, optional and unique attributes - VP diagram "LDM01 Logical Data Model"
Figure 30. Physical data model, part 1: customers, carts, orders, payments and shipments - VP diagram "ERD01"
Figure 31. Physical data model, part 2: catalogue, stock, reviews and the technical tables - VP diagram "ERD02"
Figure 32. The database created from the generated script, compared with the application schema, filled by the application and checked with statements that must fail

Figure 33. Design classes, part 1: customers, carts and orders with typed attributes, methods, key fields and compositions - VP diagram "DCL01"
Figure 34. Design classes, part 2: payment, shipment and review of an order - VP diagram "DCL02"
Figure 35. Design classes, part 3: catalogue, promotion and event-based stock - VP diagram "DCL03"
Figure 36. Layers of the application: the six packages, their classes and the <<use>> dependencies between them - VP diagram "PKG01"
Figure 37. One slice through the layers: the classes that take part in adding a product to the cart - VP diagram "DCL04"
Figure 38. Checkout through the layers - VP diagram "SD05 Checkout Through the Layers (Design)"
Figure 39. Packages and classes generated from the model, compiled, imported and compared with the hand-written application
Figure 40. Catalogue page with the multimodal search box (UC04, UC05)
Figure 41. Product page with AI review insights and recommendations (UC06, UC33, UC34)
Figure 42. Product page of a bag: the specification table comes from bag_detail, the detail table named by the category (UC06, BR-CAT-01)
Figure 43. Voice search result with scores and the processing trace (UC05)
Figure 44. Checkout with promotion WELCOME10, shipping and simulated payment options (UC08, UC31, UC32)
Figure 45. Order tracking: status timeline from order history and carrier milestones (UC11)
Figure 46. Order question "where is my latest order" answered from the customer's own orders (AI-R2)
Figure 47. Sales staff processing queue (UC19)
Figure 48. Warehouse inventory with low-stock rows and event form (UC20, UC24)
Figure 49. Review moderation queue: the spam review held by the AI noise filter (UC18, AI-R5)
Figure 50. Management dashboard computed from operational records (UC25-UC28)
Figure 51. Output of main.py demo, sections 2-8: checkout, idempotency, failure, fulfilment, AI and dashboard
Figure 52. Output of pytest: 45 tests passed

# LIST OF TABLES

Table 1. Abbreviations used in the essay
Table 2. Chapters of the essay and the course work they build on
Table 3. Business profile of ABC Online Shop
Table 4. Stakeholders, interests and evidence of success
Table 5. Information catalogue (from Assignment 01, Table 3)
Table 6. Business problems, likely causes and consequences
Table 7. Working assumptions and the questions that validate them
Table 8. Traceability from business problems to system objectives
Table 9. Stakeholder need catalogue (Assignment 02)
Table 10. Functional requirements
Table 11. Measurable non-functional requirements
Table 12. MoSCoW baseline
Table 13. Actors
Table 14. Specification of UC08 Place Order / Checkout
Table 15. Traceability from business problems to use cases
Table 16. AI requirements
Table 17. Specification of UC09 Make Payment
Table 18. Specification of UC20 Manage Inventory

Table 19. Acceptance criteria of key requirements
Table 20. Analysis questions, models and the requirements they refine
Table 21. Relationships of the analysis model and their justification
Table 22. Decision table: may an order enter Processing? (UC19)
Table 23. Decision table: may an order be cancelled? (UC12)
Table 24. Promotion rule (UC32)
Table 25. Analysis classes and their responsibilities
Table 26. CRUD matrix (selected use cases)
Table 27. Cross-model consistency checks
Table 28. AI opportunities derived from the business problems
Table 29. Data cards of the AI components
Table 30. Evaluation summary
Table 31. AI risks and the design response
Table 32. Prototype components and production replacements
Table 33. AI component specifications
Table 34. Score components of the top three results
Table 35. Business metrics of the AI features
Table 36. From problem to AI requirement, component, metric and test
Table 37. Updates of the analysis class diagram
Table 38. Relationships of the analysis class diagram
Table 39. Decisions of the logical model
Table 40. Mapping rules from analysis classes to tables
Table 41. Integrity rules expressed as database constraints
Table 42. The twenty-three tables of the database
Table 43. Updates of the design classes
Table 44. Methods of the model classes
Table 45. Relationships of the design classes and their form in code
Table 46. DAO interfaces of the dao layer
Table 47. One request through the layers: a customer adds two units of a product to the cart
Table 48. Service classes of the service layer
Table 49. Routes of the web application and the use cases they serve
Table 50. Mapping rules from the design model to Python
Table 51. Non-functional requirements and the parts of the design that serve them
Table 52. Design decisions, rejected alternatives and accepted costs
Table 53. Environment and size of the code base
Table 54. Commands
Table 55. Test modules
Table 56. From requirement to design element, code and test
Table 57. Mismatches found while implementing
Table 58. Commit history
Table 59. Requirement groups and the evidence that they are met
Table 60. Submitted files

# INTRODUCTION

ABC Online Shop is a medium-sized retailer that sells consumer products online and works with a payment provider, a shipping provider and suppliers. Its central difficulty, established in Assignment 01, is not the absence of a website but fragmented and delayed information: product data differs between places, stock figures lag behind the warehouse, orders are re-entered by hand, customers cannot see where their order is, and reports arrive after the decisions they were meant to support. Every assignment of the course looked at this shop from a different angle. Written as one essay, they become a single path from a business problem to a running program.

The interesting part of that path is the joints. A requirement written vaguely in the second week turns into an ambiguous use case, then into a class without a clear responsibility, and finally into code that nobody can test. A requirement written well can be followed all the way down: the essay shows this with several threads, for example the rule *a payment result changes an order only once*, which starts as NFR07 in Chapter 1, becomes alternative flow A3 of UC09, a guard on the order state machine in Chapter 2, an idempotent method in the design of Chapter 4 and finally a test in Chapter 5.

*Table 2. Chapters of the essay and the course work they build on*

| Chapter | Main question | Built on |
|---|---|---|
| 1. System Requirements Determination | What problem does the shop have, who is involved and where is the system boundary? | Assignment 01 (investigation) |
| 2. Requirements Analysis and Specification | What must the system do, and which behaviour, data and classes does that contain? | Assignment 02 (requirements, use cases, activity diagrams), Assignment 03 (DFD, sequence, class models) |
| 3. AI in E-commerce | Where does AI help the shop, and how do we know that it helps? | Assignment 02 (AI review analysis), Assignment 06 (multimodal search) |
| 4. System Design | How do the analysis classes become data, design classes, layers and code? | Test 01 (eComDesign), Assignment 06 (three-layer architecture), lecture on analysis-to-design steps |
| 5. Implementation and Demonstration | Does the design run, and can each function be traced back to its model? | Assignment 06 prototype, Python application abc_shop |
| 6. Conclusion | What was achieved, what is missing and what comes next? | - |

Three conventions hold throughout. First, identifiers are the ones fixed in Assignment 02 (BP, SN, FR, NFR, UC), so FR10 in Chapter 1 is the same FR10 that appears in a test name in Chapter 5. Second, every diagram is a Visual Paradigm diagram of the project TieuLuan_KHANHBQ.vpp, and the figure caption gives its name in the project. The diagrams were exported with Visual Paradigm Community Edition; because Community Edition cannot open data flow diagrams, the three DFD-type diagrams (CTX01, DFD0, DFD1) are drawn from the shapes and routes stored in the same project, in the notation Visual Paradigm uses. Third, every number in Chapters 3 and 5 comes from a program that was run: the outputs are

stored in the folder demo_results and can be reproduced with the commands of Section 5.8.

The AI chapter sits in the middle on purpose. AI functions are often presented as separate products. In this essay they are treated as one more set of requirements, with stakeholders, use cases, components, risks and tests like every other part of the shop. The design and the code therefore contain them from the start rather than as an attachment.

# CHAPTER 1. SYSTEM REQUIREMENTS DETERMINATION

Requirement determination answers two questions before any model or code exists: what is wrong in the business, and where an information system has to stand to stop it from being wrong. This chapter summarises the investigation of Assignment 01: the business, its stakeholders, the end-to-end process, the problems and their causes, and the boundary and objectives of the system. Chapter 2 turns this picture into requirements and analysis models.

## 1.1. Requirements in the Development Life Cycle

In the system development life cycle, requirements are the hinge between the business and the software. Errors made here are the most expensive ones, because every later artifact inherits them: an analysis class models a misunderstood concept, a test confirms the wrong behaviour, and the shop receives a program that works but does not help. The course follows a chain that keeps each step traceable:

*Business problems → Stakeholder needs → System requirements → Use cases → Validation*

The chain is used in both directions. Forwards, each problem must lead to at least one requirement; backwards, each requirement must be justified by a need, so that nothing is built only because it is technically interesting. Section 2.7 checks both directions with a traceability matrix.

## 1.2. Investigation Approach

The evidence is the case description of the assignment brief, analysed in Assignment 01. Where the brief only states a symptom (for example *order status is unclear*), the cause was written down as a hypothesis and an elicitation question rather than as a fact, because inventing causes produces requirements for problems the shop may not have. Four techniques were used: document analysis of the case, stakeholder analysis, process modeling of the order-to-feedback flow, and problem-cause-consequence analysis summarised as a problem tree. Questions that only the shop can answer were collected as assumptions to validate (for example which payment provider is used and how it reports timeouts); they appear again as the assumptions of Section 2.4.

## 1.3. Business Context

*Table 3. Business profile of ABC Online Shop*

| Item | Description |
|---|---|
| Organisation | Medium-sized business-to-consumer online retailer |
| Products | General consumer products organised in categories; the prototype uses shoes, bags, shirts, caps and watches |
| Customers | Visitors who browse without an account; registered customers who order, pay, track and review |
| Sales channel | Own website (responsive web storefront); no marketplace, no physical store |
| Partners | Payment provider (authorises payments), shipping provider (delivers parcels), suppliers (replenish stock) |
| Staff roles | Administrator, sales staff, warehouse staff, manager |
| Business objective | Increase reliable online sales while controlling fulfilment cost and keeping customer trust |

The business model is information-intensive. In a physical shop a customer sees the item on the shelf; online, the customer relies entirely on a description, a price, an availability flag, reviews, a payment confirmation and a delivery status. The shop's promise is therefore only as good as the information that connects customers, staff and the three external partners.

## 1.4. Stakeholder Analysis

A stakeholder has an interest in the system; an actor interacts with it directly. Keeping the two apart prevents a common error, drawing a department or a database as an actor. The table below lists the stakeholders with the evidence that would show that the system serves them.

*Table 4. Stakeholders, interests and evidence of success*

| Stakeholder | Type | Main interest | Evidence of success |
|---|---|---|---|
| Visitor / Customer | External | Find accurate products, pay safely, know where the order is | Fewer status enquiries; checkout without help |
| Administrator | Internal | One controlled source for catalogue, promotions and reviews | Audited changes; no conflicting product data |
| Sales staff | Internal | Process orders without re-entering data | Processing queue with complete order facts |
| Warehouse staff | Internal | Current stock, clear pick lists | Stock explained by events; no negative balance |
| Manager | Internal | Timely, reproducible reports | Dashboard computed from operational records |
| Payment provider | External | Uniquely referenced requests | Every result matched to exactly one attempt |
| Shipping provider | External | Correct delivery data | Every milestone matched to one shipment |
| Supplier | External | Clear replenishment requests | Requests raised when stock reaches the threshold |

The stakeholder map groups the same eight stakeholders by their position. The four internal roles work inside the shop and change its data; the four external parties exchange information

with it across the system boundary. Each box carries the one interest that the system must serve for that stakeholder, so the map doubles as a checklist for the requirements of Chapter 2: every interest returns as at least one need.

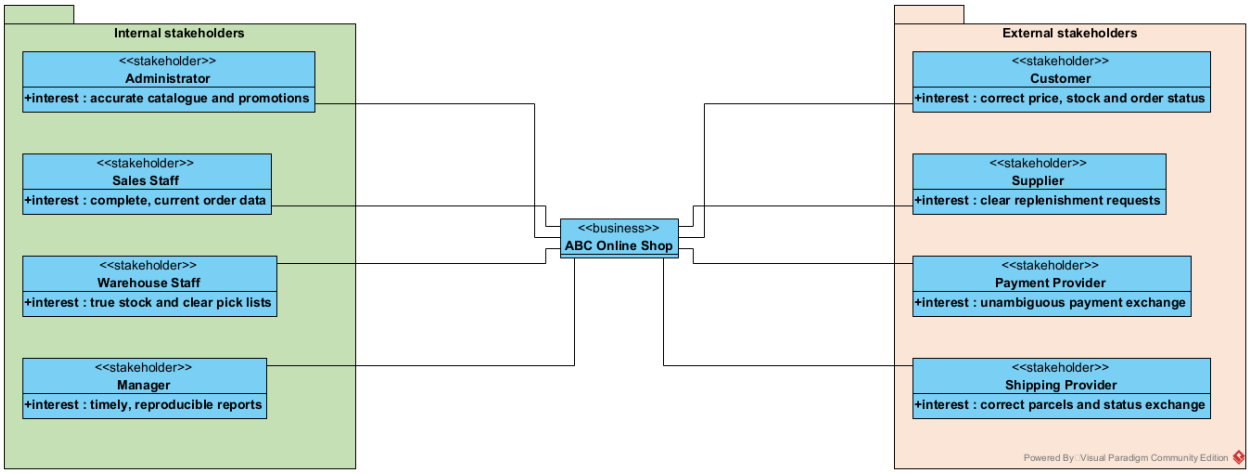



*Figure 1. Stakeholder map: internal and external stakeholders and their interest in the shop - VP diagram "STK01 Stakeholder Map"*

## 1.5. Business Process

The core business process runs from browsing to feedback in eight steps: (1) browse products, (2) build a cart, (3) validate the cart and create an order, (4) request payment, (5) reserve and update stock, (6) prepare and ship, (7) deliver and collect feedback, (8) report results and exceptions. Each step consumes information produced by an earlier one, which is why a delay anywhere becomes a problem everywhere: a stock figure that is one day old lets a customer order an item that no longer exists, which turns into a cancellation, a refund, an angry review and a wrong sales report.

The BPMN collaboration diagram shows the same process as message exchanges between five participants. The customer's pool holds what the customer does; the shop's pool is divided into the online system and the three staff lanes; payment provider, shipping provider and supplier are separate pools, because the shop does not control their work, only the messages it sends and receives (dashed message flows). The exclusive gateway *Payment verified?* is the point where the shop's information either becomes trustworthy or the order is stopped and the reserved stock is released.

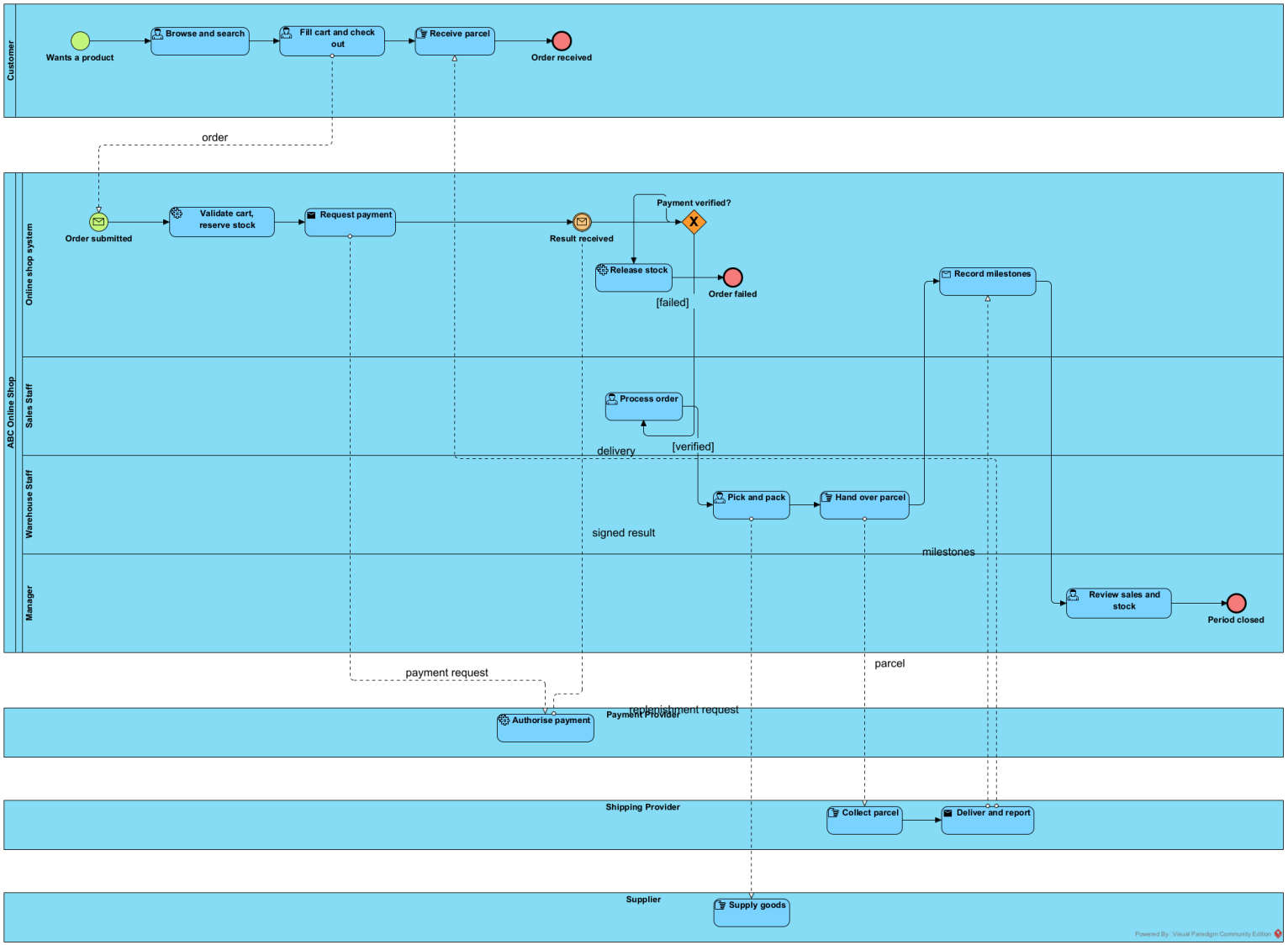



*Figure 2. High-level business process of ABC Online Shop as a BPMN collaboration: customer, shop lanes and partner pools - VP diagram "BP01 High-Level Business Process"*

## 1.6. Information Analysis

*Table 5. Information catalogue (from Assignment 01, Table 3)*

| Information item | Produced by | Used by | Quality concern |
|---|---|---|---|
| Product, category, price | Administrator | Visitors, checkout, reports | One version; changes audited |
| Customer, address | Customer | Order, delivery, service | Unique, current, access-controlled |
| Cart | Customer | Checkout | Revalidated before ordering |
| Order, status history | System from events | Customer, staff, manager | One identifier; valid states; timestamps |
| Inventory event, stock level | Warehouse, order process | Browsing, checkout, warehouse | Traceable to events; never negative |
| Payment attempt, result | System, payment provider | Order, reports | Unique reference; no card data stored |
| Shipment, milestone | Shipping provider | Customer, staff | Matched to one shipment; no regression |
| Review | Customer with delivered order | Customers, administrator | Linked to a real purchase |
| Reports | System | Manager | Reproducible for a stated period |

Two controls follow from the catalogue and shape the whole design. Identifiers must link related records across order, payment, inventory and shipment information, and a status must be derived from recorded events rather than typed in: an order is *Paid* because a verified payment result exists, not because somebody expected one.

## 1.7. Current Problems

*Table 6. Business problems, likely causes and consequences*

| ID | Problem | Likely cause (to verify) | Consequence |
|---|---|---|---|
| BP01 | Inconsistent product information | Several copies edited without one owner | Conflicting offers, disputes |
| BP02 | Outdated inventory information | Receipts and sales posted manually, in batches | Orders for unavailable stock, cancellations |
| BP03 | Manual order processing | Re-entry of cart, payment and delivery data | Delay, transcription errors |
| BP04 | Unclear order status | Milestones stored separately, no state rules | Repeated enquiries, late detection of failures |
| BP05 | Slow sales reporting | End-of-period manual consolidation | Late, irreproducible figures |
| BP06 | Fragmented customer information | No master identifier, duplicates | Delivery errors, privacy exposure |
| BP07 | Limited real-time management information | Operational events disconnected | Reactive decisions |

The problem tree of Assignment 01 places one central problem, *fragmented and delayed information management*, between three causes (manual re-entry, separate records without shared identifiers, unclear ownership and state rules) and three groups of consequences (order

and fulfilment errors, unclear status, slow reporting). The tree also explains the priorities: inventory accuracy and order status come first because they affect revenue, customer trust and staff workload at the same time, while reporting can only be as good as the operational records it reads.

The problem tree is read from top to bottom, from causes to effects. The top row holds the root causes, the second row the six observable problems of Assignment 01 (BP01-BP06) that they produce. All six lead to one core problem: order, stock and payment information is fragmented and late. At the bottom are the effects the business feels: customers lose trust, sales are lost or cancelled, and management decides late. The root causes, not the symptoms, are what the requirements of Chapter 2 address: a single product source, stock as typed events, verified payment results, defined order states, carrier milestones with references and reports computed from records.

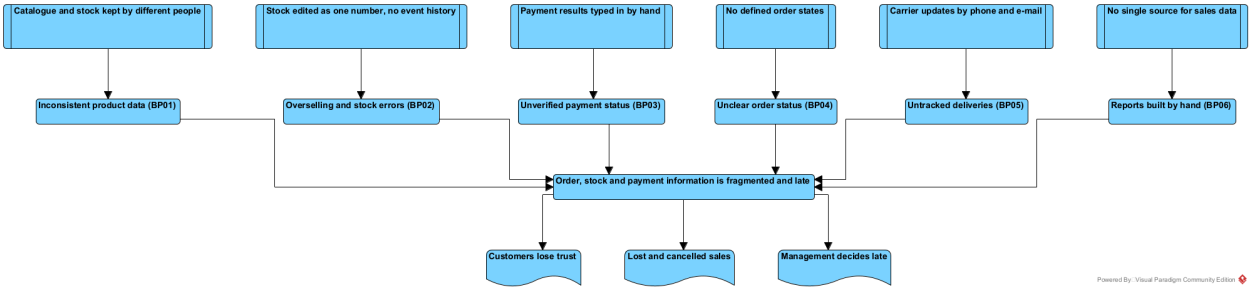



*Figure 3. Problem tree: root causes, the six business problems, the core problem and its effects - VP diagram "PT01 Problem Tree"*

## 1.8. Scope, Boundary and Context

The system boundary decides what the shop's software does itself and what it only exchanges with others. Inside are authentication and roles, customer profiles, catalogue and search, cart and checkout, order and payment-status coordination, inventory, shipment-status integration, promotions, reviews and reports. Outside are payment authorisation and settlement, carrier routing and physical delivery, and supplier operations: the shop sends a uniquely referenced request and records the attributable result. The figure below shows the boundary as a context diagram, in which the whole system is one process (0) and every arrow is a named data flow.

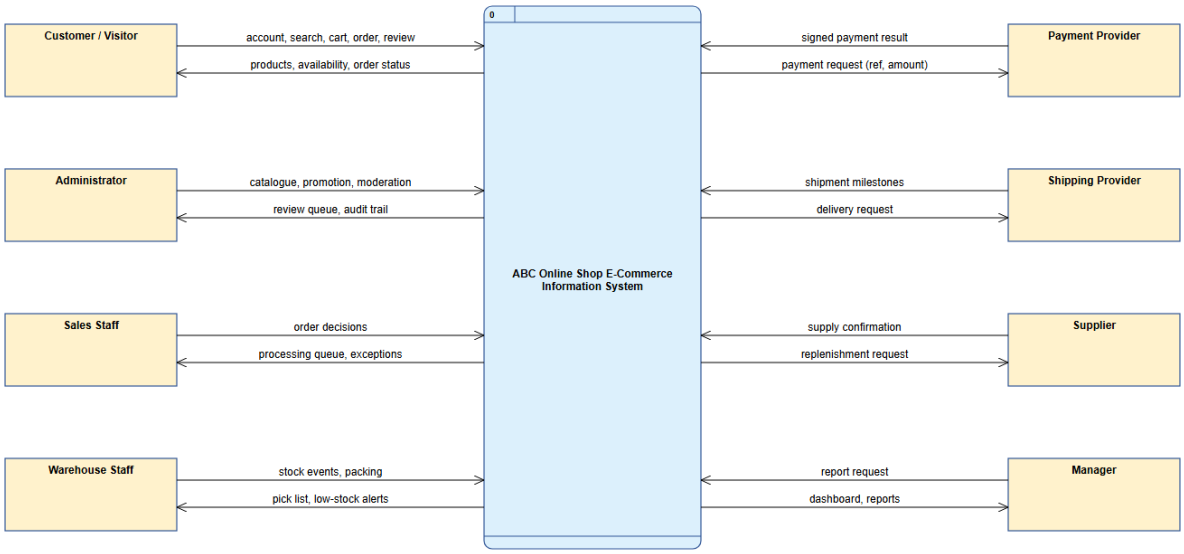



*Figure 4. System context diagram - VP diagram "CTX01 System Context Diagram"*

The context diagram is deliberately symmetric: every external entity both sends and receives. A one-way arrow would hide a requirement. For example, the shipping provider does not only receive a delivery request; it also sends milestones, and without that return flow the system could never answer the customer's question "where is my order?" (BP04).

## 1.9. System Objectives

- **O1** One controlled, role-authorised source for product, customer, promotion, order, stock, payment-status and shipment-status information (BP01, BP06).
- **O2** Record inventory events promptly and show one consistent available quantity at browsing, checkout, fulfilment and reporting (BP02).
- **O3** Coordinate checkout, payment, reservation, fulfilment, shipment and cancellation without re-entering data (BP03).
- **O4** Give customers and staff a timestamped order status derived from recorded events (BP04).
- **O5** Exchange uniquely referenced requests and results with the payment and shipping providers (BP03, BP04).
- **O6** Produce reproducible sales, inventory and exception reports from operational records (BP05, BP07).
- **O7** Protect personal and transaction data by data minimisation, role-based access and an audit trail (BP06).

## 1.10. Elicitation Questions and Assumptions

The investigation could not observe the shop directly, so every interpretation that goes beyond the case description was recorded with the question that would confirm or reject it. The list doubles as an interview plan for the first meeting with the shop's staff.

*Table 7. Working assumptions and the questions that validate them*

| ID | Working assumption | Question / evidence required |
|---|---|---|
| A1 | ABC sells general consumer products through its own website only | Which channels accept orders today: website, social media, marketplace, phone? |
| A2 | Separate copies and batch updates cause the inconsistencies | Where is each product, customer, order and stock item recorded, who owns it, how often is it updated? |
| A3 | Available stock = physical stock minus reserved quantities | When exactly is stock reserved, released, issued, received and adjusted? |
| A4 | Order status can be derived from accepted business events | Which status names, transition rules and customer-visible wordings are approved? |
| A5 | Providers expose a reference and a status for every exchange | Which providers are used; how are duplicates, timeouts and late updates reported? |
| A6 | Only customers with a delivered order may review | Can visitors review? Who approves or removes a review? |
| A7 | Management needs sales, inventory and exception reports | Which decision uses each report, how often, with which definitions? |

## 1.11. From Problems to Objectives

*Table 8. Traceability from business problems to system objectives*

| Problem | Objectives | Evidence expected after implementation |
|---|---|---|
| BP01, BP06 inconsistent product and customer data | O1, O7 | One controlled record per item; audited changes; fewer conflicting entries |
| BP02 outdated inventory | O2, O3 | Every quantity explained by events; fewer cancellations for unavailable items |
| BP03 manual order processing | O3, O5 | No re-entry; provider results matched to orders automatically |
| BP04 unclear order status | O3, O4, O5 | Timestamped status history; fewer routine status enquiries |
| BP05, BP07 slow and limited management information | O1, O6 | Reports computed from operational records with a visible refresh time |

The objectives deliberately avoid numeric targets such as "reduce enquiries by 30%". A target needs a baseline, and the shop has none yet; inventing one during the investigation would give the project a false precision. The candidate measures (status enquiries per 100 orders, stock discrepancies, orders cancelled for unavailable items, report preparation time) are recorded so that a baseline can be taken before the system goes live.

## 1.12. Chapter Summary

The shop's difficulty is not the lack of a website but fragmented and delayed information. Eight stakeholders were identified and mapped, the order-to-feedback process was modelled as a BPMN collaboration, and six problems were traced in a problem tree to their root causes. The context diagram fixes the boundary: the system owns catalogue, orders, stock, payments records, shipments and reports, and exchanges messages with the payment provider, the shipping provider and suppliers. The objectives give each root cause a measurable target.

# CHAPTER 2. REQUIREMENTS ANALYSIS AND SPECIFICATION

This chapter turns the investigation of Chapter 1 into a requirements baseline and analyses it. The first half states what the system must do: stakeholder needs, functional and non-functional requirements, their priority, the actors and the use case model with its specifications. The second half analyses how the requirements behave: activity and data flow diagrams, analysis sequence diagrams with boundary, control and entity objects, the analysis class model and the order state machine. The analysis models stay independent of technology; Chapter 4 decides how they are built.

## 2.1. Stakeholder Needs

*Table 9. Stakeholder need catalogue (Assignment 02)*

| ID | Stakeholder | Need | Priority |
|---|---|---|---|
| SN01 | Visitor / Customer | Find accurate, current product and availability information | Must |
| SN02 | Customer | Maintain selected products and quantities before ordering | Must |
| SN03 | Customer | Complete a clear and secure checkout and payment | Must |
| SN04 | Customer | View order history, processing state and delivery progress | Must |
| SN05 | Customer | Cancel an eligible order and review a verified purchase | Should |
| SN06 | Administrator | Maintain products, categories, promotions and reviews from one source | Must |
| SN07 | Sales staff | Process orders and answer enquiries without re-entering information | Must |
| SN08 | Warehouse staff | See current stock and record every inventory movement promptly | Must |
| SN09 | Warehouse staff | Prepare shipments and coordinate traceable replenishment | Must |
| SN10 | Manager | Obtain timely dashboards and reproducible sales reports | Should |
| SN11 | Manager | See inventory, order, payment, shipment and exception information | Must |
| SN12 | All users / ABC | Protect personal and transaction information by role and purpose | Must |
| SN13 | Payment provider | Receive a uniquely referenced request and return an attributable result | Must |
| SN14 | Shipping provider | Receive correct delivery data and return traceable milestones | Must |

## 2.2. Functional Requirements

Each functional requirement is an observable "The system shall" statement, has one identifier and one MoSCoW priority, and is allocated to a use case. None of them prescribes a database, a framework or a provider's internal design; those decisions belong to Chapter 4.

*Table 10. Functional requirements*

| ID | The system shall ... | Pri. | UC |
|---|---|---|---|
| FR01 | allow a Visitor to register a Customer account with validated identity and contact data | M | UC01 |
| FR02 | authenticate users and authorise functions according to their roles | M | UC02 |
| FR03 | allow a Customer to view and update profile and delivery address | S | UC03 |
| FR04 | display the active catalogue and categories to a Visitor | M | UC04 |
| FR05 | allow a Visitor to search and filter active products | M | UC05 |
| FR06 | display description, current price, promotion and available-to-order status | M | UC06 |
| FR07 | allow a Customer to add an available product, change its quantity or remove it | M | UC07 |
| FR08 | validate the cart, recheck availability and calculate the authoritative total before ordering | M | UC08, UC29-31 |
| FR09 | create one uniquely identified order linked to customer, lines, address, totals and status history | M | UC08 |
| FR10 | send a uniquely referenced payment request and record each attributable result once | M | UC09 |
| FR11 | allow a Customer to view own order history and details | M | UC10 |
| FR12 | show current processing and shipment status with the last update time | M | UC11 |
| FR13 | allow an eligible order to be cancelled according to status and refund rules | S | UC12 |
| FR14 | allow a Customer with a delivered order line to submit one review for that product | C | UC13 |
| FR15 | allow an Administrator to create, update, publish and categorise products | M | UC14-15 |
| FR16 | allow an Administrator to maintain promotion rules and moderate reviews | S | UC16, UC18 |
| FR17 | allow authorised staff to find and update permitted customer information with an audit trail | S | UC17 |
| FR18 | allow Sales Staff to progress orders only through valid state transitions | M | UC19 |
| FR19 | record receipts, reservations, releases, issues and adjustments and recalculate stock | M | UC20 |
| FR20 | provide a pick list and record packing against the order | M | UC21 |
| FR21 | send a uniquely referenced delivery request and record attributable milestones | M | UC22-23 |
| FR22 | issue and track a replenishment request when stock reaches its threshold | S | UC24 |
| FR23 | provide authorised dashboards and reproducible reports for a selected period | M | UC25-28 |
| FR24 | apply a promotion only when code, dates, scope and minimum total are valid | S | UC32 |

## 2.3. Non-functional Requirements

*Table 11. Measurable non-functional requirements*

| ID | Category | Requirement / acceptance criterion |
|---|---|---|
| NFR01 | Performance | 95% of browse and search requests complete within 2 s under normal load |
| NFR02 | Performance | Checkout validation and total calculation within 3 s for 95% of requests (provider time excluded) |
| NFR03 | Availability | Monthly availability at least 99.5%; RPO 15 min, RTO 2 h |
| NFR04 | Security | Personal, authentication, payment-status and delivery data in transit use TLS 1.2 or later |
| NFR05 | Access control | Role-based least privilege; a Customer accesses only own profile and orders |
| NFR06 | Auditability | Role, order-state, inventory, price and promotion changes record actor, time, identifier, before/after |
| NFR07 | Integrity | Payment and shipment requests use unique references; a replay never duplicates a transaction, stock event or state change |
| NFR08 | Usability | 90% of first-time users complete checkout in at most five screens without help |
| NFR09 | Maintainability | Automated tests cover order, payment, inventory and shipment rules; every Must rule has a regression test |
| NFR10 | Privacy | Only data needed by the task is shown; raw payment credentials are never stored |
| NFR11 | Compatibility | Essential customer functions work on the two latest versions of Chrome, Edge and Safari |
| NFR12 | Scalability | With 500 concurrent users NFR01 still holds and 30 checkouts per minute complete without loss |

The NFRs are written so that each can fail. "The system should be secure" cannot fail and therefore cannot be tested; "a replay never duplicates a transaction" (NFR07) can, and Chapter 5 shows the test that tries exactly that. Writing them this way early was the most useful single decision of Assignment 02, because it turned quality wishes into design constraints.

## 2.4. Prioritisation and Assumptions

*Table 12. MoSCoW baseline*

| Class | Requirements | Decision rule |
|---|---|---|
| Must | FR01-02, FR04-12, FR15, FR18-21, FR23; NFR01-02, NFR04-07, NFR09-10 | Needed for a secure, traceable sales lifecycle |
| Should | FR03, FR13, FR16-17, FR22, FR24; NFR03, NFR08, NFR11 | Important; a temporary manual process can bridge Release 1 |
| Could | FR14; NFR12 | Valuable once the transaction path is stable |
| Won't (Release 1) | Cash on delivery, guest checkout, native apps, marketplace integration | Outside the first release or unresolved |

The baseline rests on assumptions that only the shop can confirm: authenticated checkout, online payment only, stock reserved when the order is created, controlled order states, uniquely referenced provider messages and reviews limited to delivered order lines. They are listed because each one changes the design if it is false; for example, cash on delivery would

add an order state between *Shipped* and *Paid*.

## 2.5. Actors and the Use Case Model

*Table 13. Actors*

| Actor | Type | Use cases |
|---|---|---|
| Visitor | External human | UC01 Register, UC04 Browse, UC05 Search, UC06 View Product Details |
| Customer (specialises Visitor) | External human | UC02-03, UC07-08, UC10-13 |
| Staff (abstract) | Generalisation | UC02 Login, shared by the four staff roles |
| Administrator | Internal human | UC14-18 catalogue, categories, promotions, customers, reviews |
| Sales Staff | Internal human | UC17 Manage Customers, UC19 Process Orders |
| Warehouse Staff | Internal human | UC20-22, UC24 inventory, shipment, replenishment |
| Manager | Internal human | UC25-28 dashboard and reports |
| Payment Provider | External system | UC09 Make Payment |
| Shipping Provider | External system | UC22 Arrange Delivery, UC23 Update Shipment Status |
| Supplier | External organisation | UC24 Manage Replenishment |

The general use case diagram gives the overview on one page: every actor and the main use case of each package, with Customer as a specialisation of Visitor and the include relationship between checkout and payment. The four detailed diagrams that follow give the complete model, package by package.

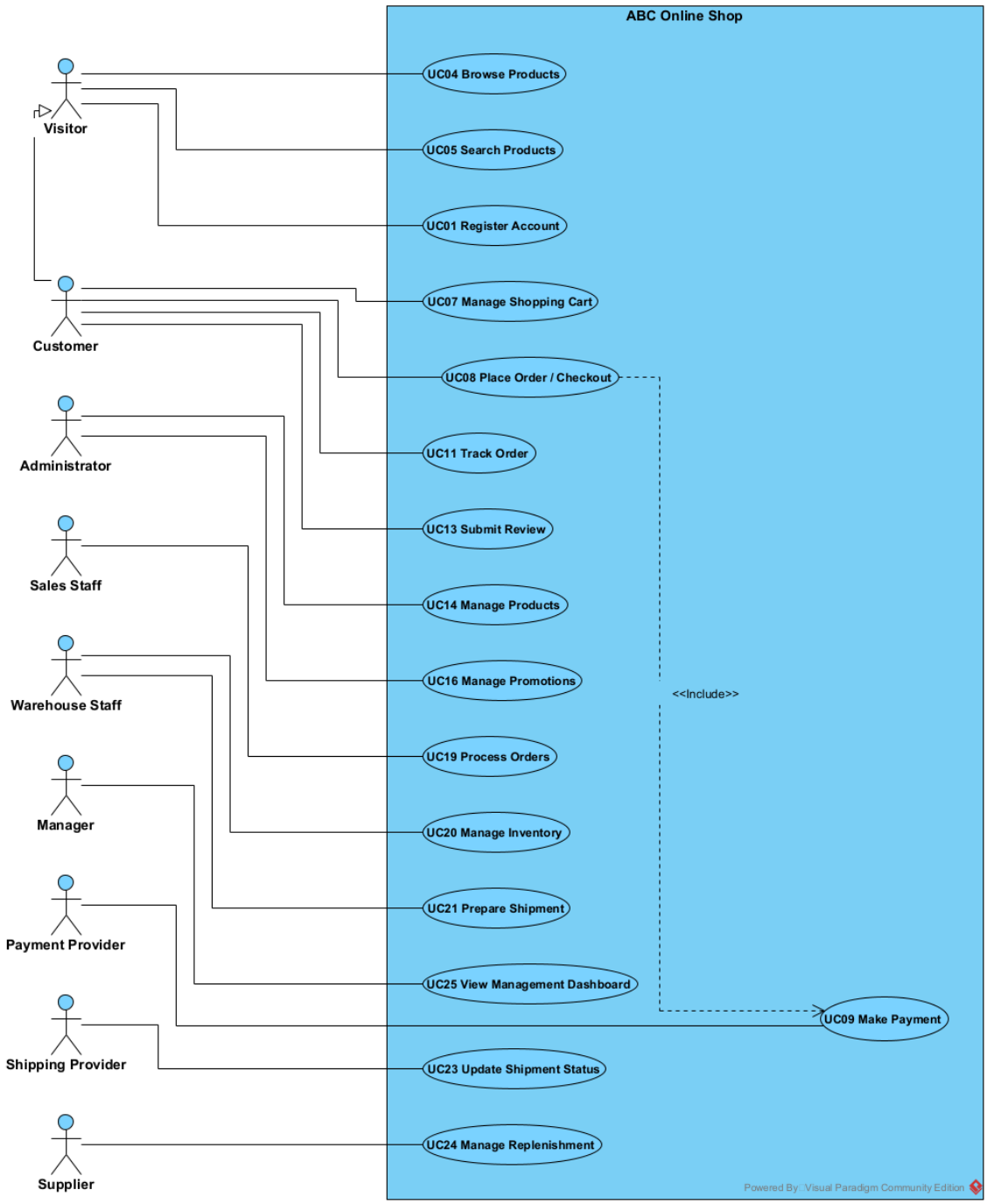



*Figure 5. General use case diagram: all actors and the main use case of each package - VP diagram "UC00 General Use Case Diagram"*

Drawing 10 actors and 32 use cases in one picture produces a diagram in which every line crosses several others. The model is therefore shown in four diagrams, one per package of the use case catalogue of Assignment 02. Every use case appears in exactly one diagram; an actor appears in each diagram where it has an association.

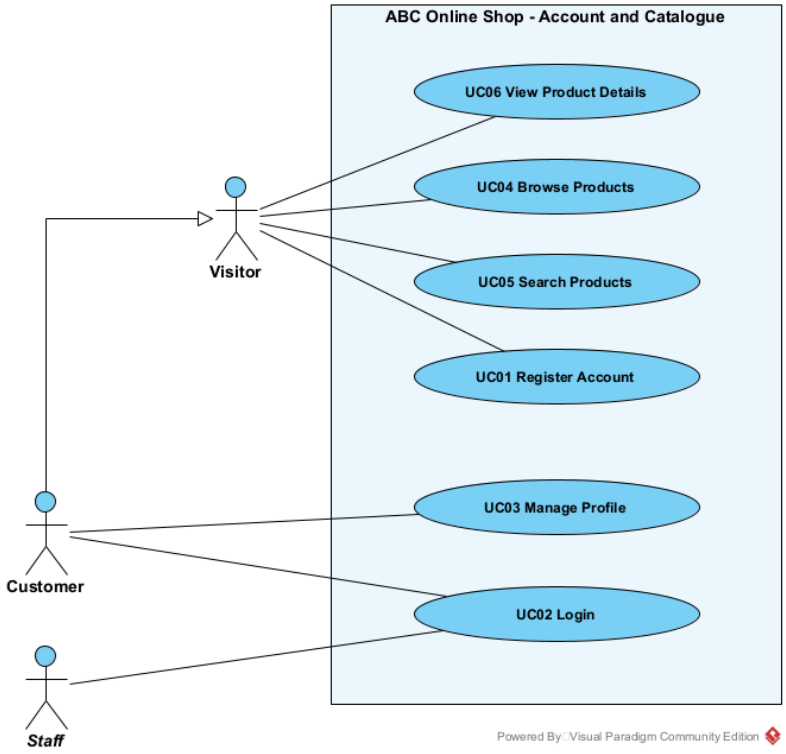



*Figure 6. Use cases of the Account and Catalogue package - VP diagram "UCD1 Account and Catalogue"*

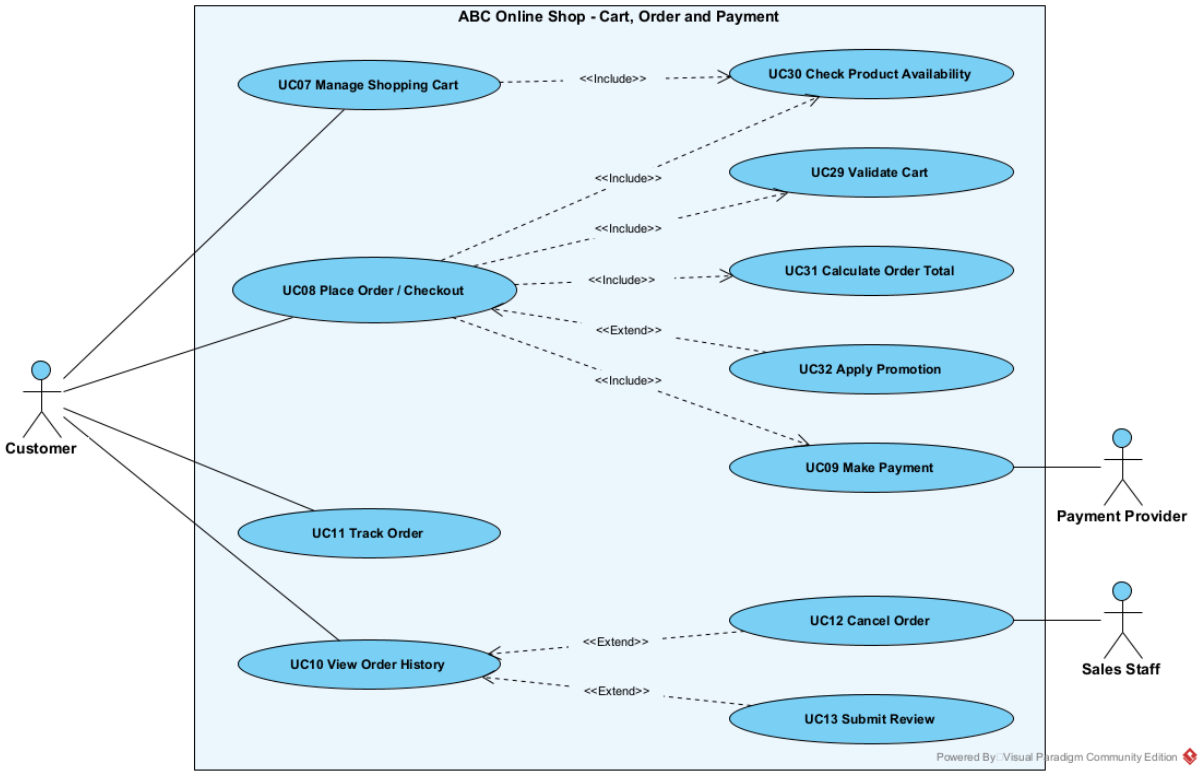



*Figure 7. Use cases of the Cart, Order and Payment package - VP diagram "UCD2 Cart, Order and Payment"*

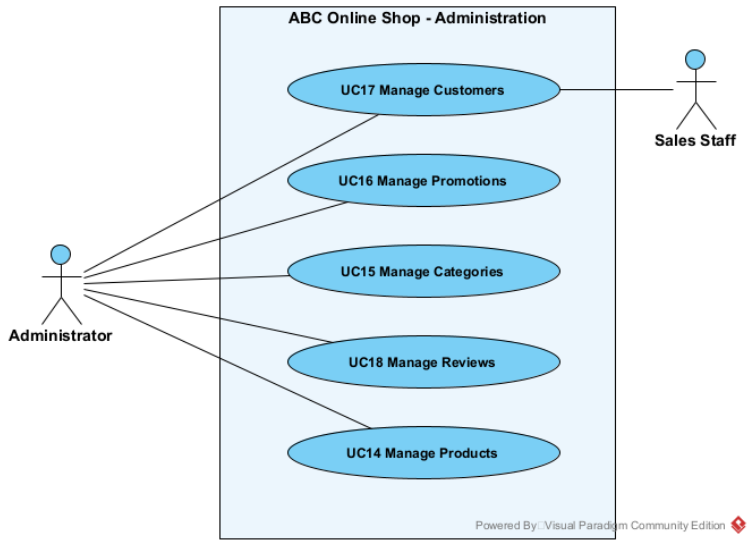



*Figure 8. Use cases of the Administration package - VP diagram "UCD3 Administration"*

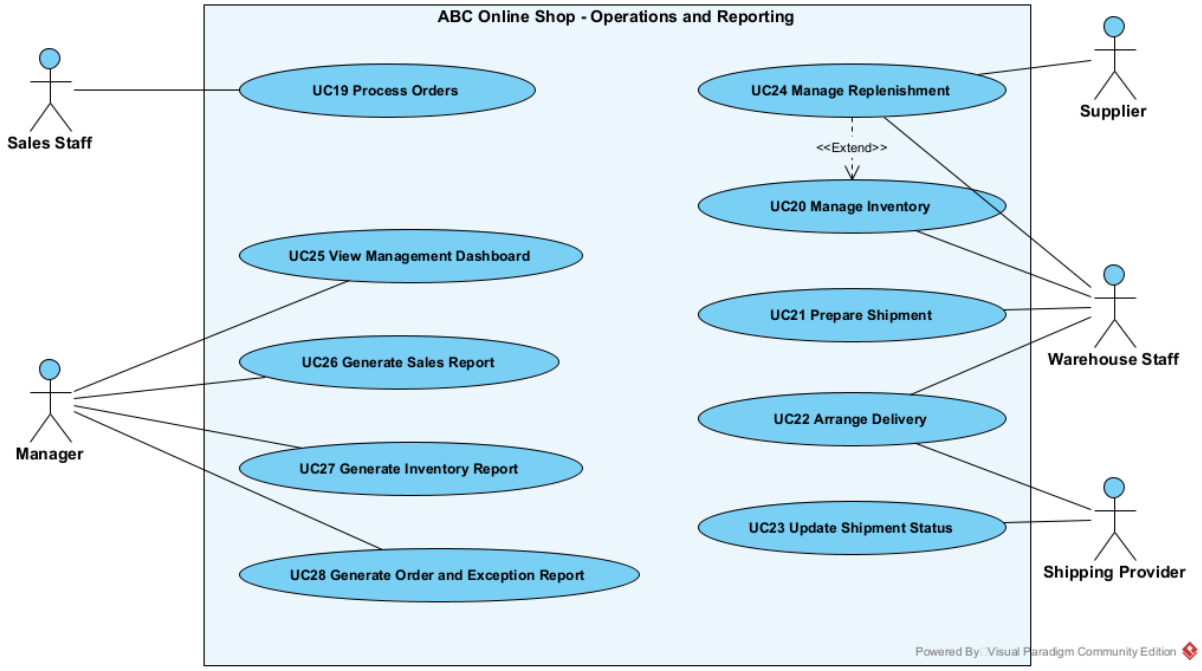



*Figure 9. Use cases of the Operations and Reporting package - VP diagram "UCD4 Operations and Reporting"*

The relationships follow the rules of the lecture. UC08 Place Order *includes* UC29 Validate Cart, UC30 Check Product Availability, UC31 Calculate Order Total and UC09 Make Payment, because under the Release 1 assumptions these always happen during checkout; UC07 includes UC30 because every cart change must check the orderable quantity. UC32 Apply Promotion *extends* UC08 only when the customer enters a code; UC12 Cancel Order and UC13 Submit Review extend UC10 View Order History at the point where an eligible order is selected; UC24 Manage Replenishment extends UC20 when stock reaches its threshold. The abstract Staff actor carries the single Login association so that the four staff roles keep separate, least-privilege use cases.

## 2.6. Use Case Specification: UC08 Place Order

*Table 14. Specification of UC08 Place Order / Checkout*

| Field | Specification |
|---|---|
| Primary actor / supporting | Customer / Payment Provider through UC09 |
| Goal | Convert a valid cart into one paid, confirmed, traceable order |
| Preconditions | Customer authenticated; cart contains at least one active product |
| Success guarantee | One order with validated lines, address, total, payment reference, reservations and a Paid/Confirmed history entry; cart cleared |
| Minimal guarantee | No duplicate order or charge; stock released if payment fails; cart kept |
| Main flow | 1 Customer selects Checkout. 2 System validates the cart (UC29) and availability (UC30). 3 Customer confirms the delivery address. 4 Optional promotion code (UC32). 5 System calculates the total (UC31). 6 Customer confirms; system reserves stock and creates one Payment Pending order with an idempotency key. 7 System requests payment (UC09). 8 Verified success: order Paid/Confirmed, cart cleared, confirmation shown. |
| Alternative flows | A1 inactive line, A2 insufficient stock: return to cart, nothing created. A3 invalid address. A4 invalid promotion: continue without discount. A5 payment declined: Payment Failed, reservations released, cart kept. A6 timeout: stay Payment Pending, reconcile by reference. A7 repeated submission: return the existing order. |
| Exception | E1 reservation or order creation fails: roll back both, preserve the cart |
| Requirements | FR08-FR10, FR24; NFR02, NFR05-NFR07, NFR10 |

UC08 was chosen for full specification because it touches more requirements than any other use case and because its alternative flows are where the shop currently loses money. Chapter 5 implements each flow of this table and tests A2, A5, A6, A7 and E1 individually.

## 2.7. Validation and Traceability

*Table 15. Traceability from business problems to use cases*

| Problem | Needs | Requirements | Use cases |
|---|---|---|---|
| BP01 product inconsistency | SN01, SN06 | FR04-06, FR15, NFR06 | UC04-06, UC14-15 |
| BP02 outdated inventory | SN08-09 | FR08, FR19, FR22, NFR07 | UC20, UC24, UC30 |
| BP03 manual order processing | SN02-03, SN07 | FR07-10, FR18 | UC07-09, UC19, UC29-31 |
| BP04 unclear status | SN04, SN14 | FR11-12, FR21 | UC10-11, UC22-23 |
| BP05 slow reporting | SN10 | FR23 | UC25-28 |
| BP06 fragmented customer data | SN12 | FR01-03, FR17, NFR05, NFR10 | UC01-03, UC17 |
| BP07 limited real-time information | SN11 | FR23, NFR01 | UC25-28 |

The requirements were validated against five properties. **Clear**: every statement has an actor, an action, an object and a condition. **Consistent**: order states, provider references and reservation rules use the same terms everywhere. **Complete**: the matrix has no orphan problem, need, requirement or primary use case; the supporting UC29-UC32 are reached

through include and extend. **Feasible**: provider-internal work is outside the boundary. **Verifiable**: every FR has an observable outcome and every NFR a threshold.

## 2.8. Requirements for the AI Extensions

The AI functions of Chapter 3 are specified as Release 2 requirements in the same form, so that they can be traced and tested like the rest. They extend existing use cases instead of replacing them, and each has a non-AI fallback.

*Table 16. AI requirements*

| ID | Requirement | Extends | Source |
|---|---|---|---|
| AI-R1 | Search products by keywords, by a spoken sentence, by a photo, or by text and photo together | UC05 | A6 |
| AI-R2 | Answer order questions ("where is my latest order?") only from the customer's own orders | UC10-11 | A6, FR11 |
| AI-R3 | Rank results by text match, image similarity and stock, and show the component scores | UC05 | A6 |
| AI-R4 | Summarise published reviews per aspect (quality, price, delivery, service) with evidence | UC06 (UC33) | A02 AI part |
| AI-R5 | Hold reviews that look like spam for moderation instead of publishing them | UC13, UC18 | A02 AI part |
| AI-R6 | Recommend similar in-stock products and products for a customer's purchase history | UC06, UC10 (UC34) | new |
| AI-N1 | Every AI component is replaceable behind a fixed interface (transcribe, encode, analyse, similar) | - | NFR09 |
| AI-N2 | Results are deterministic and explainable: same input, same output, visible scores | - | NFR06 |
| AI-N3 | If an AI component fails or lacks data, the page still works with a non-AI fallback | - | NFR03 |

## 2.9. Further Use Case Specifications

UC08 depends on two use cases that carry most of the integrity rules: UC09 Make Payment and UC20 Manage Inventory. Their specifications, condensed from Assignment 02, are the source of the payment and stock code in Chapter 5.

*Table 17. Specification of UC09 Make Payment*

| Field | Specification |
|---|---|
| Actors | Customer (primary), Payment Provider (supporting); triggered by UC08 for a Payment Pending order |
| Goal | Record an authoritative online-payment outcome without raw credentials or duplicates |
| Main flow | 1 System creates a payment attempt with a unique reference. 2 System sends reference, amount and currency to the provider. 3 Customer authorises at the provider. 4 Provider returns a verifiable result. 5 System authenticates the source and matches reference, amount and currency. 6 System records success once and changes the order to Paid/Confirmed. |
| Alternative flows | A1 declined: Payment Failed, reservations released, cart kept. A2 timeout: Pending, reconcile later, never "pay again". A3 duplicate callback: acknowledge, no second effect. A4 late success on a closed order: refund exception. |
| Exceptions | E1 reference, amount or currency mismatch: quarantine, order unchanged. E2 unauthenticated result: reject and alert. |
| Business rules | Store status and references, never card data; a retry creates a new attempt reference; provider retries are idempotent |

*Table 18. Specification of UC20 Manage Inventory*

| Field | Specification |
|---|---|
| Actors | Warehouse Staff (primary); UC08, UC09, UC12 and UC21 create events automatically; Supplier through UC24 |
| Goal | Maintain current, explainable stock through traceable events |
| Main flow | 1 Staff selects a product and an event type; system shows physical, reserved, available and recent events. 2 Staff enters quantity, reference and reason. 3 System validates role, product, quantity, unique reference and resulting balance. 4 System records an immutable event and recalculates the balance atomically. 5 At or below threshold, system offers UC24. 6 System shows the new balance and the event id. |
| Alternative flows | A1 reservation, A2 issue, A3 release (order-driven); A4 correction recorded as a reason-coded ADJUST; A5 threshold reached |
| Exceptions | E1 negative resulting balance: reject atomically. E2 duplicate reference: return the earlier event, no change. |
| Business rules | available = physical - reserved; all quantities >= 0; an issue or release never exceeds its reservation |

## 2.10. Acceptance Criteria

For the requirements with the highest risk, acceptance criteria were written in the Given-When-Then form so that they can become tests without reinterpretation. Chapter 5 shows that each of them became exactly one test.

*Table 19. Acceptance criteria of key requirements*

| Req. | Given | When | Then |
|---|---|---|---|
| FR09 / A7 | a cart with one product and checkout key k-1 | the customer submits checkout twice with k-1 | one order exists and one payment attempt was made |
| FR10 / NFR07 | a pending payment attempt | the provider sends a forged result, a wrong amount, then the correct result twice | the order is Paid/Confirmed exactly once |
| FR11 / NFR05 | order O002 belongs to customer C002 | customer C001 asks for O002 | the answer is "not found"; C002 sees the order |
| FR19 | P05 with 10 units received under reference GRN-77 | the same receipt is recorded again, then an adjustment of -20 | stock stays 10; the adjustment is rejected |
| FR21 | a shipment with milestones 1 and 2 | milestone 2 arrives again and milestone 1 arrives late | both are ignored and the status does not move back |

## 2.11. Managing Requirement Change

Requirements changed three times after Assignment 02, and each change was recorded with its reason instead of silently editing the baseline. (1) The AI requirements AI-R1 to AI-N3 were added as Release 2 extensions, so the 32 Release 1 use cases stayed unchanged. (2) The order state machine gained the transition Delivery Failed → Delivered when implementing UC23 showed that a carrier can deliver after a failed attempt without a new pick-up message. (3) The definition of a purchase (used by reviews and recommendations) was made explicit: an order counts only if it was not cancelled and its payment did not fail. Each change was propagated through the traceability matrix to the diagrams and tests that depend on it.

## 2.12. From Requirements to Analysis Models

*Table 20. Analysis questions, models and the requirements they refine*

| Question | Model (VP diagram) | Refines |
|---|---|---|
| Who does what, in which order? | Activity and swimlane diagrams (AD01-AD03) | UC08, UC19-23, UC20 |
| Which data flows between processes and stores? | DFD level 0 and level 1 (DFD0, DFD1) | Context diagram CTX01 |
| Which objects collaborate in a scenario? | Sequence diagrams with BCE lifelines (SD01, SD02) | UC08, UC11 |
| Which concepts and relationships exist? | Analysis class diagram (CL01) | Information catalogue |
| Which states can an order be in? | State machine (ST01) | FR12, FR13, FR18 |

## 2.13. Activity Modeling

An activity diagram shows the control flow of one use case; a swimlane adds who performs each action. The swimlane version is the more useful one for this shop, because most of its problems are hand-offs between parties. The checkout diagram has three lanes: Customer, System and Payment Provider. It makes two facts visible that the text of UC08 only implies. First, the system reserves stock *before* it asks for payment, so that a paid order can always be fulfilled. Second, the payment result is a decision of the system, not of the provider: the provider returns a signed result and the system decides whether it is a verified success.

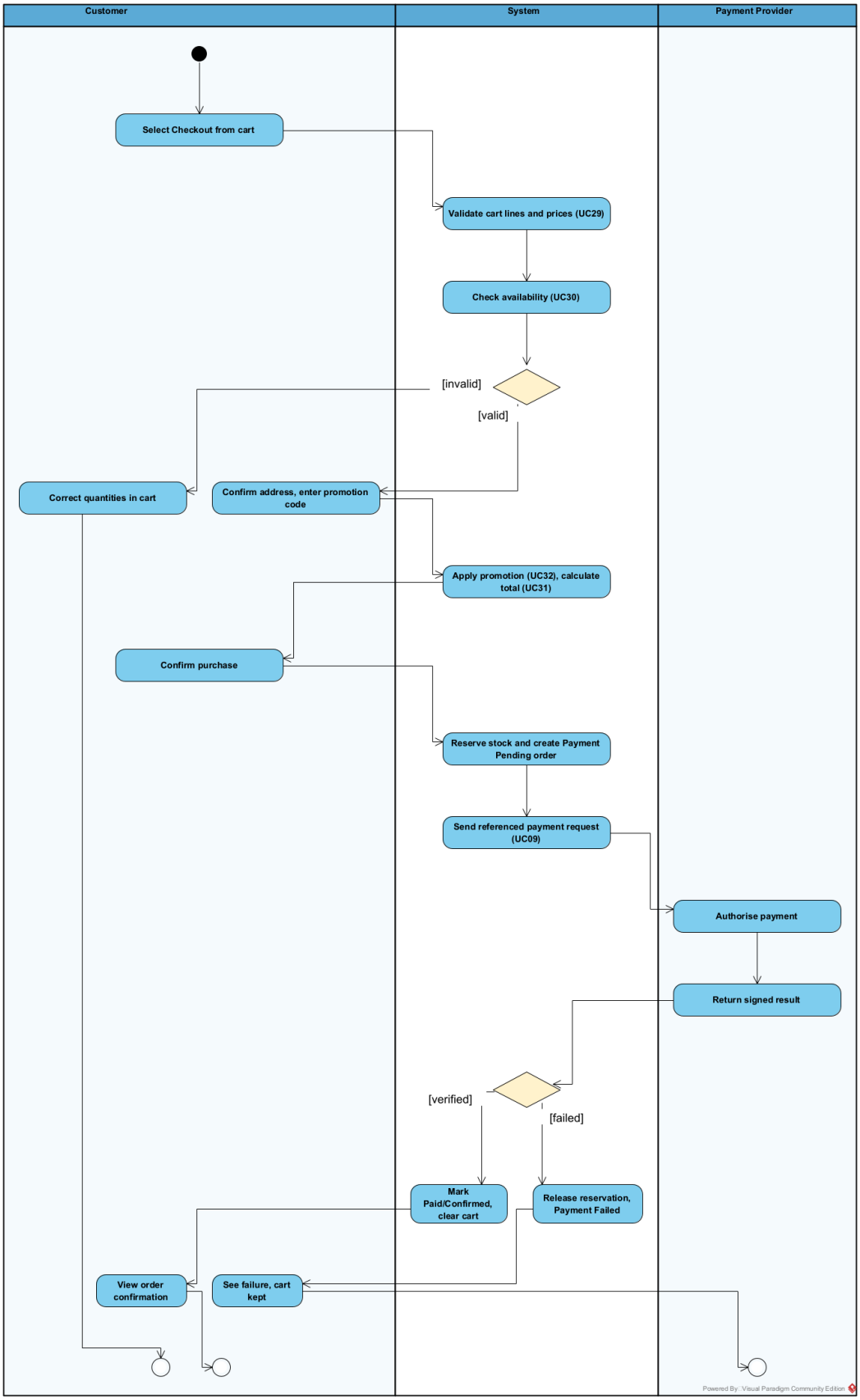



*Figure 10. Place order activity with swimlanes - VP diagram "AD01 Place Order (Swimlane)"*

The diagram has two decision nodes; the question of each decision is written into the guards of its outgoing flows ([valid]/[invalid], [verified]/[failed]), so every flow carries exactly one condition. The failure branches end in their own final nodes rather than looping back, because

the use case specification says that after a validation failure the customer returns to the cart, which is a different use case (UC07). On the [failed] branch after payment, the system releases the reservation and records Payment Failed before the customer sees the failure, which is the order of the statements in PaymentService.handle_result (Chapter 5).

Fulfilment crosses four parties, which is exactly where Assignment 01 located the delays of BP03 and BP04. The second swimlane diagram follows a paid order from the processing queue of the sales staff to the milestones of the carrier. The eligibility decision checks two facts that come from earlier use cases: a verified payment (UC09) and complete reservations (UC08 step 6). An order that fails either check is put On Hold with a recorded reason instead of being silently skipped.

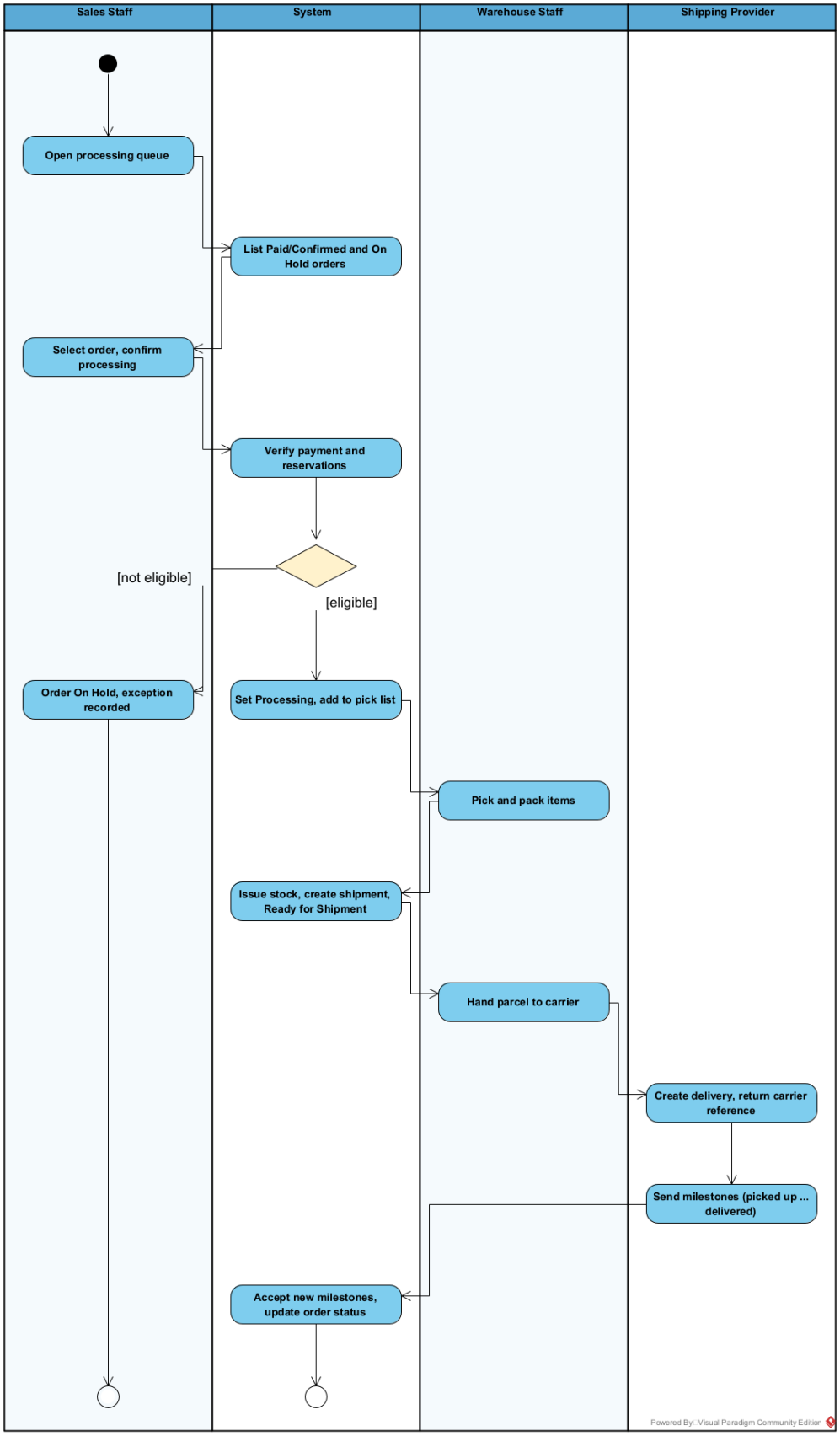



*Figure 11. Order processing and shipment - VP diagram "AD02 Process and Ship Order (Swimlane)"*

Inventory management is a single-actor use case and is drawn without lanes. Its interest lies in the order of the checks: the duplicate-reference check comes first, so that a repeated goods-receipt note can never add stock twice (NFR07); the balance check comes second, so

that no event can produce a negative quantity; and the threshold check comes last, after the event is recorded, and only offers replenishment (UC24) instead of starting it, because a person decides how much to order.

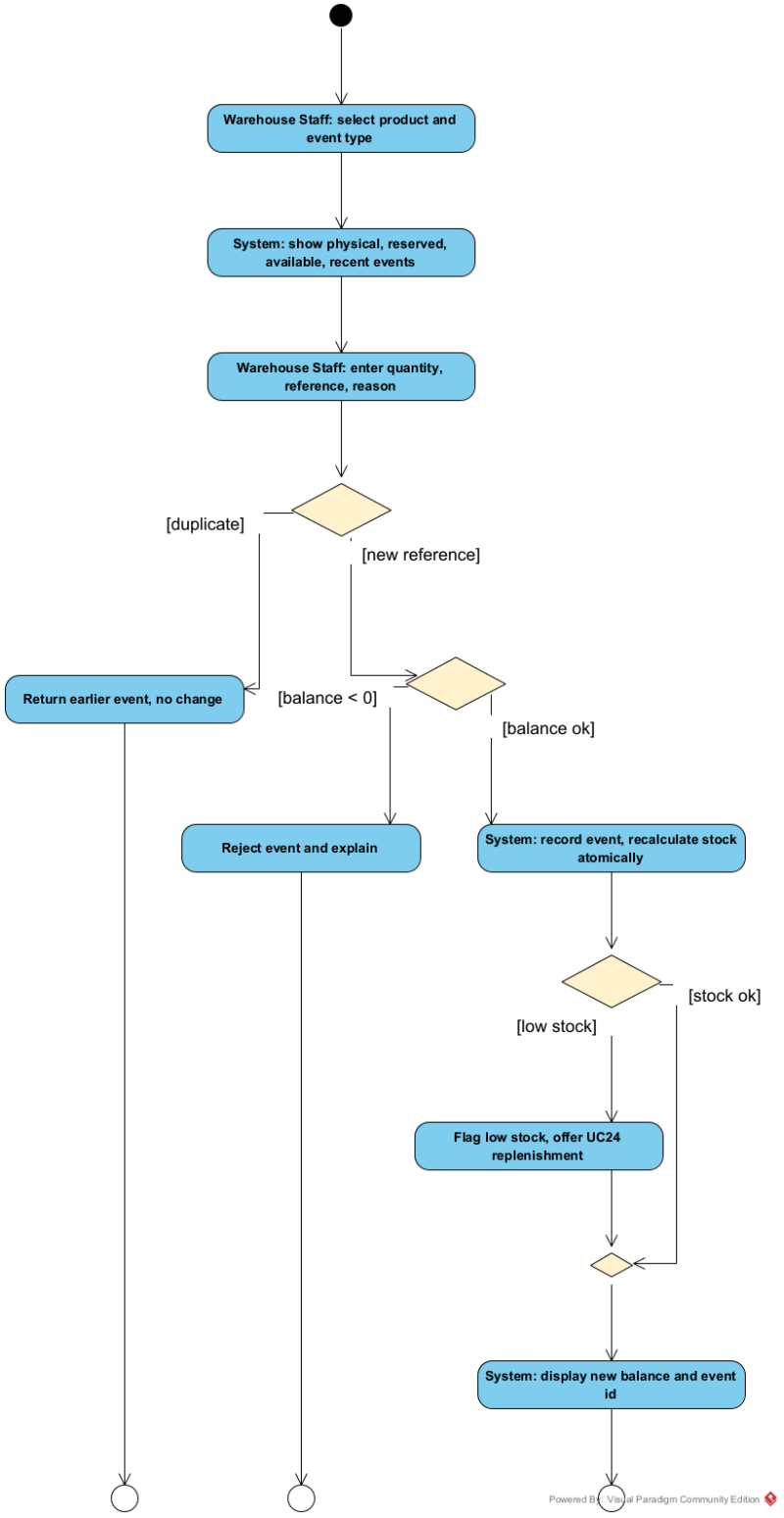



*Figure 12. Manage inventory - VP diagram "AD03 Manage Inventory"*

## 2.14. Data Flow Modeling

The context diagram of Chapter 1 treats the system as one process. The level 0 DFD opens it into eight processes and six data stores. The decomposition follows the business process of Section 1.5, so each process has one clear owner of its data. Two rules of DFD notation were checked on every flow: a data store is never connected directly to another store or to an external entity, and every process has at least one input and one output. A flow is named after the data it carries, never after an action (*payment request*, not *send payment*).

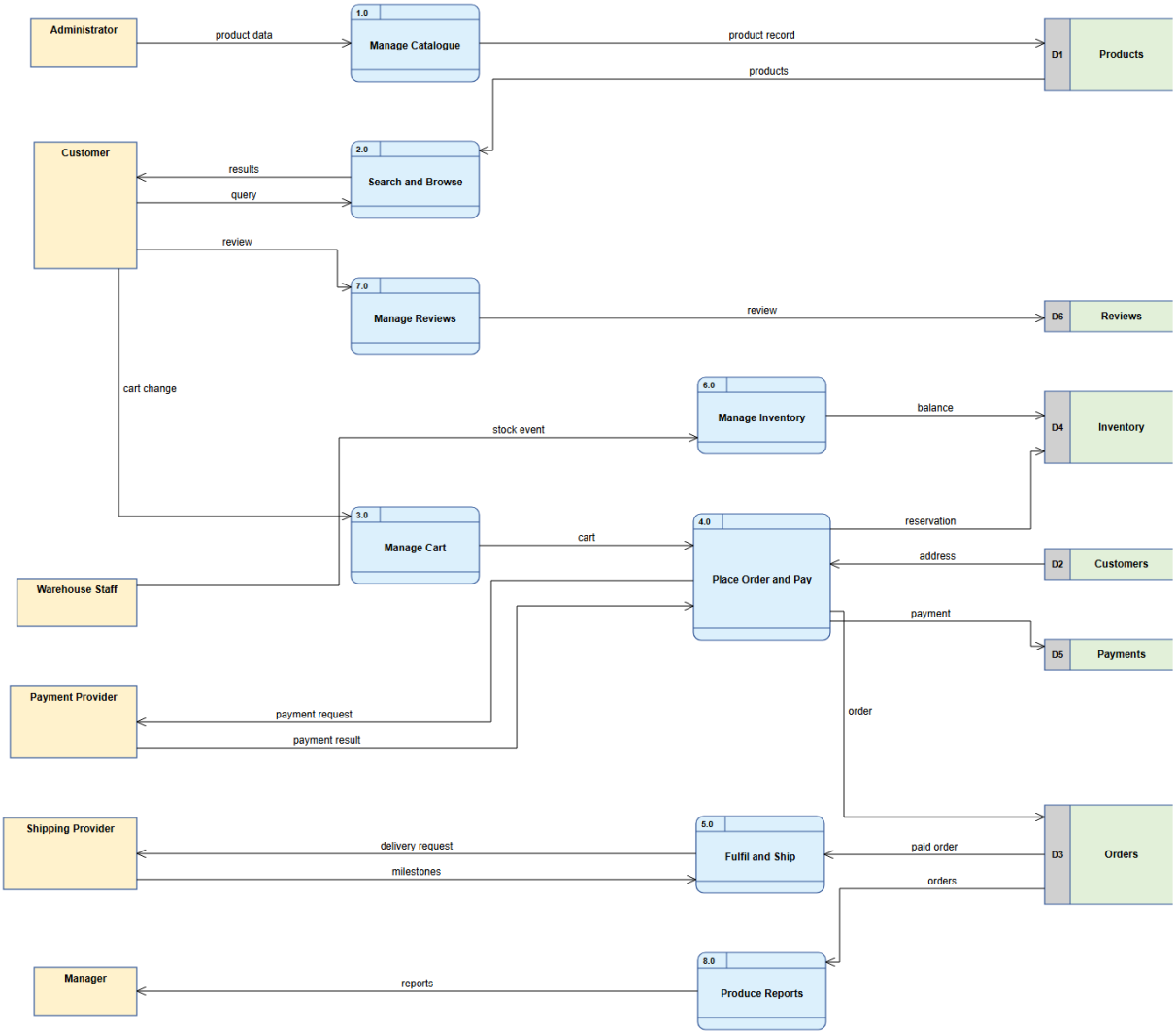



*Figure 13. Level 0 data flow diagram - VP diagram "DFD0 Level 0 Diagram"*

The level 0 diagram also shows where the shop's information becomes inconsistent today. The Products store D1 is read by process 2.0 (search) and by checkout, and written only by 1.0; the Inventory store D4 is written by 4.0 (reservations) and 6.0 (receipts) and read by everyone. With one writer per kind of change, two copies of the same fact cannot exist, which is the analysis form of objective O1.

Process 4.0 Place Order and Pay is the most complex one and is decomposed at level 1 into five processes. The decomposition is balanced: the level 1 diagram has the same external flows as process 4.0 on level 0 (checkout request and confirmation with the customer, payment request and result with the provider, order, reservation and payment written to D3, D4 and

D5). Two stores appear only at level 1, Carts and Promotions, because they are internal to the checkout.

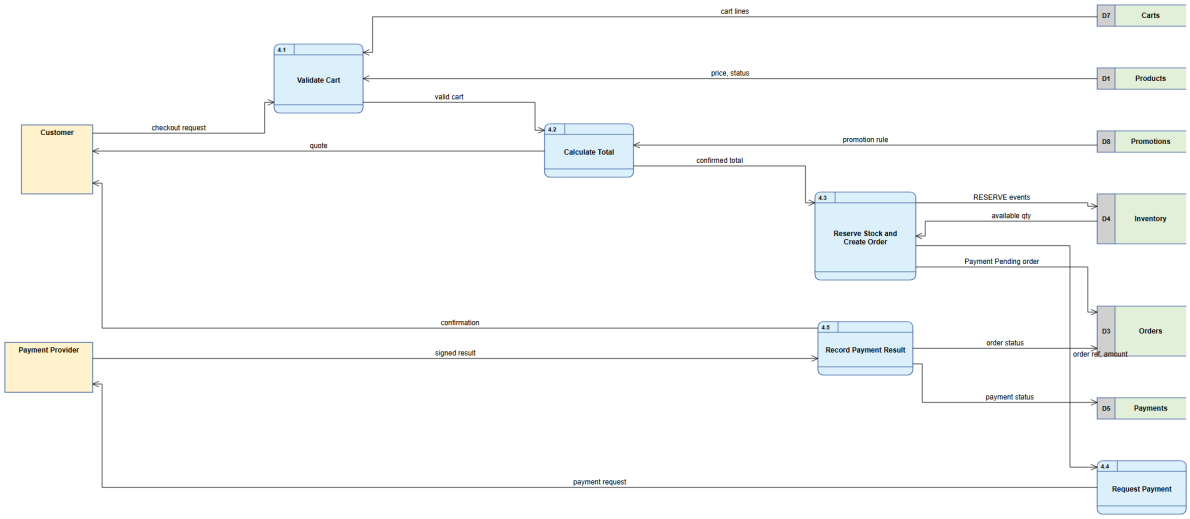



*Figure 14. Level 1 DFD of process 4.0 - VP diagram "DFD1 Level 1 - 4.0 Place Order and Pay"*

## 2.15. Interaction Modeling

A sequence diagram assigns the steps of a scenario to objects. In analysis the objects are typed with the boundary, control and entity stereotypes: a **boundary** object talks to an actor or to an external system (CheckoutForm, PaymentGateway), a **control** object coordinates one use case (CheckoutController), and an **entity** object holds information that outlives the scenario (Cart, StockLevel, Order). The rule that an actor only talks to boundaries, and entities only answer, keeps the scenario readable and already suggests the layers of the design.

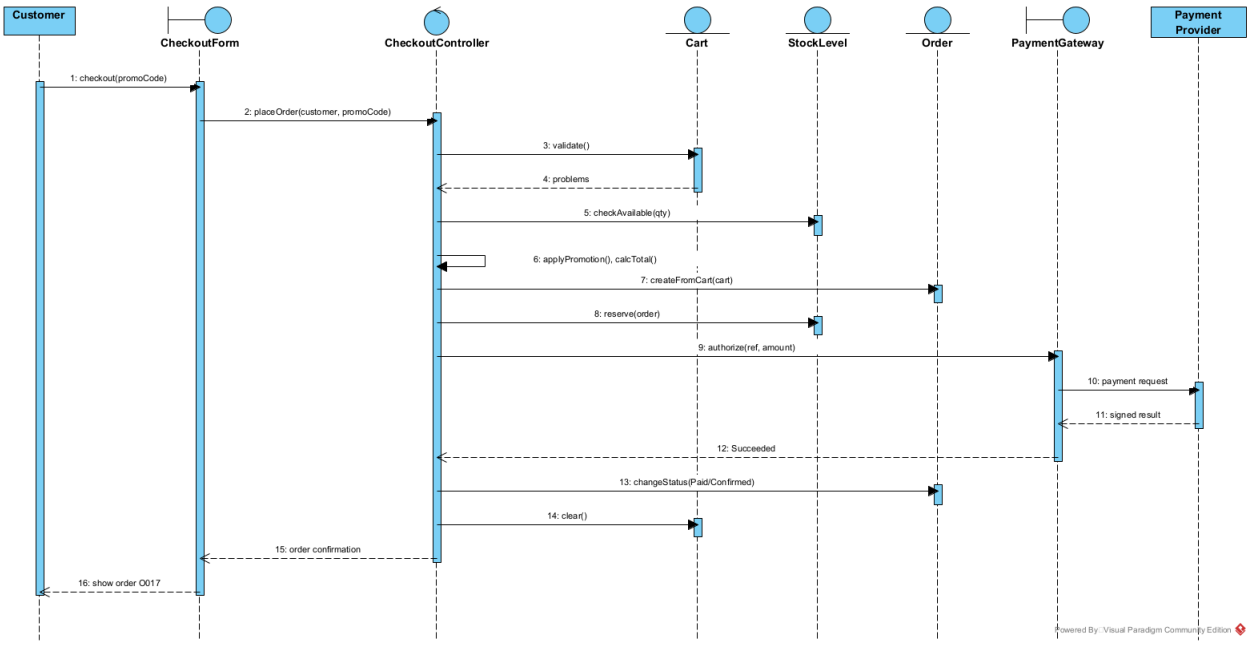



*Figure 15. Sequence diagram of the main checkout scenario - VP diagram "SD01 Place Order (Analysis)"*

The sixteen messages follow the main flow of UC08 in order. Messages 3-6 are the included use cases UC29-UC31; message 8 reserves stock before payment (step 6); messages 9-12 are

UC09, where the PaymentGateway boundary is the only object that knows the provider. Message 13 changes the order status only after message 12 has reported a verified success. An alternative flow such as A5 (payment declined) is a different scenario and is therefore not drawn into this diagram; it is covered by the activity diagram and by a test.

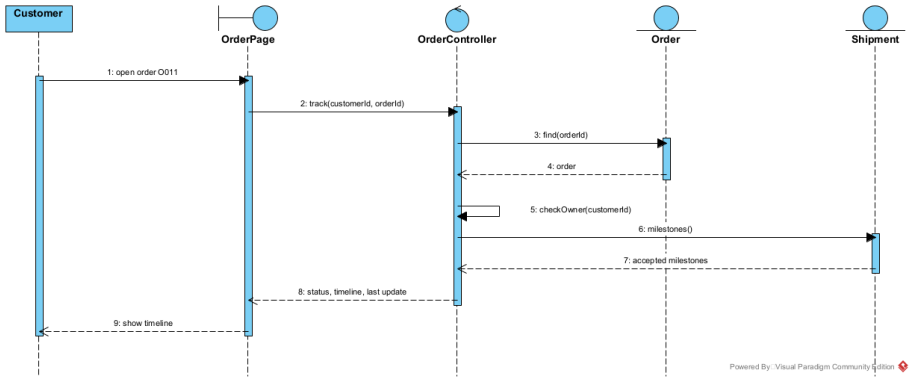



*Figure 16. Sequence diagram of order tracking - VP diagram "SD02 Track Order (Analysis)"*

Tracking looks simple but contains the privacy rule of NFR05: message 5, checkOwner(customerId), happens before any shipment data is read. If the order belongs to someone else, the controller answers exactly as if the order did not exist, so that a customer cannot learn which order numbers are in use (UC11 E2).

## 2.16. Analysis Class Model

The classes were found by the noun-phrase method applied to the use case specifications and the information catalogue, then filtered: nouns that name an actor (Manager), a screen (checkout page) or a single attribute (price) were removed. Eighteen classes remained, and the product detail hierarchy added five more (Section 2.16). They carry attributes but no operations yet; operations are added in the first design step (Section 4.3).

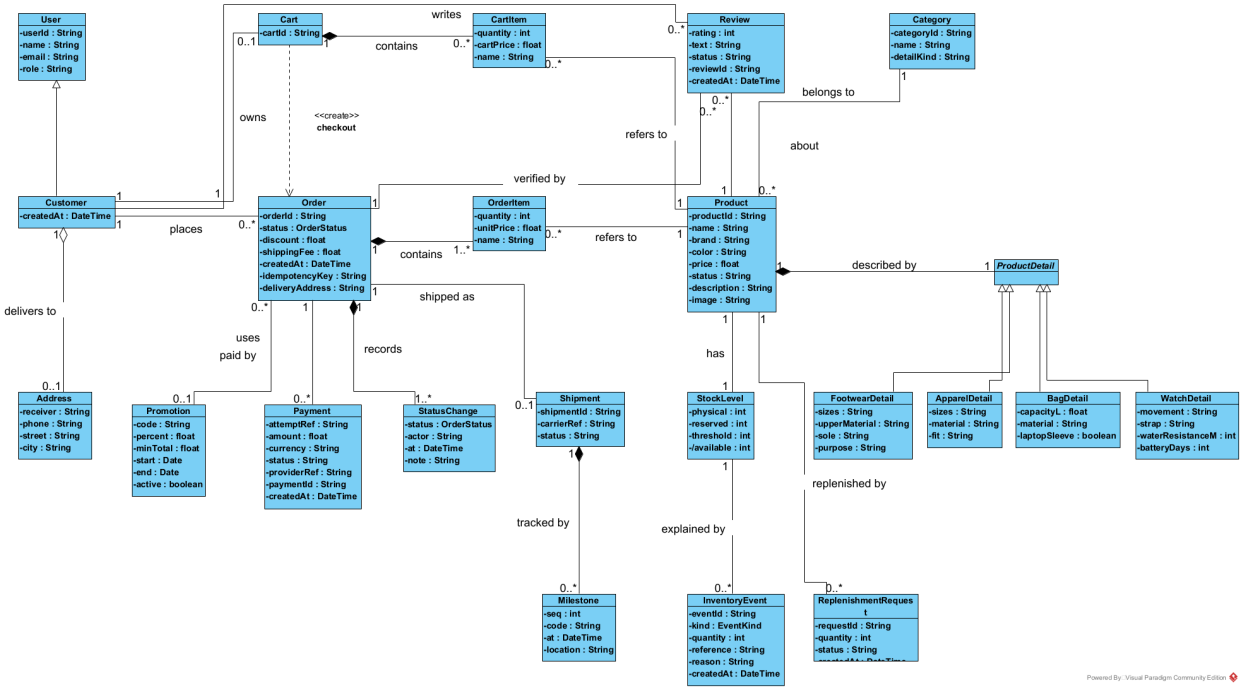



*Figure 17. Analysis class diagram - VP diagram "CL01 Analysis Class Diagram"*

Following the lecturer's remark on the first version, the catalogue part of the class model no longer keeps every attribute of every product kind in one Product class. Product holds what all products share: name, brand, colour, price, description, image and status. What only one group of products has lives in a detail class: sizes, upper material, sole and purpose for footwear; sizes, material and fit for apparel; capacity, material and laptop sleeve for bags; movement, strap, water resistance and battery life for watches. ProductDetail is abstract, so a product is always described by exactly one concrete kind, and Category.detailKind says which kind its products use. A new product group means a new detail class and a new category, not a new column that is empty for all other products.

*Table 21. Relationships of the analysis model and their justification*

| Relationship | Kind | Justification |
|---|---|---|
| Customer - User | Generalisation | A customer is a user with an address and order history; staff are users with other roles |
| Customer - Address | Aggregation 1 - 0..1 | The address exists for delivery and can be replaced; old orders keep a copy of it |
| Cart - CartItem | Composition 1 - 0..* | A cart line has no meaning outside its cart and is deleted with it |
| Order - OrderItem | Composition 1 - 1..* | An order has at least one line; lines are fixed after creation |
| Order - StatusChange | Composition 1 - 1..* | The append-only history that explains the current status (BP04) |
| Order - Payment | Association 1 - 0..* | Several attempts are possible (A5 retry); at most one succeeds |
| Order - Shipment | Association 1 - 0..1 | A shipment exists only after packing |
| Shipment - Milestone | Composition 1 - 0..* | Milestones are carrier messages about one shipment |
| Product - StockLevel | Association 1 - 1 | Stock is kept apart from the catalogue so that two roles do not edit one record |
| StockLevel - InventoryEvent | Association 1 - 0..* | Every quantity is explained by events (FR19) |
| Review - Order, Product | Association | A review is verified by a delivered order of that product (FR14) |

Two modelling decisions deserve a comment. CartItem stores its own *cartPrice* and OrderItem its own *unitPrice* instead of reading Product.price, because the price at the time of adding and the price paid are historical facts; if the administrator changes a price, old orders must not change. And StockLevel is a separate class rather than an attribute of Product: in Assignment 01 the catalogue and the stock were maintained by different people, and putting both in one class would recreate the conflict of BP01 and BP02. The derived attribute */available* (physical minus reserved) is marked with a slash because it is never stored.

## 2.17. Order State Machine

BP04, unclear order status, is in the end a missing state model. The state machine makes the allowed transitions explicit; any transition that is not drawn is forbidden. Each transition is labelled with *event [guard] / action*, so the diagram also says when stock is reserved, released or issued.

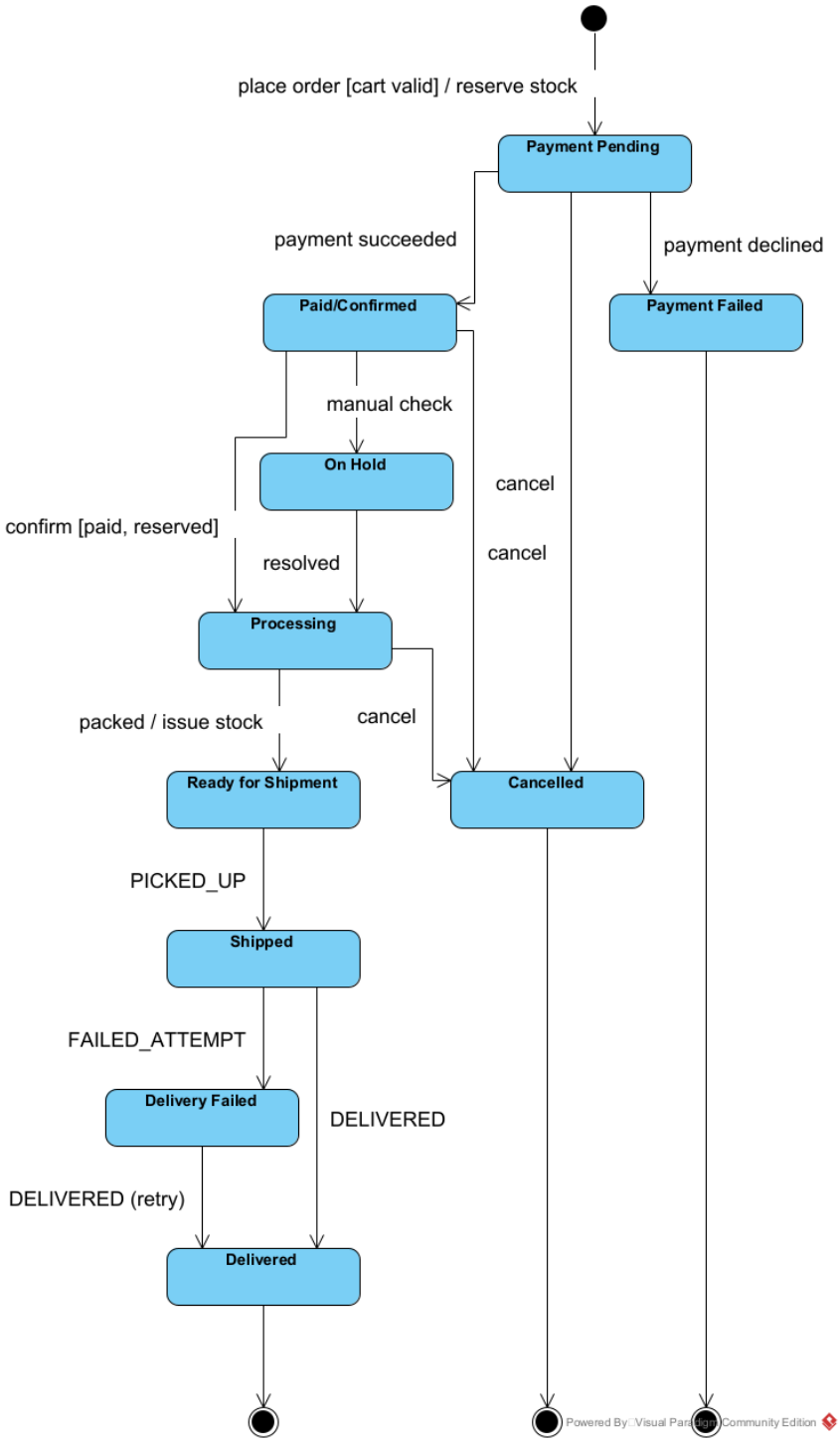



*Figure 18. Order state machine - VP diagram "ST01 Order State Machine"*

The state machine resolves several questions that the use cases left open. A cancellation is possible until packing (Payment Pending, Paid/Confirmed, On Hold, Processing) and impossible afterwards, which makes FR13's "eligible order" precise. Payment Failed is a final

state: a new attempt creates a new order instead of reviving the old one. An order On Hold returns to Processing once the reason is resolved, and a delivery that fails once can still end in Delivered when the carrier's next attempt succeeds. The same table of transitions appears in the code as a dictionary, and a model test checks that a skipped state is rejected.

## 2.18. Business Rules and Decision Logic

The decision nodes of the activity diagrams hide business rules. Writing them as decision tables makes sure that every combination of conditions has exactly one outcome, which is what the code must implement.

*Table 22. Decision table: may an order enter Processing? (UC19)*

| Rule | Payment verified | All lines reserved | Blocking exception | Outcome |
|---|---|---|---|---|
| R1 | Yes | Yes | No | Processing; added to the pick list |
| R2 | No | - | - | On Hold: "payment is not confirmed" |
| R3 | Yes | No | - | On Hold: "product is not reserved" |
| R4 | Yes | Yes | Yes | On Hold with the recorded reason |

*Table 23. Decision table: may an order be cancelled? (UC12)*

| Current status | Who | Outcome |
|---|---|---|
| Payment Pending | Owner or Sales Staff | Cancelled; reservations released |
| Paid/Confirmed, On Hold, Processing | Owner or Sales Staff | Cancelled; reservations released; refund initiated and audited |
| Ready for Shipment, Shipped, Delivered | anyone | Rejected: "can no longer be cancelled" |
| any status | another customer | "order not found" (no disclosure) |

*Table 24. Promotion rule (UC32)*

| Condition | Result |
|---|---|
| code unknown | no discount; message "promotion does not exist"; checkout continues (A4) |
| today outside [start, end] or subtotal < minimum | no discount; message "not valid for this order" |
| valid | discount = subtotal x percent / 100, rounded to cents; shipping calculated after discount |

## 2.19. Class Responsibilities

*Table 25. Analysis classes and their responsibilities*

| Class | Responsibility | Key rule |
|---|---|---|
| User / Customer | Identity, role, contact; customer owns address, cart and orders | Least privilege by role |
| Product / Category | What is sold and how it is grouped | Only active products are sold |
| Promotion | A discount rule with dates and minimum total | Valid only in its period |
| Cart / CartItem | Tentative choice with the price at the time of adding | Revalidated before ordering |
| Order / OrderItem | Commitment with authoritative prices and address | Lines fixed after creation |
| StatusChange | One entry of the append-only order history | Never updated or deleted |
| Payment | One attempt with its reference and result | At most one success per order |
| StockLevel / InventoryEvent | Current balance and the events that explain it | available = physical - reserved >= 0 |
| Shipment / Milestone | Parcel and carrier messages | Sequence numbers only increase |
| Review | Opinion linked to a delivered purchase | One per customer, product and order |

## 2.20. CRUD Matrix

The CRUD matrix crosses use cases with data entities and shows which use case creates (C), reads (R), updates (U) or deletes (D) which data. It is a completeness check: every entity must be created somewhere and read somewhere, and an entity updated by many use cases deserves a closer look at concurrency.

*Table 26. CRUD matrix (selected use cases)*

| Use case | Product | Stock / Event | Cart | Order / History | Payment | Shipment | Review |
|---|---|---|---|---|---|---|---|
| UC04-06 Browse, Search, View | R | R | | | | | R |
| UC07 Manage Cart | R | R | CRU | | | | |
| UC08 Place Order | R | RU / C | RU | C | | | |
| UC09 Make Payment | | U / C (release) | | U / C | CU | | |
| UC11 Track Order | | | | R | | R | |
| UC12 Cancel Order | | U / C | | U / C | R | | |
| UC13 Submit Review | | | | R | | | C |
| UC14 Manage Products | CRU | C | | | | | |
| UC18 Manage Reviews | | | | | | | RU |
| UC19 Process Orders | | R | | RU / C | R | | |

| Use case | Product | Stock / Event | Cart | Order / History | Payment | Shipment | Review |
|---|---|---|---|---|---|---|---|
| UC20 Manage Inventory | R | RU / C | | | | | |
| UC21-23 Shipment | | U / C (issue) | | U / C | | CRU | |
| UC25-28 Reports | R | R | | R | R | | |

Two observations from the matrix shaped the design. Stock is updated by six use cases, so stock changes are funnelled through one service method (InventoryService.record) inside a transaction. And no use case deletes an order, a payment or a stock event: corrections are new records, which is what makes every figure explainable.

## 2.21. Consistency Across Models

*Table 27. Cross-model consistency checks*

| Check | Result |
|---|---|
| Every lifeline of SD01/SD02 is a class of CL01 or a boundary/control of the use case | Passed (Cart, StockLevel, Order, Shipment are entities) |
| Every data store of DFD0 corresponds to classes of CL01 and tables of ERD01 | Passed (D1 Products = product, category; D4 Inventory = stock_level, inventory_event) |
| Every state of ST01 is a value of Order.status and every transition is reachable from a use case | Passed; Delivery Failed reached only from UC23 |
| Every message of SD01 corresponds to an action of AD01 | Passed; message 6 groups UC31 and UC32 |
| Multiplicities of CL01 match foreign keys of ERD01 | Passed; Customer-Cart 0..1 became UNIQUE(customer_id) |
| DFD1 is balanced with process 4.0 of DFD0 | Passed (same external flows) |

## 2.22. Chapter Summary

The baseline has 14 stakeholder needs, 24 functional and 12 non-functional requirements, 9 AI requirements and 32 use cases, shown in a general and four detailed use case diagrams and specified in full for the use cases that carry the business. The analysis refines them with four activity diagrams, two DFD levels, two sequence diagrams, the analysis class diagram with the product detail hierarchy and the order state machine, and the models agree on six cross-checks. Two rules that the design must keep are now precise: stock and status are explained by events, and every external exchange carries a unique reference.

# CHAPTER 3. AI IN E-COMMERCE

Artificial intelligence is useful to ABC Online Shop only where it improves a business activity that the earlier chapters already described. This chapter therefore does not start from techniques. It starts from the problems and use cases of Chapters 1 and 2, selects three places where AI helps, analyses each one as a requirement with data, components, risks and a fallback, and measures the result on test sets.

## 3.1. AI as an Enhancement Layer

In the lecture material on integrating AI in the analysis phase, AI is placed as an enhancement layer on top of the information system, not as a replacement for it. The shop still needs a correct catalogue, correct stock and correct orders; a search model that ranks perfectly but shows yesterday's stock would make BP02 worse. The consequence for this essay is a simple rule: every AI component reads the shop's own operational records through the same DAOs as the rest of the system, and every AI result has a non-AI fallback.

## 3.2. Opportunity Analysis

*Table 28. AI opportunities derived from the business problems*

| Activity / problem | AI opportunity | Decision |
|---|---|---|
| Finding products (UC05, SN01) | Search by voice and by photo; tolerate synonyms | Build: multimodal search (from A6) |
| Judging a product (UC06, BP01) | Summarise many reviews by aspect with evidence | Build: review insights |
| Moderating reviews (UC18) | Flag spam and empty reviews for a person | Build: noise filter, human decides |
| Discovering products (UC06, UC10) | Recommend similar or complementary products | Build: content + co-purchase recommender |
| Answering status questions (BP04) | Natural-language order questions | Build, restricted to own orders |
| Pricing, fraud scoring | Dynamic prices, payment risk models | Not now: no data, provider handles fraud |
| Demand forecasting (SN09) | Forecast replenishment quantities | Later: needs months of clean inventory events |

The last two rows matter as much as the first five. AI is not used where the shop lacks the data (demand forecasting needs a history of reliable inventory events, which the system is only now starting to record) or where an external party already carries the responsibility (fraud decisions belong to the payment provider, Section 1.8). An AI feature built on data that does not exist would only add a new way to be wrong.

## 3.3. AI Use Cases

The AI functions are modelled as extensions of the Release 1 use cases. The four ways of searching are specialisations of UC05 Search Products: they share the goal and the flow *retrieve, rank, display* and differ only in how the input is read. UC33 View Review Insights and UC34 View Recommendations extend UC06 View Product Details, because a product page must remain usable without them. A speech/vision engine appears as a secondary actor so that the boundary of the AI service is visible.

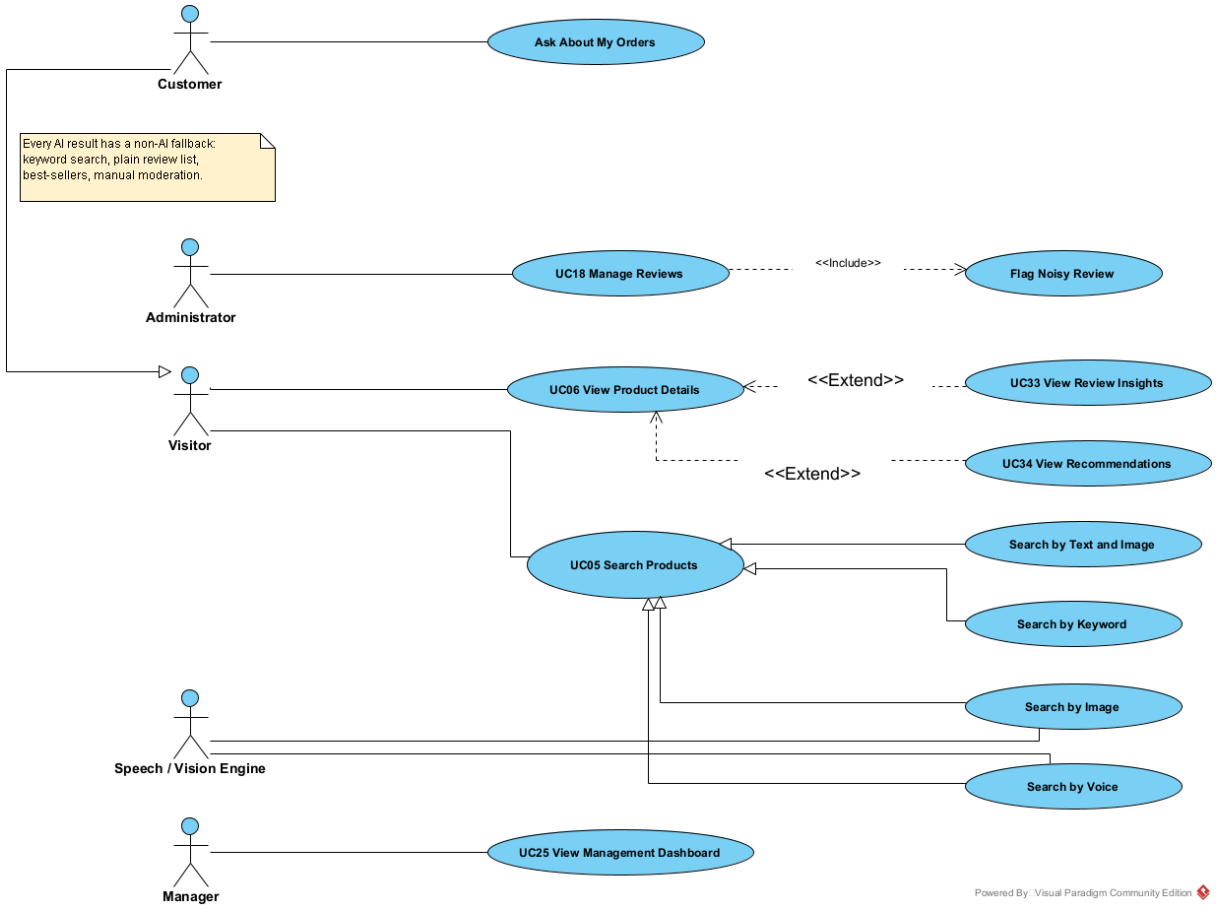



*Figure 19. AI-enabled use cases - VP diagram "UC_AI AI-Enabled Use Cases"*

## 3.4. Multimodal Search: Principles

The search service of Assignment 06 is built on one principle: different inputs are converted into one common query representation, and from there the processing is the same.

$$\textit{text } q_t \rightarrow z_t, \quad \textit{voice } q_v \rightarrow \textit{text} \rightarrow z_t, \quad \textit{image } q_i \rightarrow z_i, \quad \textit{then retrieval} \rightarrow \textit{ranking} \rightarrow \textit{results}$$

**Query understanding.** QueryService normalises the text, removes stop words, extracts a price filter ("under 100 dollars" → max_price = 100) and maps synonyms ("sneakers" → shoes, "handbag" → bag). The synonym table is new in this essay; it addresses the main error class reported in Assignment 06. A sentence that contains the word *order* becomes an order query that always carries the customer id of the session (AI-R2).

**Voice.** SpeechService.transcribe(audio) returns text. In the prototype the speech recognition is **simulated**: the argument already is the sentence that a speech-to-text engine would return. The

method is the seam where a real engine such as Whisper is plugged in without changing any caller (AI-N1).

**Images.** ImageService.encode(image) is a hand-made encoder f(I) → z of 128 values: a 4×4×4 RGB histogram of the object's pixels (colour) and an 8×8 occupancy grid of the object mask (shape), both L2-normalised. The background colour is estimated from the image border, so a photo on a different background still matches. Retrieval uses cosine similarity and keeps the top k:

$$s(I_q, I_p) = z_q^T z_p / (||z_q|| \, ||z_p||), \quad R(q) = TopK_{x \in X} s(q, x)$$

## 3.5. Retrieval and Ranking

Retrieval and ranking are separate steps and separate classes. SearchService.retrieve builds the candidate set C(q) from keyword matches and from the image index; RankingService.rank orders it. The separation lets the business change its ranking policy without touching retrieval. The final score combines three signals:

$$S(x|q) = \alpha \cdot S_{text} + \beta \cdot S_{image} + \gamma \cdot S_{business}$$

- $S_{text}$ = matched query words / number of query words; $S_{image}$ = cosine similarity (0..1);
- $S_{business}$ = 0.8·[available > 0] + 0.2·min(available, 20)/20, read from the shop's own stock (stock-aware ranking);
- text and voice: (α, β, γ) = (0.9, 0, 0.1); image: (0, 0.9, 0.1); text + image: $\alpha = 0.9 \cdot \lambda_t$, $\beta = 0.9 \cdot \lambda_i$, $\lambda_t + \lambda_i = 1$ (default 0.5/0.5).

The architecture keeps the three layers of Assignment 06, now inside the shop: the presentation layer is the shop's controller and view, the application-intelligence layer contains the five AI services, and the data layer contains the same ProductDAO, InventoryDAO and OrderDAO that the checkout uses, plus VectorIndex and ImageStorage. Every dependency points downwards.

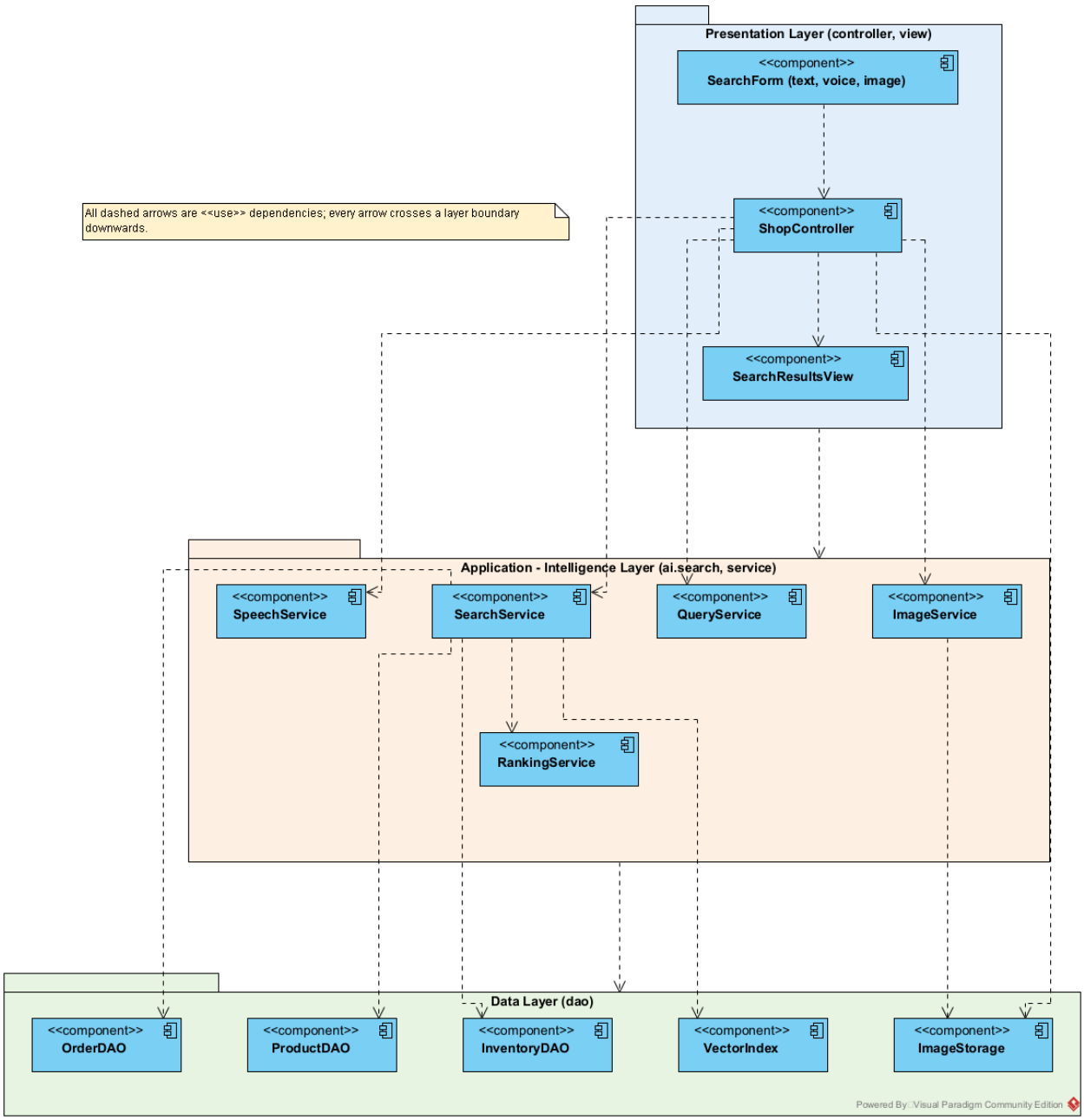



*Figure 20. Three-layer architecture of the search subsystem - VP diagram "CMP01 AI Search Three-Layer Architecture"*

One dependency needed care. VectorIndex must call the encoder to index the product images, but the encoder lives in the layer above. Instead of letting the data layer import upwards, the composition root passes the encoding function in: VectorIndex.build(storage, products, encode). The two sequence diagrams below show voice and image search with the real method names of the code.

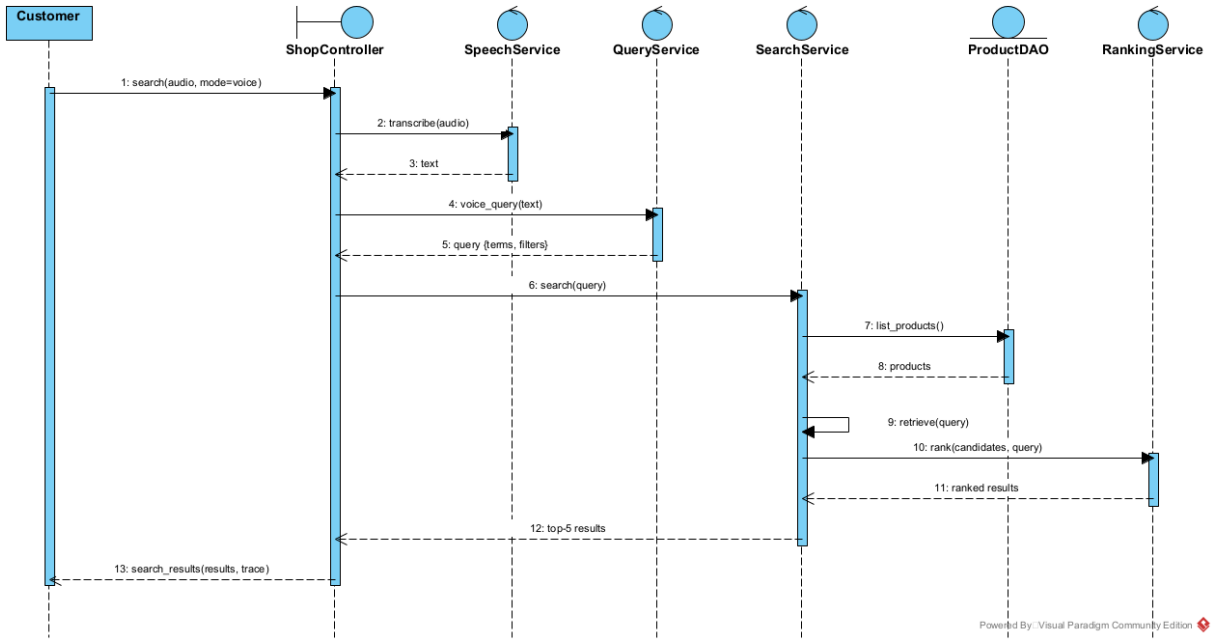



*Figure 21. Voice search - VP diagram "SD03 Voice Search"*

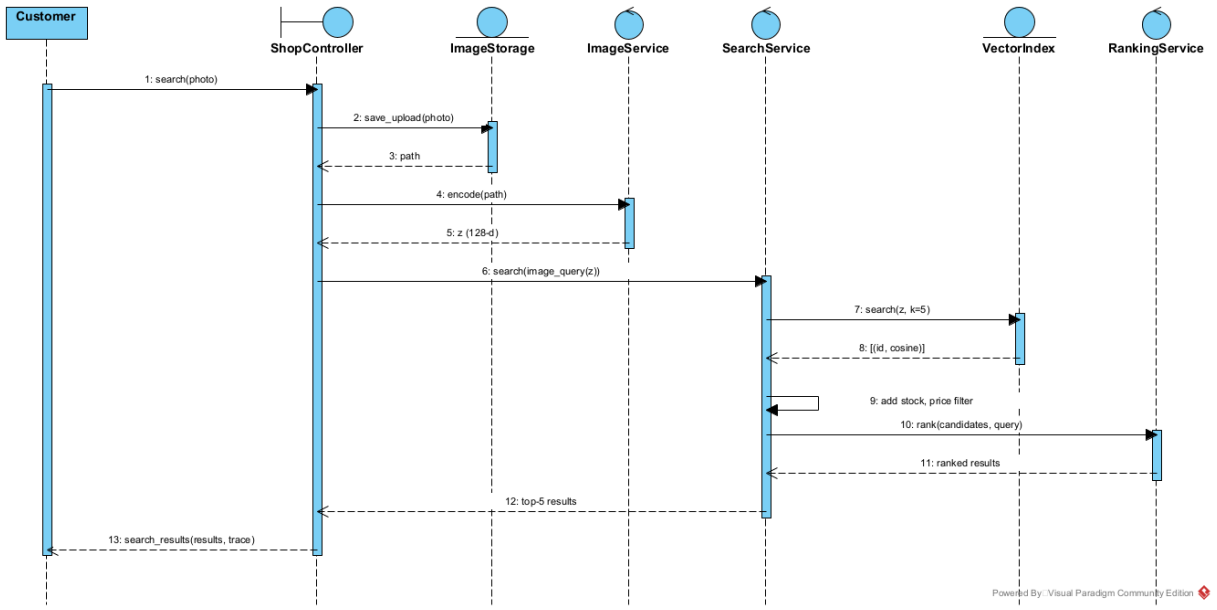



*Figure 22. Image search - VP diagram "SD04 Image Search"*

Running the four query types on the seeded shop gives the trace below. Each line names the component that performs the step, so a run can be followed on the sequence diagrams. The voice query shows the stock-aware ranking: Nike Court Sneakers matches both words and is the only Nike shoe under 100 dollars, but it is out of stock, so its business score is 0 and its final score 0.900 instead of 0.990. The text + image query shows why combining modalities helps: three products match both words, and only the photo can tell that the customer wants the red pair.

```
$ python main.py demo   (section 1)
  input (text): 'black shoes'
  QueryService -> terms=['black', 'shoes'] filters={}
  SearchService.retrieve -> 8 candidates
  RankingService.rank -> top results (alpha, beta, gamma) = (0.9, 0, 0.1)
    #  Product               Price Avail  keyword  cosine  S(x|q)
    1  Nike Running Shoes      120     6        2  0.000   0.986
    2  Black Cotton T-Shirt     15    43        1  0.000   0.550
    3  Black Leather Bag        80    17        1  0.000   0.547
    4  Adidas Running Shoes    100    14        1  0.000   0.544
    5  Blue Sports Shoes        75    12        1  0.000   0.542

  input (voice): <audio of 'find nike sneakers under 100 dollars'>
  SpeechService.transcribe -> 'find nike sneakers under 100 dollars' (simulated STT)
  QueryService -> terms=['nike', 'shoes'] filters={'max_price': 100.0}
  SearchService.retrieve -> 4 candidates
  RankingService.rank -> top results (alpha, beta, gamma) = (0.9, 0, 0.1)
    #  Product               Price Avail  keyword  cosine  S(x|q)
    1  Nike Court Sneakers      85     0        2  0.000   0.900
    2  Red Baseball Cap         22    17        1  0.000   0.547
    3  Adidas Running Shoes    100    14        1  0.000   0.544
    4  Blue Sports Shoes        75    12        1  0.000   0.542

  input (image): query_red_shoe.png
  ImageService.encode -> 128-d embedding
  SearchService.retrieve -> 5 candidates
  RankingService.rank -> top results (alpha, beta, gamma) = (0, 0.9, 0.1)
    #  Product               Price Avail  keyword  cosine  S(x|q)
    1  Red Running Shoes       110     7        0  0.959   0.950
    2  Red Canvas Backpack      60     7        0  0.808   0.814
    3  Red Baseball Cap         22    17        0  0.790   0.808
    4  Nike Running Shoes      120     6        0  0.504   0.540
    5  Blue Sports Shoes        75    12        0  0.495   0.538

  input (text + image): 'running shoes' + query_red_shoe.png
  ImageService.encode -> 128-d embedding
  QueryService -> terms=['running', 'shoes'] filters={}
  SearchService.retrieve -> 7 candidates
  RankingService.rank -> top results (alpha, beta, gamma) = (0.45, 0.45, 0.1)
    #  Product               Price Avail  keyword  cosine  S(x|q)
    1  Red Running Shoes       110     7        2  0.959   0.969
    2  Nike Running Shoes      120     6        2  0.504   0.763
    3  Adidas Running Shoes    100    14        2  0.000   0.544
    4  Blue Sports Shoes        75    12        1  0.495   0.540
    5  Red Baseball Cap         22    17        0  0.790   0.452
```

*Figure 23. Output of the search part of main.py demo: text, voice, image, text + image*

## 3.6. Review Insights

Assignment 02 proposed a PhoBERT aspect-based sentiment model to turn reviews into evidence for customers and managers. The prototype keeps the same interface, analyse(reviews) → per-aspect sentiment with evidence, but implements it with a transparent lexicon so that it runs offline and every label can be explained. The pipeline is drawn as an activity diagram; the decision nodes are the two places where the component falls back instead of guessing.

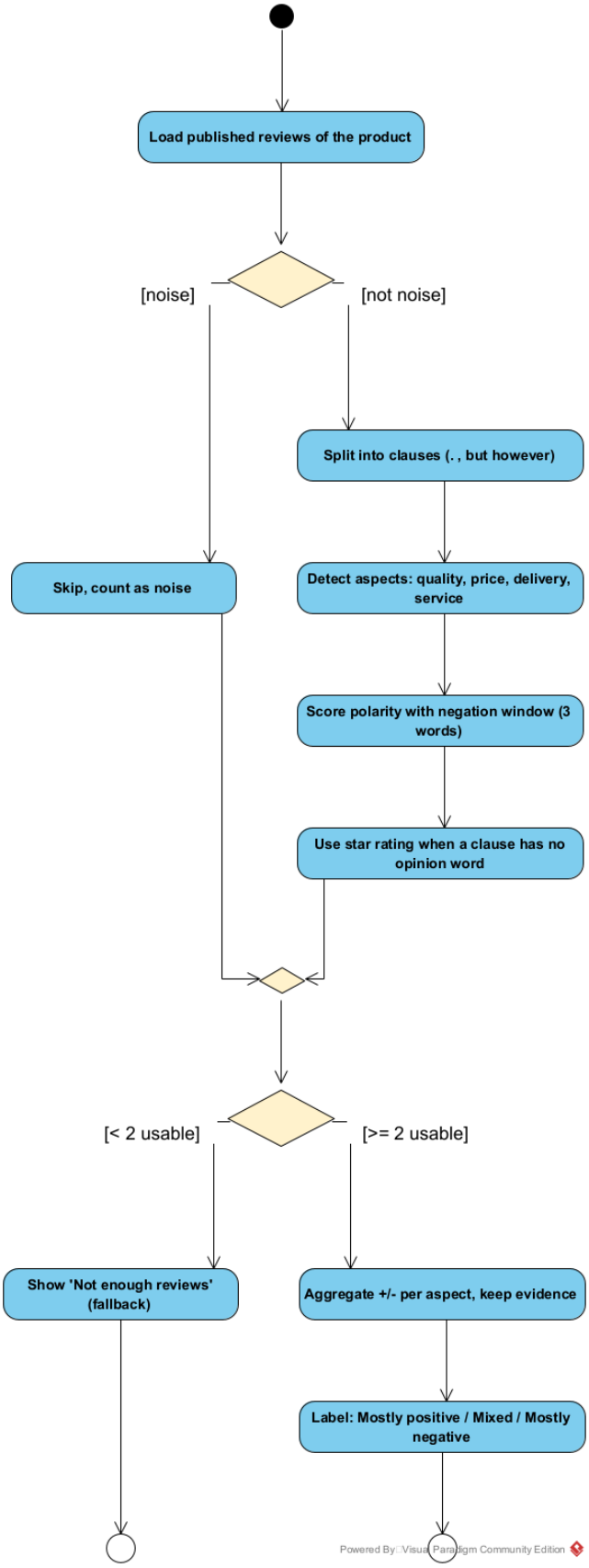



*Figure 24. Review insight pipeline - VP diagram "AD04 Review Insight Pipeline"*

- **Noise filter.** Links, long digit strings (phone numbers), repeated characters and texts with fewer than two words are noise. Noise is skipped in the statistics and, when a customer submits it, the review waits for moderation (AI-R5).

- **Clauses.** A review is split at sentence punctuation, commas and the words *but*/*however*, because one review often praises one aspect and criticises another.
- **Aspects and polarity.** Each clause is matched against four aspect vocabularies (quality, price, delivery, service; English and a few Vietnamese words). Polarity is the sum of positive and negative words, with a negation in the three previous words flipping a word ("not fast" is negative).
- **Aggregation.** Positive and negative counts per aspect, the positive share, two evidence sentences, and a label: Mostly positive (share ≥ 0.7), Mixed (≥ 0.4) or Mostly negative. With fewer than two usable reviews the label is "Not enough reviews" (AI-N3).

## 3.7. Recommendation

The recommender combines two signals computed from the shop's records. Content similarity is the cosine of TF-IDF vectors built from name, category, brand, colour and description; it works for a new product that nobody has bought yet. Co-purchase counts how often two products were in the same non-cancelled order; it captures complements that text cannot (a cap bought with running shoes).

$$score(p, q) = 0.7 \cdot cos(tfidf_p, tfidf_q) + 0.3 \cdot copurchase(p, q) / max_r\ copurchase(p, r)$$

Out-of-stock products are never recommended, and a customer is never recommended something already bought (cancelled and failed orders do not count as purchases). These are business rules, not model parameters; they are enforced in RecommenderService and checked by a test.

## 3.8. Data Requirements

*Table 29. Data cards of the AI components*

| Component | Input data and source | Quality and constraints |
|---|---|---|
| Multimodal search | 16 products (name, category, brand, colour, price) from ProductDAO; stock from InventoryDAO; 16 catalogue images and 10 query photos (synthetic drawings) | Images are drawings, so accuracy does not transfer to real photos; stock is read live |
| Speech | Transcript standing for audio (simulation) | No audio stored; a real engine must not keep recordings |
| Review insights | Published reviews from ReviewDAO (14 seeded); 30 hand-labelled reviews with 39 aspect labels for evaluation | Only reviews linked to a delivered order; no personal data used |
| Recommender | Product text; 16 seeded orders (co-purchase) | Cold start handled by content similarity |
| Order questions | Orders of the session customer only (OrderDAO.find_by_customer) | Never reads another customer's orders (NFR05) |

## 3.9. Evaluation

The search was evaluated on 26 queries: the 20 queries of Assignment 06 and six new ones that test the synonym table, the four new products and the privacy rule (query 26 asks for another customer's order and is successful only if nothing is returned). A query is successful when the top-1 result is the expected product or order.

```
$ python evaluate.py
=== 1. Multimodal search ===
  #  Mode       Query                                Top-1 result             Score  Success
  1  text       black shoes                          Nike Running Shoes       0.986  Yes
  2  text       white polo shirt                     White Polo Shirt         0.988  Yes
  3  text       find Nike shoes under 100 dollars    Nike Court Sneakers      0.900  Yes
  4  text       red bag                              Red Canvas Backpack      0.987  Yes
  5  text       cheap watch under 60                 Silver Analog Watch      0.542  Yes
  6  text       leather handbag                      Black Leather Bag        0.997  Yes
  7  voice      find running shoes                   Adidas Running Shoes     0.994  Yes
  8  voice      show me a blue cap                   Blue Denim Cap           1.000  Yes
  9  voice      i want a smart watch                 Black Smart Watch        0.985  Yes
 10  image      query_black_shoe.png                 Nike Running Shoes       0.955  Yes
 11  image      query_red_shoe.png                   Red Running Shoes        0.965  Yes
 12  image      query_white_shoe.png                 Adidas Running Shoes     0.906  Yes
 13  image      query_black_bag.png                  Black Leather Bag        0.982  Yes
 14  image      query_blue_cap.png                   Blue Denim Cap           0.990  Yes
 15  image      query_silver_watch.png               Silver Analog Watch      0.905  Yes
 16  image      query_black_tshirt.png               Black Cotton T-Shirt     0.987  Yes
 17  multimodal running shoes + query_red_shoe.png   Red Running Shoes        0.976  Yes
 18  multimodal nike + query_white_shoe.png          Nike Court Sneakers      0.844  Yes
 19  text       find my order O001                   O001                        -   Yes
 20  voice      where is my latest order             O016                        -   Yes
 21  text       white sneakers                       Adidas Running Shoes     0.994  No (expected P05)
 22  voice      blue smartwatch                      Blue Smart Watch         0.983  Yes
 23  image      query_red_cap.png                    Red Baseball Cap         0.964  Yes
 24  image      query_blue_backpack.png              Blue Canvas Backpack     0.967  Yes
 25  multimodal cap + query_blue_cap.png             Blue Denim Cap           0.995  Yes
 26  text       find my order O002                   (nothing)                   -   Yes

 Total queries: 26   Successful: 25   Success rate: 96.2%
 Per mode: image 9/9, multimodal 3/3, order 3/3, text 6/7, voice 4/4
```

*Figure 25. Output of evaluate.py, part 1: multimodal search on 26 test queries*

The success rate is 25/26 = 96.2% (text 6/7, voice 4/4, image 9/9, multimodal 3/3, order 3/3). The review analyser and the recommender were evaluated on their own test data:

```
$ python evaluate.py   (continued)
=== 2. Review insights (aspect-based sentiment) ===
 Reviews: 30   gold aspect labels: 39   predicted: 38
 Aspect detection  precision 0.97  recall 0.95  F1 0.96
 Polarity accuracy on detected aspects: 35/37 = 0.95
 Errors:
   - extra aspect  quality   "The price is too expensive for this quality."
   - missed aspect delivery  "Good material but shipping took two weeks, very slow."
   - polarity      quality   "Leather looks cheap but the price is fair."
   - missed aspect quality   "Hang dep, giao hang nhanh, gia hop ly."
   - polarity      price     "Expensive, but the quality makes it worth it."

=== 3. Recommender ===
 Same-category hit rate of top-3 similar products: 27/48 = 0.56
 Catalogue coverage: 15/15 in-stock products appear in some top-3 list
 Out-of-stock products recommended: 0
```

*Figure 26. Output of evaluate.py, parts 2 and 3: review insights and recommender*

*Table 30. Evaluation summary*

| Component | Metric | Value |
|---|---|---|
| Search | Top-1 success rate | 25 / 26 = 96.2% |
| Search | Image queries / order queries (incl. privacy) | 9 / 9, 3 / 3 |
| Review insights | Aspect detection precision / recall / F1 | 0.97 / 0.95 / 0.96 |
| Review insights | Polarity accuracy on detected aspects | 35 / 37 = 0.95 |
| Recommender | Same-category hit rate of top-3 | 27 / 48 = 0.56 |
| Recommender | Coverage / out-of-stock recommended | 15 / 15 / 0 |

## 3.10. Error Analysis

- **Query 18 "nike" + white shoe photo.** In Assignment 06 this query failed: the expected Nike Court Sneakers is out of stock, loses 0.09 business points, and Nike Running Shoes won. For V4 the catalogue and query photos were redrawn with clearer shapes (abc_shop/tools/make_images.py); the court sneakers now match the photo with cosine 0.876 against 0.647 for the running shoes, which outweighs the stock penalty (0.844 against 0.827). The margin is small, so the case still shows that γ can override the visual match and must be tuned on real behaviour.
- **Query 21 "white sneakers".** The synonym table maps *sneakers* to *shoes*, so the word that identified the court sneakers is lost and Adidas Running Shoes (white, in stock) wins. Mapping to both words, or embedding-based text matching, would keep the specific term. A fix for one error class created another, which is why the test set is run on every change.
- **Review insights.** The five errors are typical lexicon limits: an aspect word used as a reference ("too expensive for this quality"), an opinion split from its aspect by a comma ("shipping took two weeks, very slow"), two aspects with opposite polarity in one clause, and missing Vietnamese vocabulary. Sarcasm ("Oh great, another two weeks") was classified correctly only because the star rating broke the tie.

- **Recommender.** A hit rate of 0.56 is low only if the goal is "more of the same". Many top-3 lists mix a same-category item with a complement bought together (shoes and caps), which is intended by the 0.3 co-purchase weight.

## 3.11. Risks, Fallbacks and Human Control

*Table 31. AI risks and the design response*

| Risk | Effect | Response (fallback / control) |
|---|---|---|
| Wrong top result | Customer misses the product | Show top-5 with scores; keyword search and category browsing always available |
| Speech or image engine unavailable | Voice/photo search fails | Text search unchanged; error message instead of empty page |
| Ranking hides out-of-stock items | Customer cannot find a known product | Out-of-stock items are ranked lower, never removed from text results |
| Spam or abusive review | Misleading product page | Noise is held for moderation; an Administrator publishes or rejects (UC18) |
| Misleading review summary | Wrong purchase decision | Evidence sentences shown next to the counts; full review list below |
| Order question leaks data | Privacy breach | Order queries carry the session customer id; tested with another customer's order |
| Feedback loop in recommendations | Popular items get more popular | Content similarity keeps new items visible; coverage measured |

Human control is designed in at two points. Reviews flagged as noise are never published automatically; they appear in the moderation queue, where a person decides. And no AI component can change an order, a price or a stock level: AI only ranks, labels and suggests. Every state change still goes through the services and rules of Chapter 4.

## 3.12. Technology Options for a Production Version

*Table 32. Prototype components and production replacements*

| Prototype | Production option | What stays the same |
|---|---|---|
| SpeechService (simulated) | Whisper or a cloud speech API with Vietnamese support | transcribe(audio) -> text |
| ImageService (colour + shape) | CLIP image encoder (shared space with text) | encode(image) -> vector |
| VectorIndex (in memory) | FAISS or a vector database | search(z, k) -> [(id, score)] |
| Lexicon review analyser | PhoBERT fine-tuned for ABSA (Assignment 02) | analyse(reviews) -> aspects, label |
| TF-IDF + co-purchase | Matrix factorisation / two-tower model with click data | similar(id, k), for_customer(id, k) |

The right-hand column is the important one: the interfaces were fixed in Chapter 1 as AI-N1, so each replacement is a change in one class plus the composition root, and the 40 tests of Chapter 5 continue to describe the behaviour that the shop expects.

## 3.13. AI Component Specifications

*Table 33. AI component specifications*

| ID / component | Input → output | Interface | Fallback |
|---|---|---|---|
| AIC-01 SpeechService | audio → text | transcribe(audio) -> str | Text box; message "please type your search" |
| AIC-02 QueryService | text → {type, terms, filters} or order query | text_query, voice_query, image_query, multimodal_query, order_query | Raw words as terms |
| AIC-03 ImageService + VectorIndex | image → 128-d vector → top-k ids | encode(path); VectorIndex.search(z, k) | Category browsing |
| AIC-04 Search + RankingService | query → ranked products with scores | search(query, k) -> [result] | Keyword-only ranking |
| AIC-05 ReviewInsightService | reviews → aspects, counts, evidence, label | analyse(reviews); is_noise(text) | Plain review list |
| AIC-06 RecommenderService | product / customer → products with scores | fit(); similar(id, k); for_customer(id, k) | Same-category products |

## 3.14. Worked Example: Ranking a Combined Query

The combined query "running shoes" with a photo of a red shoe shows how the formula of Section 3.5 decides. With $\lambda_t = \lambda_i = 0.5$ the weights are $\alpha = \beta = 0.45$ and $\gamma = 0.1$. All three running shoes match both query words, so $S_{text} = 2/2 = 1$ for each of them; the photo separates them.

*Table 34. Score components of the top three results*

| Product | S_text | S_image (cosine) | Available | S_business | S(x\|q) |
|---|---|---|---|---|---|
| Red Running Shoes | 1.00 | 0.959 | 7 | 0.8 + 0.2·7/20 = 0.870 | 0.45 + 0.432 + 0.087 = 0.969 |
| Nike Running Shoes | 1.00 | 0.504 | 6 | 0.8 + 0.2·6/20 = 0.860 | 0.45 + 0.227 + 0.086 = 0.763 |
| Adidas Running Shoes | 1.00 | 0 (not in image top-5) | 14 | 0.8 + 0.2·14/20 = 0.940 | 0.45 + 0 + 0.094 = 0.544 |

The example also shows a limit: Adidas Running Shoes receives no image score at all because the image retrieval keeps only the five most similar pictures, and the red cap and red backpack are more similar in colour than a white shoe. With a larger k, or with a neural encoder that captures shape better than colour, it would get a small positive score. The order of the results would not change, which is why k = 5 was kept.

## 3.15. Worked Example: Analysing One Review

The review "Comfortable shoes but delivery was late by four days." (3 stars) is processed as follows. It is not noise (more than two words, no link). It is split at *but* into "comfortable shoes" and "delivery was late by four days". The first clause contains the quality word *comfortable*, which is also a positive word: quality +1. The second contains the delivery word *delivery* and the negative word *late*: delivery −1. The star rating is not needed because both clauses carry an opinion. On the product page, this review adds one positive quality count and one negative delivery count, and may be shown as evidence for either aspect.

## 3.16. Business Evaluation

Technical metrics say whether a component works; business metrics say whether it helps the shop. The prototype cannot measure the second kind, because it has no real customers, so the metrics are defined now and attached to the AI requirements, to be collected with an A/B comparison after release.

*Table 35. Business metrics of the AI features*

| Feature | Metric | Linked problem / need |
|---|---|---|
| Multimodal search | Search-to-cart rate; share of searches with no click; voice/photo usage | SN01, BP01 |
| Order questions | Status enquiries to staff per 100 orders | BP04, SN04 |
| Review insights | Return rate of products with negative aspects; time to resolve a recurring complaint | SN06, SN10 |
| Noise filter | Share of spam that reaches the product page; moderation time | SN06 |
| Recommendations | Click-through and add-to-cart rate of recommended items; items per order | Business objective (sales) |

## 3.17. AI Traceability

*Table 36. From problem to AI requirement, component, metric and test*

| Need / problem | AI requirement | Component | Measured by | Test |
|---|---|---|---|---|
| SN01 find products | AI-R1, AI-R3 | AIC-01..04 | Top-1 success 96.2% | test_image_and_multimodal_search |
| BP04, NFR05 status, privacy | AI-R2 | AIC-02, OrderDAO | 3/3 order queries | test_order_questions_only_see_own_orders |
| SN06 controlled reviews | AI-R4, AI-R5 | AIC-05 | F1 0.96 | test_review_insights_handle_negation_and_noise |
| Sales objective | AI-R6 | AIC-06 | Coverage 15/15 | test_recommender_never_suggests_... |
| NFR09 maintainability | AI-N1 | all | Interfaces fixed | test_dependencies_only_point_downwards |

## 3.18. Chapter Summary

Three AI components were analysed like any other requirement: they come from business problems, extend existing use cases, read the shop's own data, have fallbacks and are measured. The search keeps the Assignment 06 design and reaches 96.2% top-1 success on 26 queries; the review analyser reaches an aspect F1 of 0.96; the recommender never suggests unavailable or already-bought items. The honest limits (synthetic images, simulated speech, a lexicon instead of a trained model) are stated together with the production replacements that the interfaces allow.

# CHAPTER 4. SYSTEM DESIGN

This chapter follows the six steps our lecturer set for the design part of the essay. I start again from the class diagram of the analysis (Section 4.1), turn it into a data model and a real database (Section 4.2), update the design class diagram with typed attributes, methods and relationships (Section 4.3), place the classes in layers and show how they depend on each other (Section 4.4), generate code from that layered model (Section 4.5) and close with the decisions and their costs (Section 4.6). The work builds on the four transformation steps "from analysis to design" that we practised in Test 01, and on the search architecture of Assignment 06.

All diagrams of the chapter belong to the same Visual Paradigm project as the rest of the essay (TieuLuan_KHANHBQ.vpp) and were drawn in Visual Paradigm Community Edition. I drew them through the Visual Paradigm MCP plugin, which adds shapes and lines through Visual Paradigm's Open API and routes every line horizontally and vertically around the boxes. The finishing touches the plugin cannot make, such as a keyword on a dependency or the size of the multiplicity labels, are done by a small plugin of my own (essay.gen). That plugin also reads the finished model back. It writes the SQL script of Section 4.2 and exports the design classes for the code generator of Section 4.5. The database and the code therefore come from the diagrams, not from a second copy typed by hand.

## 4.1. Class Diagram in Analysis

The analysis class diagram is the model of Section 2.16, built from the use case specifications with the noun-phrase method. For this chapter I updated it and checked it once more against the use cases. It names the things the shop has to remember and how they are connected. It has attributes but no methods, no screens and no decisions about storage. Every later step changes it, so I first list what I updated, then describe it in words and explain every relationship; each change in Sections 4.2 to 4.5 should be traceable to a line in this section.

*Table 37. Updates of the analysis class diagram*

| Element | Before (Section 2.5) | Updated | Reason |
|---|---|---|---|
| ReplenishmentRequest | missing | New class with requestId, quantity, status, createdAt | UC24 creates a request that the supplier answers; it is information the shop keeps |
| Customer - Review | missing | Association writes, 1 to 0..* | A review is written by a customer; Review - Order alone does not say by whom |
| Cart - Order | missing | Dependency <<create>> checkout | Checkout uses the cart to create the order; no lasting link remains |
| Customer - Cart | 1 to 1 | 1 to 0..1 | The cart is created when the first product is added |
| Association names | eleven of eighteen named | every association named | Each line reads as a sentence: Order contains OrderItem |
| Attributes | some identifiers and dates missing | paymentId, reviewId, createdAt, location, description, image, name in the lines | Needed by the use cases (UC06 shows the image, UC11 the location) |

### 4.1.1. The classes in words

The twenty-three classes fall into four groups. The **people** group describes who uses the shop. User is anyone who logs in, with an id, a name, an email address and a role. Customer is a kind of User that also has a registration date. Staff accounts are plain Users whose role is sales, warehouse, admin or manager; they have no attributes of their own, so they need no subclass. Address holds the receiver, phone, street and city of one delivery address.

The **shopping** group describes what a customer intends to buy and what was bought. Cart is the customer's open basket, and CartItem is one line of it, with a quantity and the cart price shown while the customer is still shopping. Order is created at checkout. It has a status, the discount, the shipping fee, the delivery address copied at that moment and the idempotency key of the checkout form. OrderItem is one line of the order with the unit price actually charged. StatusChange is one entry of the order's history: who changed the status, when and why. Promotion is a discount code with a percentage, a minimum total and a period of validity.

The **fulfilment** group describes what happens after checkout. Payment is one payment attempt, with the attempt reference sent to the provider, the amount, the currency and the result. Shipment is one delivery with the carrier's reference, and Milestone is one carrier message (picked up, in transit, delivered) with a sequence number. Review is the customer's rating and text about a product they received.

The **catalogue and stock** group describes what the shop sells. Product has a name, a brand, a colour, a price, a description, an image and a status (active or unpublished), and belongs to one Category. Its stock is a separate StockLevel with the physical and reserved quantities and a low-stock threshold. The derived attribute /available is physical minus reserved. The attributes that only one group of products has live in the abstract ProductDetail and its four kinds (FootwearDetail, ApparelDetail, BagDetail, WatchDetail); Category.detailKind names the

kind its products use. Every change of the stock is an InventoryEvent of a kind (receipt, reserve, release, issue, adjust) with a reference that identifies its cause, and a ReplenishmentRequest asks the supplier for more.

Two attributes look like duplicates but mean different things. CartItem.cartPrice and OrderItem.unitPrice both copy Product.price, but the cart price may still change before checkout, while the order price is a historical fact: the price the customer actually paid. Order.deliveryAddress is the same kind of copy, because a customer who moves must not change where an old parcel went.

### 4.1.2. Relationships

Choosing the kind of each relationship is where an analysis class diagram says the most about the business. The table gives every relationship its kind, its multiplicity and the reason behind it. Read the multiplicity from left to right: "1 to 0..*" between Customer and Order means that one customer places zero or more orders and every order is placed by exactly one customer. The updated diagram follows the table.

*Table 38. Relationships of the analysis class diagram*

| Relationship | Kind | Multiplicity | Reason |
|---|---|---|---|
| Customer - User | Generalization | - | A customer is a user with a registration date |
| Customer delivers to Address | Aggregation | 1 to 0..1 | The address belongs to the customer but can be replaced |
| Customer owns Cart | Association | 1 to 0..1 | At most one open cart, created on first use |
| Customer places Order | Association | 1 to 0..* | Any number of orders, each with one customer |
| Customer writes Review | Association | 1 to 0..* | Each review has one author |
| Cart contains CartItem | Composition | 1 to 0..* | Lines live only inside their cart; an empty cart is allowed |
| Order contains OrderItem | Composition | 1 to 1..* | An order without lines cannot exist |
| Order records StatusChange | Composition | 1 to 1..* | The history is part of the order and starts with its creation |
| CartItem, OrderItem refers to Product | Association | 0..* to 1 | Each line is about one product |
| Order paid by Payment | Association | 1 to 0..* | Several attempts are possible (UC09 A5 retry) |
| Order uses Promotion | Association | 0..* to 0..1 | At most one code per order |
| Order shipped as Shipment | Association | 1 to 0..1 | A shipment exists only after packing |
| Shipment tracked by Milestone | Composition | 1 to 0..* | Carrier messages about one shipment |
| Review verified by Order; Review about Product | Association | 0..* to 1 | A review needs a delivered order of that product (FR14) |
| Product belongs to Category | Association | 0..* to 1 | Every product is in one category |
| Product has StockLevel | Association | 1 to 1 | Stock is kept apart so that two roles do not edit one record (BP01, BP02) |

| Relationship | Kind | Multiplicity | Reason |
|---|---|---|---|
| StockLevel explained by InventoryEvent | Association | 1 to 0..* | Every quantity is explained by events (FR19) |
| Product replenished by ReplenishmentRequest | Association | 1 to 0..* | UC24 may ask several times for one product |
| Cart to Order, <<create>> checkout | Dependency | - | The cart is used to create the order; the order does not remember the cart |

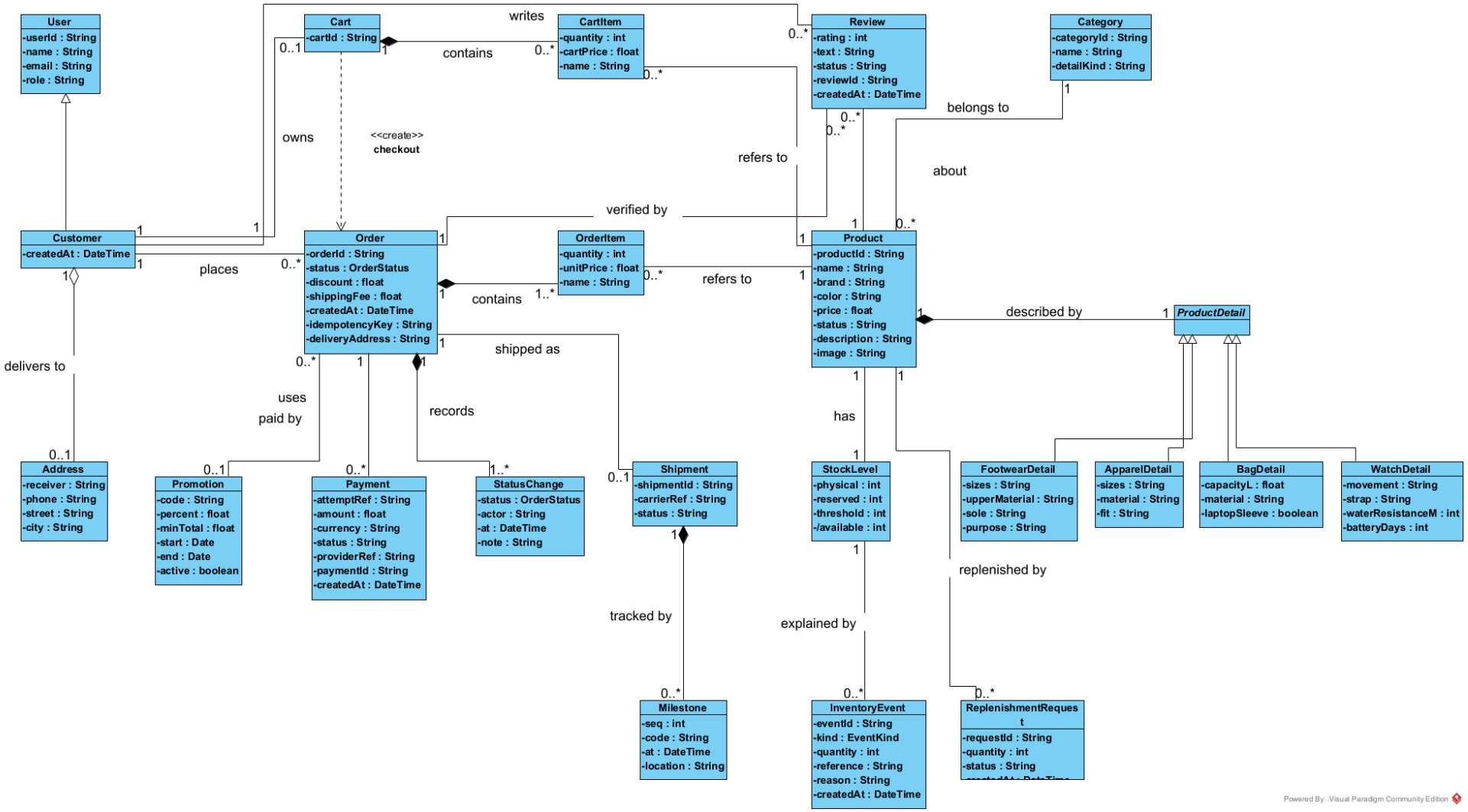



*Figure 27. Analysis class diagram of ABC Online Shop, updated with association names, multiplicities and the checkout dependency - VP diagram "CL01 Analysis Class Diagram"*

The dependency from Cart to Order is the choice I would defend first. An association would claim that every order remembers the cart it came from, which is not true: the cart is emptied after checkout and used again. A dependency with the <<create>> keyword and the name checkout says exactly what happens. In the design this relationship becomes a method, Order.create_from_cart(cart), and not a stored key. The composition Order-StatusChange is the other one worth a word: the history is not a log kept beside the order, it is part of the order, and the state machine of Section 2.17 can only be trusted because every transition leaves an entry.

## 4.2. Data Model and Database

The data model decides what is stored and how the stored records are linked. It was built in three levels, each drawn in Visual Paradigm: a conceptual model of entities and relationships, a logical model of relations with keys, and a physical model of SQLite tables. The SQL script was then generated from the physical model and a working database was created from it (Section 4.2.5).

### 4.2.1. Conceptual data model

The conceptual model names the things the shop keeps data about and how they are related, without attributes or keys. It is drawn in Visual Paradigm's ER notation (crow's foot): a bar is one, a circle zero, a foot many. Nineteen entities appear. Three of them show decisions that are not visible in the class model. ORDER_LINE and CART_LINE are entities of their own, because a line has a quantity and a price of its own. STAFF is related to STATUS_CHANGE (0..1 to 0..*), because a status change is made either by a staff member or by the system. And PRODUCT is related to PRODUCT_DETAIL one to one: every product has exactly one description of its own kind.

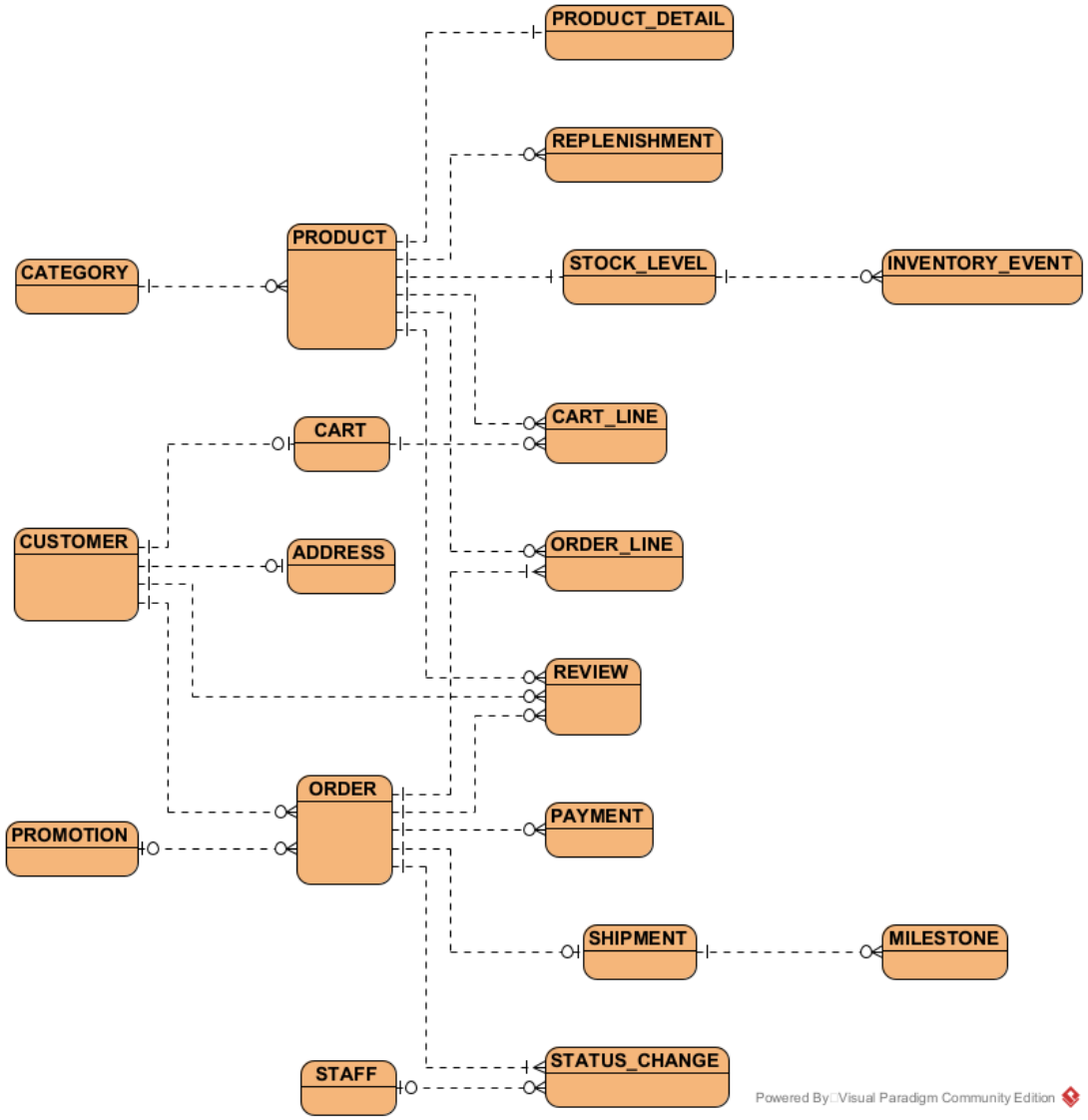



*Figure 28. Conceptual data model: entities and relationships with their multiplicities - VP diagram "CDM01 Conceptual Data Model"*

### 4.2.2. Logical data model

The logical model gives every entity its attributes and keys, still without a particular database: generic types, primary keys, foreign keys, optional (N) and unique (U) columns. The names are the names of the relations; the physical model of Section 4.2.4 turns them into SQL names and types. Three mapping decisions shape it.

*Table 39. Decisions of the logical model*

| Question | Decision | Why |
|---|---|---|
| How is the generalization User-Customer stored? | One relation UserAccount with a role column (single table) | The subclass adds no attribute that needs its own relation; login stays in one place |
| How are the product details stored? | One relation per detail kind whose key product_id is also the foreign key to Product (class table) | No empty columns; each kind has its own NOT NULL rules; a new kind is a new relation, not an ALTER of Product |
| How does a product know which detail relation to read? | Category.detail_kind names the kind | The application reads Product joined with Category and then exactly one detail relation |
| Where do order lines get their price? | OrderItem keeps name and unit_price | A historical fact that must not change with the catalogue (3NF exception by intent) |

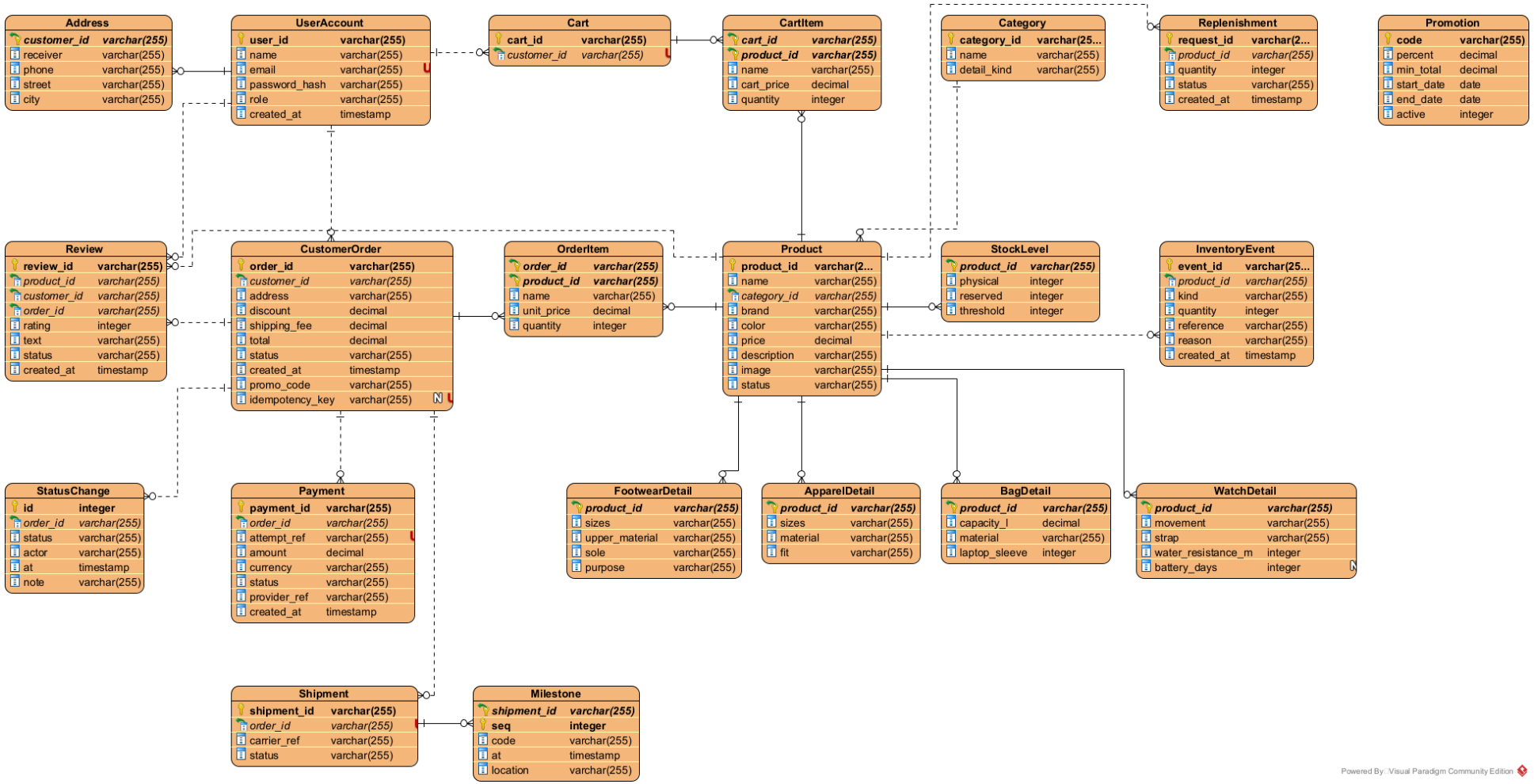



*Figure 29. Logical data model: relations with attributes, keys, optional and unique attributes - VP diagram "LDM01 Logical Data Model"*

The rule that a product's detail row lives in the relation named by its category (BR-CAT-01) cannot be expressed by a foreign key alone, because the key product_id is the same in all four detail relations. It is enforced twice: CatalogService refuses a product whose detail kind differs from its category, and the database has a trigger on each detail table that rejects such a row. Both are tested.

### 4.2.3. From classes to tables

Most classes become one table each, but several relationships needed a decision. The table lists the decisions and the reason for each.

*Table 40. Mapping rules from analysis classes to tables*

| Class or relationship | Decision in the database | Reason |
|---|---|---|
| User and Customer (generalization) | One table users with a role column | Customer adds only a date; staff add nothing. One table keeps login in one place |
| Address (aggregation, 0..1) | Table address whose key customer_id is also the foreign key | One address per customer; deleting the customer removes it |
| Cart (1 to 0..1) | cart.customer_id NOT NULL UNIQUE | UNIQUE is the database form of 0..1 |
| Compositions (lines, history, milestones) | Child key contains the parent key: (cart_id, product_id), (order_id, product_id), (shipment_id, seq) | A line cannot exist without its parent; one line per product |
| Order | Table customer_order | ORDER is a reserved word in SQL |
| Order uses Promotion | Column promo_code, no foreign key | The code is kept as it was used, even if the promotion is later deleted |
| Order.deliveryAddress | Column address (text) | A historical copy, not a reference |
| Cart to Order, <<create>> | No table and no key | It is an event, not stored data |
| Rules of Chapter 2 (BR, NFR07) | CHECK and UNIQUE constraints | The database rejects what the code might miss |
| Technical needs | Tables sequence (readable ids O001) and audit_log (NFR06) | Not in the analysis; required by design |

*Table 41. Integrity rules expressed as database constraints*

| Constraint | Table | Requirement |
|---|---|---|
| CHECK (physical >= 0, reserved >= 0, physical >= reserved) | stock_level | FR19, BR20: never negative |
| UNIQUE (product_id, kind, reference) | inventory_event | NFR07: an event is recorded once |
| UNIQUE (idempotency_key) | customer_order | UC08 A7: one order per submission |
| UNIQUE (attempt_ref) | payment | FR10: one result per attempt |
| UNIQUE (order_id) | shipment | One shipment per order (Release 1) |
| PRIMARY KEY (shipment_id, seq) | shipment_milestone | UC23: duplicate milestones ignored |
| UNIQUE (customer_id, product_id, order_id); CHECK rating 1-5 | review | FR14: one review per purchase |
| CHECK (total > 0), price > 0 | customer_order, product | Valid amounts |

The order table is called *customer_order* because ORDER is a reserved word in SQL. The status history is a table of its own with an auto-increment key, so rows are only ever appended. The audit_log table has no foreign keys on purpose: it must keep its rows even if the referenced record is later removed.

### 4.2.4. Tables and relationships

The result is twenty-three tables: twenty-one from the twenty-three analysis classes (User and Customer share the table users, the abstract ProductDetail has no table of its own, and StatusChange becomes order_status_history) plus the technical tables sequence and audit_log. The table lists them with their keys; a key written as PK, FK is a primary key that is also a foreign key.

*Table 42. The twenty-three tables of the database*

| Table | Primary key | Foreign keys | Holds |
|---|---|---|---|
| users | user_id | - | Customers and staff with role and password hash |
| address | customer_id (PK, FK) | customer_id → users | One delivery address per customer |
| cart | cart_id | customer_id → users (UNIQUE) | Open baskets |
| cart_item | cart_id, product_id | cart_id → cart, product_id → product | Basket lines with cart price |
| customer_order | order_id | customer_id → users | Orders with status, totals, idempotency key |
| order_item | order_id, product_id | order_id → customer_order, product_id → product | Order lines with the price paid |
| order_status_history | id | order_id → customer_order | Every status change |
| payment | payment_id | order_id → customer_order | Payment attempts (attempt_ref UNIQUE) |
| shipment | shipment_id | order_id → customer_order (UNIQUE) | One delivery per order |
| shipment_milestone | shipment_id, seq | shipment_id → shipment | Carrier messages in sequence |
| category | category_id | - | Product categories |
| product | product_id | category_id → category | Attributes every product has: name, brand, colour, price, status |
| footwear_detail, apparel_detail, bag_detail, watch_detail | product_id (PK, FK) | product_id → product | Attributes of one product group; category.detail_kind says which table |
| stock_level | product_id (PK, FK) | product_id → product | Physical, reserved, threshold |
| inventory_event | event_id | product_id → product | Typed stock events |

| Table | Primary key | Foreign keys | Holds |
|---|---|---|---|
| replenishment_request | request_id | product_id → product | Requests to the supplier |
| review | review_id | product_id → product, customer_id → users, order_id → customer_order | Verified reviews |
| promotion | code | - | Discount codes |
| audit_log | id | - | Who changed what (NFR06) |
| sequence | name | - | Counters for readable ids |

The twenty-two foreign keys are of two kinds, and it helps to read them by meaning. **Ownership** keys tie a record to the record it belongs to: a cart line to its cart, an order line, a payment, a shipment and a status change to their order, a milestone to its shipment, an address and a cart to their customer. The parent is mandatory and the key is NOT NULL. **Reference** keys point to something that lives on independently, such as the product of a line or of a review, or the category of a product. Deleting an order line must not delete the product. In ERD01 and ERD02 the notation is Visual Paradigm's crow's foot: a bar means one, a circle zero and the three-pronged foot many. A gold key marks a primary key, a key with a green arrow a column that is both primary and foreign key, N a column that may be empty and the red U a unique column. The model is split into two diagrams so that the column names stay readable; users, product and customer_order appear in both as views of the same tables.

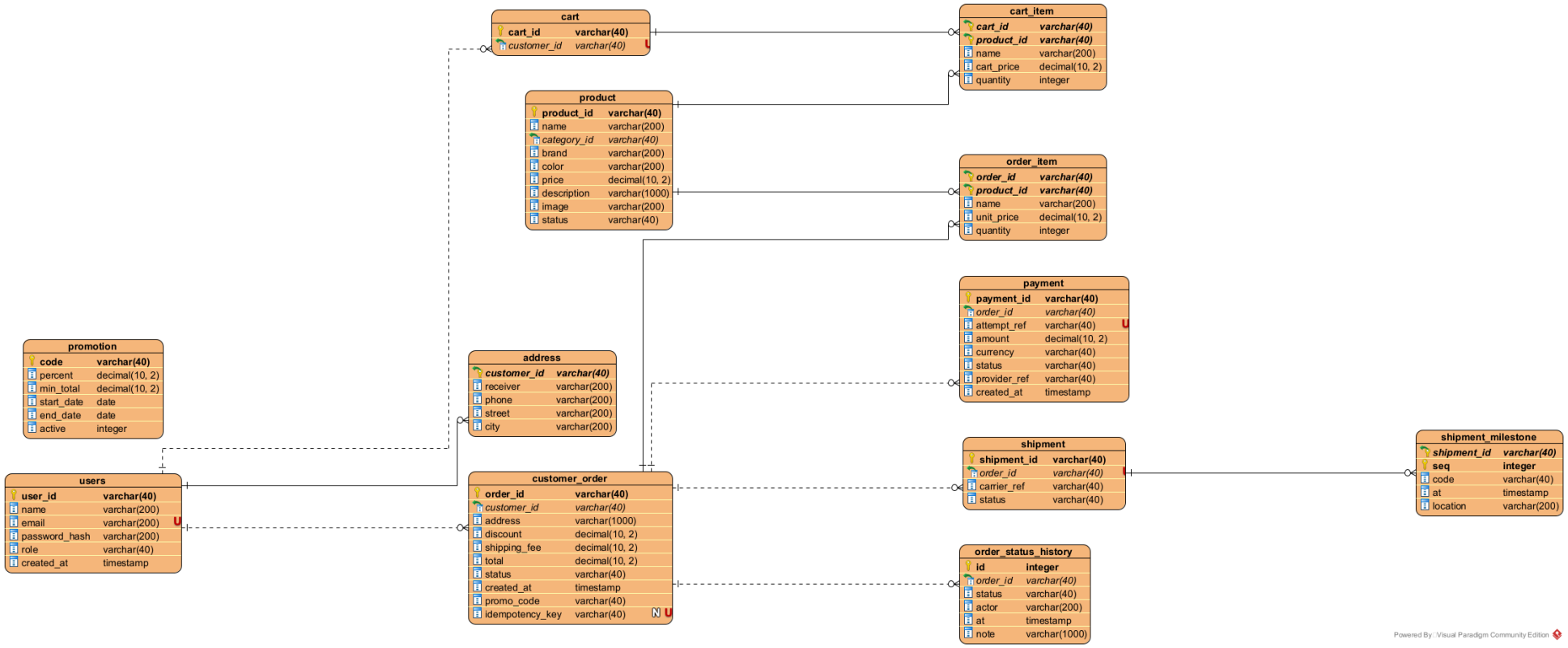



*Figure 30. Physical data model, part 1: customers, carts, orders, payments and shipments - VP diagram "ERD01"*

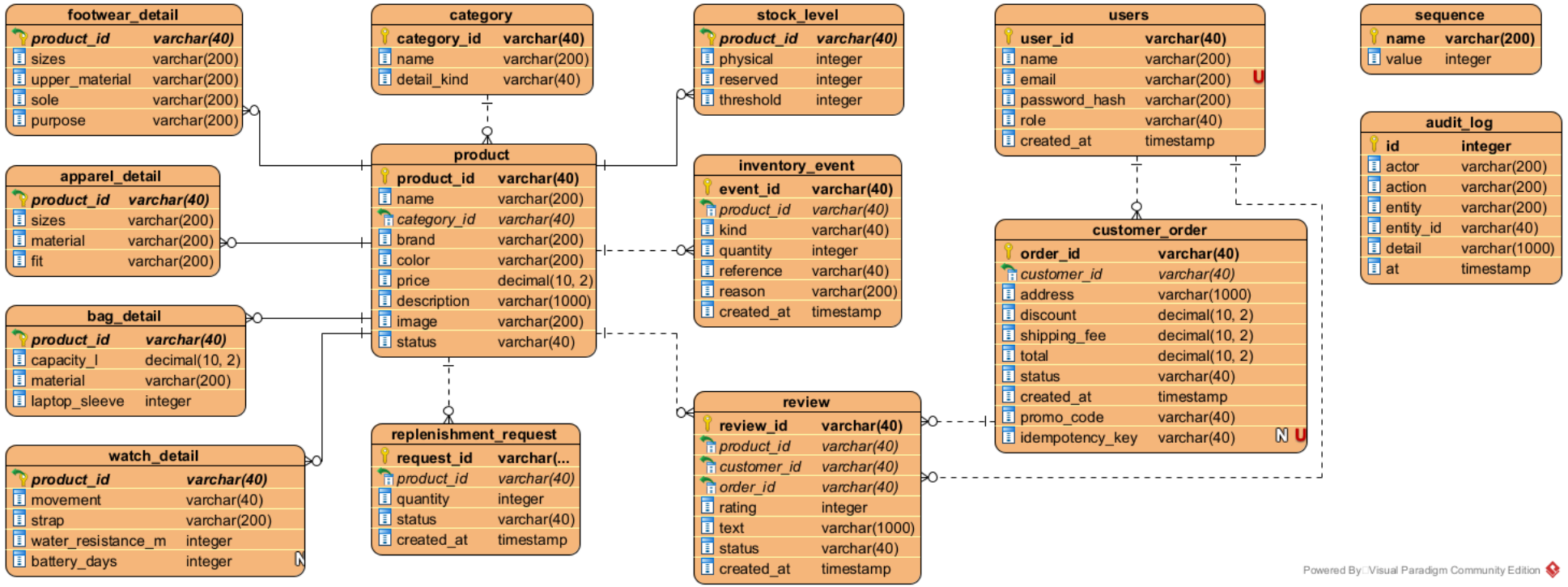



*Figure 31. Physical data model, part 2: catalogue, stock, reviews and the technical tables - VP diagram "ERD02"*

Before generating the script I checked the tables against the normal forms. **1NF**: every column holds one value; an address is split into receiver, phone, street and city, and everything that repeats (lines, payments, milestones, events) has its own table. **2NF**: the composite keys are (cart_id, product_id), (order_id, product_id) and (shipment_id, seq), and every other column of those tables depends on the whole key; a line's name and price belong to that line, not to the product alone. **3NF**: no column depends on another non-key column. Three columns look like exceptions but are not. order_item.unit_price and order_item.name are the facts at checkout, and customer_order.total is the amount the customer confirmed; they must not change when the catalogue changes. The schema is therefore in third normal form, with these intended historical values.

### 4.2.5. Generating the database

Visual Paradigm generates SQL from a physical ERD through Tools > DB > Generate Database, which first needs a database configuration (a JDBC driver and a dialect) in the project. Its programming interface (DatabaseManager.generateCreateSQL) failed for every database type with "Dialect class not found" when it was called without that configuration, so the script of this section was not produced by that menu. Instead of typing the script by hand, I added a DDL export to my essay.gen plugin. It walks the ER model table by table and writes each column with its type and its NOT NULL, UNIQUE, DEFAULT and CHECK rules, then the primary key and every foreign key, taken from the relationship lines. The CHECK and DEFAULT rules are stored in the model itself, in the column specification of each table, so they are visible in Visual Paradigm and not only in the script. Tables are written parents first, so that every REFERENCES clause points to a table that already exists. The result is design/create_tables.sql with 23 CREATE TABLE statements and 22 foreign keys. Two of the generated statements look like this:

```
CREATE TABLE stock_level (
  product_id VARCHAR(40) NOT NULL,
  physical INTEGER NOT NULL CHECK (physical >= 0),
  reserved INTEGER NOT NULL CHECK (reserved >= 0 AND reserved <= physical),
  threshold INTEGER NOT NULL DEFAULT 3,
  PRIMARY KEY (product_id),
  FOREIGN KEY (product_id) REFERENCES product (product_id)
);
CREATE TABLE cart_item (
  cart_id VARCHAR(40) NOT NULL,
  product_id VARCHAR(40) NOT NULL,
  name VARCHAR(200) NOT NULL,
  cart_price DECIMAL(10,2) NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  PRIMARY KEY (cart_id, product_id),
  FOREIGN KEY (cart_id) REFERENCES cart (cart_id),
  FOREIGN KEY (product_id) REFERENCES product (product_id)
);
```

Each constraint carries a requirement. The CHECK on stock_level is the database side of "stock never becomes negative" (FR19, BR20). The UNIQUE key on customer_order.idempotency_key is UC08 A7: one order per submitted form. The UNIQUE key on cart.customer_id is the multiplicity 0..1 of Customer owns Cart. Keeping these rules in the database as well as in the code is intended redundancy: the code gives the user a clear message, and the database guarantees that no other path, such as a manual correction script,

can break the rule.

To see whether the script really works I created the database from it. The program design/build_db.py runs create_tables.sql on an empty SQLite file and compares the result with abc_shop/dao/schema.sql, the schema the application ships with. It then lets the real application seed one month of shop history into the generated database, through the same services the web pages use. Finally it runs six statements that the constraints must reject.

```
> python build_db.py

[1] create_tables.sql (generated from ERD01 + ERD02) executed: 23 tables, 22 foreign keys
    address, apparel_detail, audit_log, bag_detail, cart, cart_item
    category, customer_order, footwear_detail, inventory_event, order_item, order_status_history
    payment, product, promotion, replenishment_request, review, sequence
    shipment, shipment_milestone, stock_level, users, watch_detail

[2] Compared with abc_shop/dao/schema.sql: 21 of 23 tables identical (columns, NOT NULL, keys, foreign keys, UNIQUE, CHECK)
    inventory_event: differs in UNIQUE (product_id, kind, reference)
    review: differs in UNIQUE (customer_id, product_id, order_id)
    4 triggers (detail row of the category's kind, BR-CAT-01) exist only in schema.sql

[3] The application seeded one month of history into the generated database:
    users=8, product=16, customer_order=16, order_item=25,
    payment=16, shipment=10, inventory_event=60, review=14
    first order: O001 of Be Quoc Khanh, 2 line(s), total 145.00, payment Succeeded, shipment DELIVERED

[4] Rules the database enforces by itself:
  rejected: order line for a product that does not exist -> FOREIGN KEY constraint failed
  rejected: second cart for the same customer            -> UNIQUE constraint failed: cart.customer_id
  rejected: more reserved than physical stock            -> CHECK constraint failed: reserved >= 0 AND reserved <= physi
  rejected: review rating 6                              -> CHECK constraint failed: rating BETWEEN 1 AND 5
  rejected: repeated idempotency key (double submit)     -> UNIQUE constraint failed: customer_order.idempotency_key
  rejected: user with an unknown role                    -> CHECK constraint failed: role IN ('customer','sales','wareho
    6 of 6 rejected
```

*Figure 32. The database created from the generated script, compared with the application schema, filled by the application and checked with statements that must fail*

Block [2] shows the one real gap between model and schema. Two rules are composite unique keys: an inventory event is recorded once per (product, kind, reference), and a customer reviews a product once per order. Visual Paradigm's ERD marks uniqueness per column, and the Community Edition has no tool for a unique index over several columns. These two keys are therefore written only in schema.sql, and the services check them before inserting (InventoryService looks up the reference first, ReviewService.exists()). Block [3] is the strongest test: 16 orders with their lines, payments, shipments, stock events and reviews were stored in tables generated from the diagram, without a single error. Block [4] shows that the database itself refuses an order line for an unknown product, a second cart, a reservation above the physical stock, a rating of 6, a double-submitted checkout and an unknown role. The four triggers of BR-CAT-01 are the second gap: an ERD has no notation for a trigger, so they too live only in schema.sql.

The menu path of Visual Paradigm Enterprise is the alternative way to produce the same database, and the steps are kept in the project's README so that it can be shown live: open ERD01 or ERD02, choose Tools > DB > Database Configuration, select SQLite with the bundled driver and the file design/abc_design_vp.db, then Tools > DB > Generate Database with "Create database" and "Export to DDL"; the DDL file it writes can be compared with create_tables.sql by build_db.py.

## 4.3. Class Diagram in Design

The design class diagram describes the classes that will be written as code. Every attribute has a type, every method has typed parameters and a return type, and every relationship says which class knows which. My starting point was the analysis model of Section 4.1 and the four steps of Test 01: add methods, replace call-only associations by use dependencies, introduce data access objects, and group the classes into packages. The first two steps are this section; the last two are Section 4.4.

### 4.3.1. What I updated

The table lists the changes from the analysis model to the design model. Most of them come from the database of Section 4.2 or from the target language, Python.

*Table 43. Updates of the design classes*

| Element | Analysis | Design | Reason |
|---|---|---|---|
| Names and types | camelCase, String, DateTime | snake_case, str, float, int, bool | The design is written for Python (PEP 8); dates travel as ISO text, as in SQLite |
| Methods | none | Typed operations in every model class | Step 1 of Test 01; code generation needs the signatures |
| Stored links to other aggregates | Associations | Directed associations named after the key field (customer_id, order_id, product_id) | These objects are loaded by different DAOs; the design keeps the key, like the table |
| Parts | Compositions | Compositions named items, history, milestones | The whole holds the list of its parts |
| Customer - Address | Aggregation | Directed association address, 0..1 | The customer object carries its address |
| Cart to Order, <<create>> | Dependency | Order.create_from_cart(cart) + <<use>> Order → Cart | A call, not a stored link |
| Cart - Product | via CartItem | <<use>> Cart → Product for add_item(product, quantity) | The method reads the product's id, name and price |
| Order uses Promotion | Association | Directed association promo_code, 0..1 | The order keeps the code it used (Section 4.2.3) |
| ReplenishmentRequest | Class | No class; a row written by InventoryDAO | It has no behaviour; a class would only carry data the DAO already returns |
| Customer.orders | - | Not in the model | Orders are found by OrderDAO.find_by_customer; the field I once had in the code was never used and was removed |

The change I thought about longest is the third row. In Step 2 of Test 01 I followed the lecturer's rule: when a method of class A takes an object of class B and uses its attributes, the design shows a <<use>> dependency from A to B instead of the association. I kept that rule for the calls, Order.create_from_cart(cart) and Cart.add_item(product). But for links that are stored, a dependency says too little: a cart must know whose cart it is, although no method

takes a Customer. So I went back to associations, now directed, with the name of the field that holds the link. A name that ends in _id says that the field holds the other object's key, not the object. That is the deliberate decision "reference by id between aggregates": Payment, Shipment and Review are loaded and saved by other DAOs and other use cases than their Order, and an object reference would force every DAO to load half of the database.

### 4.3.2. Attributes and methods

The model classes keep the attributes of the analysis, now typed, with the exceptions above. Attributes are public (+), because a Python dataclass field is read directly by the DAOs and the views and the language has no private fields. The methods come from Step 1 of Test 01, where the lecturer distinguished two kinds of method. A method of the class itself works on the class's own data, such as calc_total() in Cart. A method linked to another class takes an object of that class as a parameter and uses its attributes, such as add_item(product, quantity), which reads the product's id, name and price.

*Table 44. Methods of the model classes*

| Class | Methods of the class itself | Methods linked to another class |
|---|---|---|
| User / Customer | can(permission) | update_profile(name, address : Address) |
| Address | is_complete(), one_line() | - |
| Cart / CartItem | find, update_quantity, remove_item, calc_total, is_empty, clear / sub_total | add_item(product : Product, quantity) |
| Order / OrderItem | subtotal, calc_total, change_status, can_cancel, last_update / sub_total | create_from_cart(order_id, cart : Cart, ...) |
| Payment | matches(attempt_ref, amount, currency), is_final() | - |
| Shipment | last_seq() | add_milestone(milestone : Milestone) |
| Review | validate() | - |
| Product / Promotion | is_active, search_text, update_price / is_valid, discount_for | - |
| StockLevel | available(), is_low() | apply(event : InventoryEvent) |

Three placements are worth explaining. calc_total() is in Cart and Order while sub_total() is in CartItem and OrderItem: each line computes its own amount and the whole adds the lines, so no class reaches inside another to multiply prices; each class is the information expert for its own total. change_status() is in Order, not in a service, because the transition table of Section 2.17 is a property of the order: a service decides *when* to change a status, never *whether* a transition is legal. And apply(event) is in StockLevel, because the rule "never negative" belongs to the stock, not to whoever records the event.

The three figures show the updated design classes, split like the ERD for legibility: customers, carts and orders, then payment, shipment and review of an order, then the catalogue and the stock. A class that belongs to another figure appears with its name only, for example Product in the first figure and Order in the second and third.

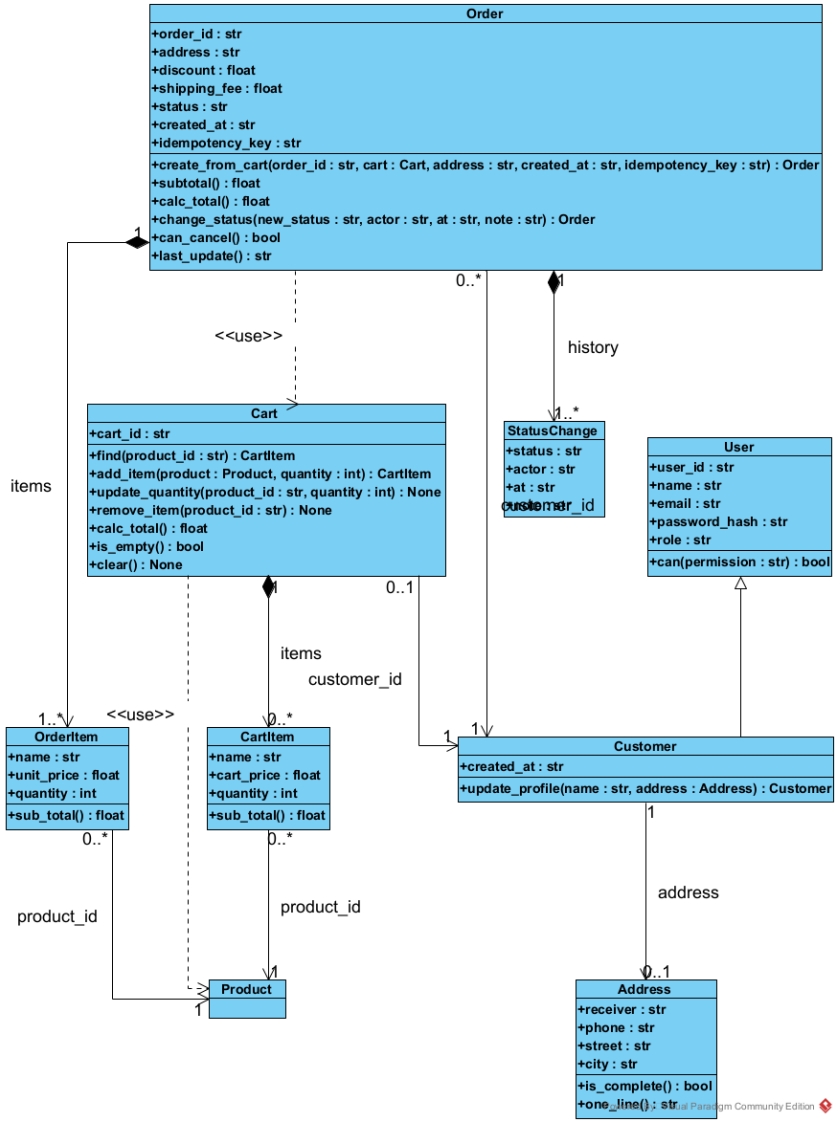



*Figure 33. Design classes, part 1: customers, carts and orders with typed attributes, methods, key fields and compositions - VP diagram "DCL01"*

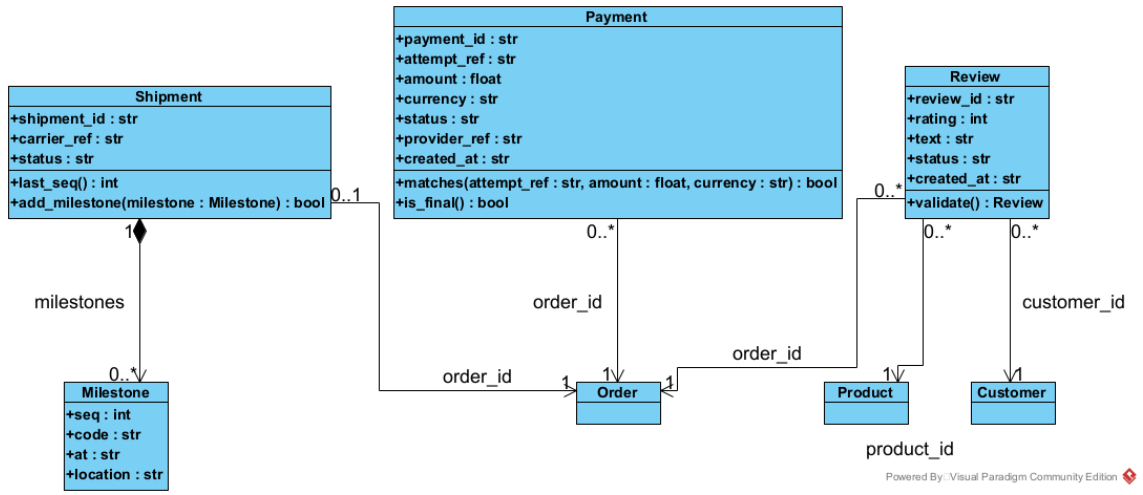



*Figure 34. Design classes, part 2: payment, shipment and review of an order - VP diagram "DCL02"*

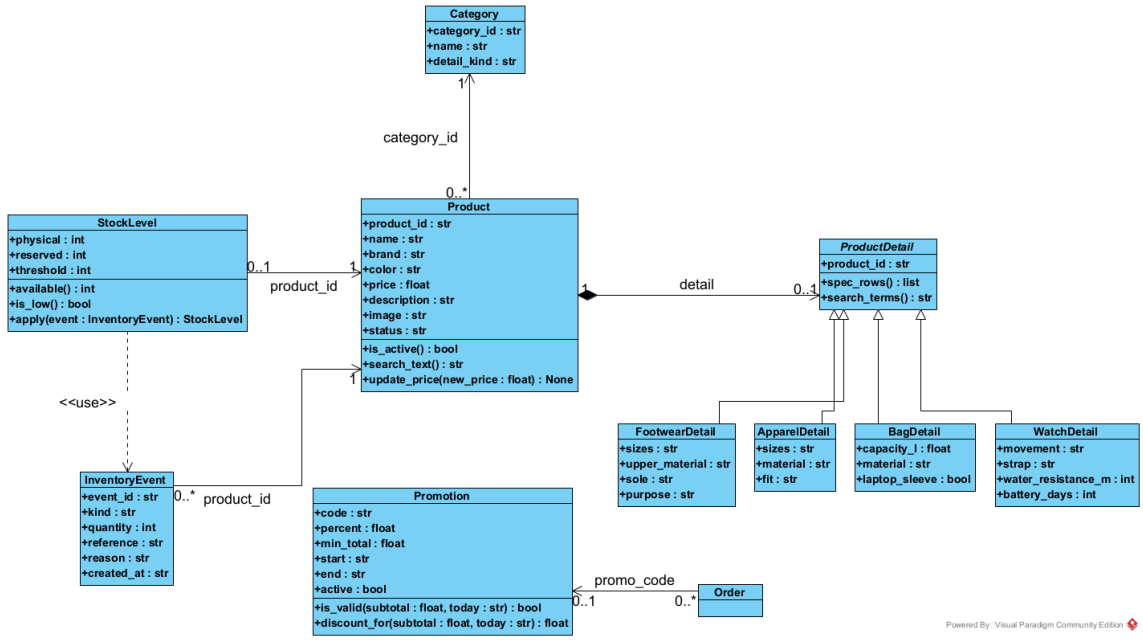



*Figure 35. Design classes, part 3: catalogue, promotion and event-based stock - VP diagram "DCL03"*

### 4.3.3. Relationships

The table explains every relationship of the design classes and how it appears in the code. Read together with the relationship table of Section 4.1.2, it shows what happened to each analysis relationship.

*Table 45. Relationships of the design classes and their form in code*

| Relationship | Kind and multiplicity | Name | Form in the code |
|---|---|---|---|
| Customer - User | Generalization | - | class Customer(User) |
| Customer → Address | Association, 1 to 0..1 | address | Field address: Optional[Address] |
| Cart, Order with their lines; Order with StatusChange; Shipment with Milestone | Composition, 1 to 0..* / 1..* | items, history, milestones | Field list[...] with default_factory=list |
| Cart, Order, Review → Customer | Association, 0..1 / 0..* to 1 | customer_id | Field customer_id: str (key) |
| CartItem, OrderItem, StockLevel, InventoryEvent, Review → Product | Association, 0..* to 1 | product_id | Field product_id: str (key) |
| Payment, Shipment, Review → Order | Association, 0..* / 0..1 to 1 | order_id | Field order_id: str (key) |
| Product → Category | Association, 0..* to 1 | category_id | Field category_id: str (key) |
| Order → Promotion | Association, 0..* to 0..1 | promo_code | Field promo_code: str (code) |
| Order, Cart, StockLevel <<use>> | Dependency | - | Parameter types of create_from_cart, add_item, apply |

For a key field the direction of the association follows the foreign key of Section 4.2: the class whose table holds the key holds the field, so Cart → Customer matches cart.customer_id and CartItem → Product matches cart_item.product_id. For a composition the direction is reversed: in the database the key sits in the part (order_item.order_id), but in the code the whole holds the list of parts, because the parts are created and read through their owner and never on their own.

## 4.4. Layers, Classes and Relationships

Web-based e-commerce systems are usually built in layers, and the lecture recommends three layers in the MVC style. I compared this with two alternatives. A single-layer design, in which each page reads and writes the database itself, is quick to start but repeats the root cause of the shop's problems: data handled in many places in many ways. Microservices (catalogue, orders, payments, search as separate services) would suit a large shop with several teams. Here they would add network calls and distributed consistency problems to a checkout that must reserve stock and create an order atomically. A **layered monolith** is the right size: one deployable application, divided inside so that each part can change alone (NFR09), and divisible into services later along the same package lines.

### 4.4.1. Layers and packages

The design keeps the view, controller, DAO and model packages of the lecture's MVC model and adds two packages. A **service** package between controller and DAO holds the business rules of each use case, because putting the checkout rules into a controller would mix HTTP handling with stock and payment logic and make both hard to test. An **ai** package sits beside the services at the same level, because the AI components of Chapter 3 are application logic that reads the same DAOs. The six packages are:

- **view**: the module templates, one function per page (home, product, cart, checkout, track, staff and report pages); it holds no rule;
- **controller**: WebApp (the router and front controller), Request and Response, and five controllers, one per group of use cases;
- **service**: ten services, one per area (auth, catalogue, cart, checkout, payment, order, inventory, shipment, review, report), and the two gateway interfaces to the providers;
- **ai**: the search components of Assignment 06 (SearchService, QueryService, SpeechService, ImageService, RankingService), ReviewInsightService and RecommenderService;
- **dao**: Database, ten DAO interfaces and their SQLite implementations (not drawn), VectorIndex and ImageStorage;
- **model**: the twenty-three design classes of Section 4.3 in eight sub-packages that follow the life cycle of the data: customer, cart, order, payment, shipment, review, product, inventory.

Dependencies run controller → view, controller → service and ai, service and ai → dao, and dao → model, plus service and ai → model, because services receive, change and return model objects. Nothing goes from view or controller to dao, nothing goes upwards, and nothing in model depends on anything. The module shop.py is the composition root, the only place that creates objects and connects the layers. It is not drawn because it depends on every package by definition. Cart and CartItem form their own sub-package, because a customer can abandon a cart without affecting any order; Order, OrderItem and StatusChange are together because they only exist after a purchase.

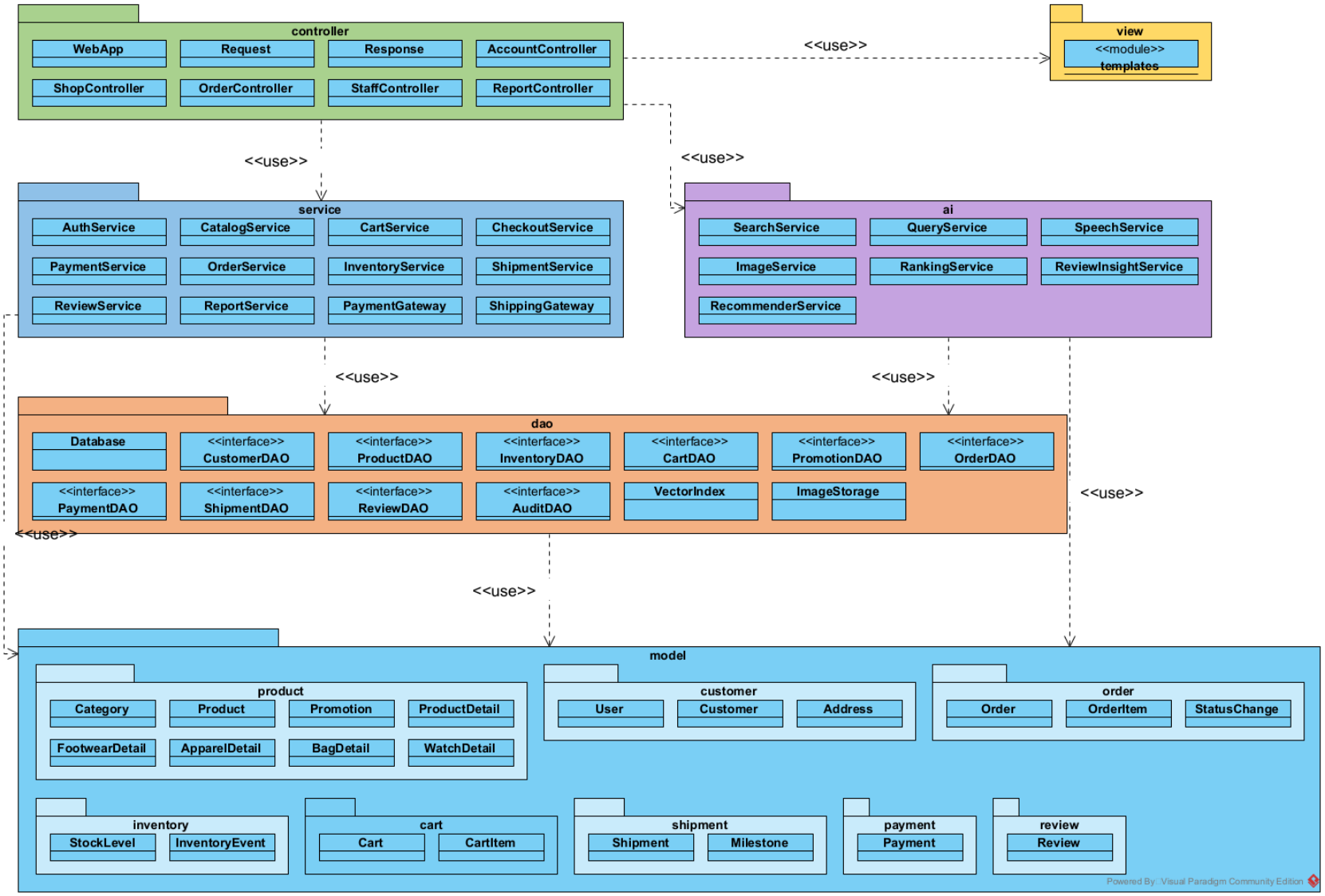



*Figure 36. Layers of the application: the six packages, their classes and the <<use>> dependencies between them - VP diagram "PKG01"*

### 4.4.2. Data access objects

The dao layer separates storage from business logic. A data access object collects every read and write for one kind of model object. It has two parts: an interface that declares *what* can be done (get_order(id), save_order(order)) and an implementation that knows *how* (SQL against the tables of Section 4.2). Services depend only on the interfaces, so the storage can change without touching them. Two decisions are worth recording. A missing record returns None or an empty list instead of raising an error, because "not found" is an expected outcome of several use cases (UC11 E2) and the service decides what it means. And all implementations share one Database object, which owns the connection and offers transaction(): a service that must change several kinds of data atomically opens one transaction, and every DAO call inside it joins it.

*Table 46. DAO interfaces of the dao layer*

| Interface | Main methods | Tables behind it |
|---|---|---|
| CustomerDAO | get_user, find_by_email, save_user, list_customers | users, address |
| ProductDAO | get_product, list_products(category_id, active_only), save_product, list_categories | product, category |
| InventoryDAO | get_stock, save_stock, find_event(product, kind, ref), add_event, add_replenishment | stock_level, inventory_event, replenishment_request |
| CartDAO | get_cart(customer_id) (created on first use), update_cart | cart, cart_item |
| PromotionDAO | get_promotion(code), save_promotion, list_promotions | promotion |
| OrderDAO | get_order, find_by_idempotency_key, find_by_customer, find_by_status, save_order | customer_order, order_item, order_status_history |
| PaymentDAO | get_by_attempt(ref), for_order, save_payment | payment |
| ShipmentDAO | get_by_order, get_by_carrier_ref, save_shipment | shipment, shipment_milestone |
| ReviewDAO, AuditDAO | for_product, exists, save_review / record, recent | review / audit_log |

### 4.4.3. One slice through the layers

A package diagram shows the layers but not the classes that talk to each other. The next figure therefore cuts one slice through all five layers: everything that takes part when a customer adds a product to the cart (UC07, with the included UC30 Check Product Availability). The templates module in view shows the cart. OrderController in controller handles the form and uses CartService. CartService uses three DAO interfaces, because it needs the cart, the product and the stock, and each DAO uses the model class it returns. The SQLite implementations are left out because they add one realization arrow per interface and no new dependency. All dashed arrows between layers point downwards, except the one from the controller to the view it renders. The same links also appear inside the class boxes, as the attributes cart_service of OrderController and carts, products and inventory of CartService. The attribute is how the object holds its collaborator in the code; the <<use>> arrow is how the layer diagram shows the dependency.

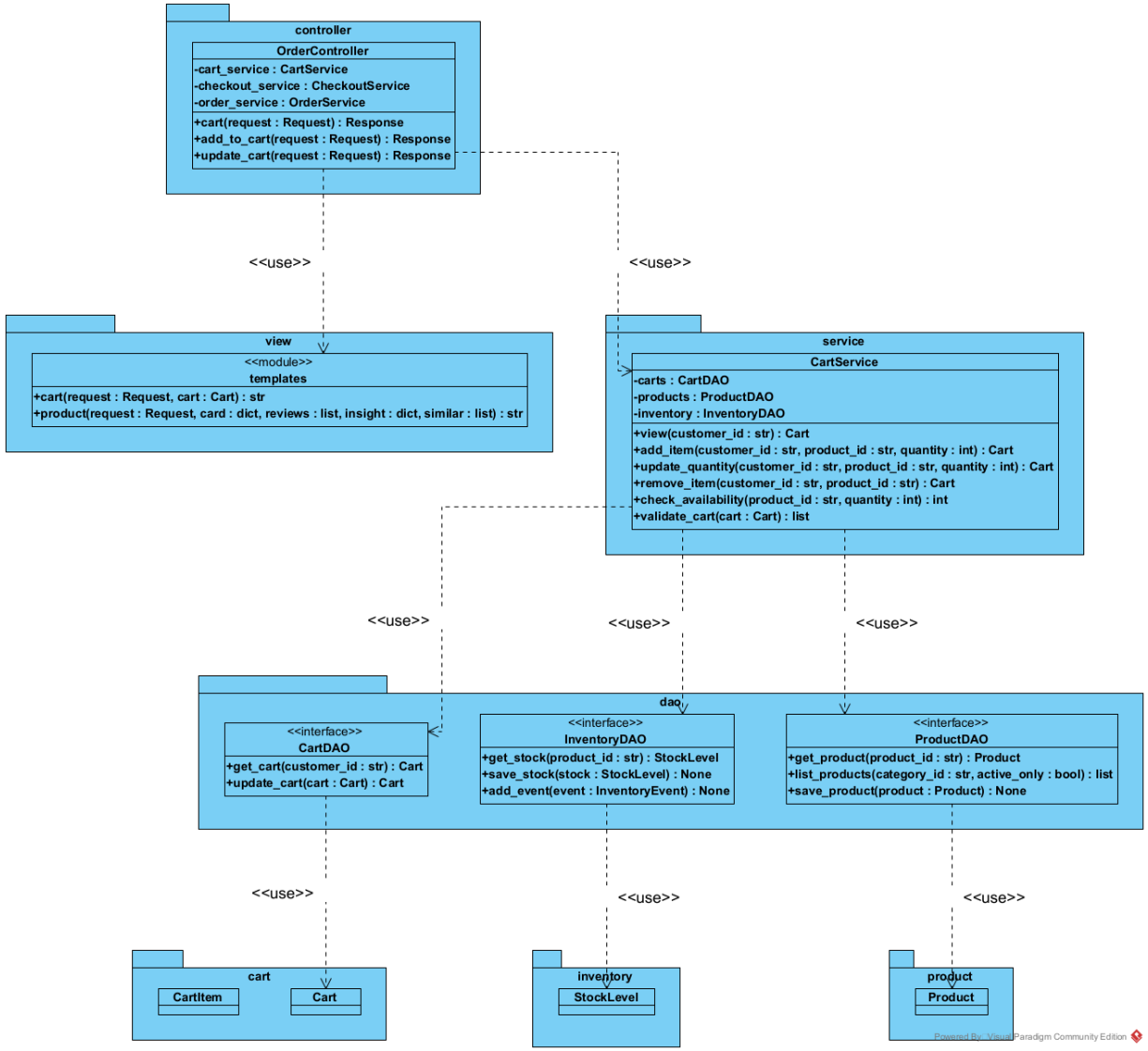



*Figure 37. One slice through the layers: the classes that take part in adding a product to the cart - VP diagram "DCL04"*

*Table 47. One request through the layers: a customer adds two units of a product to the cart*

| Step | Layer and class | What happens |
|---|---|---|
| 1 | view: templates.product | The product page posts product_id and qty to /cart/add |
| 2 | controller: WebApp, OrderController.add_to_cart | Routes the request; checks the role 'cart'; calls the service with the session customer id |
| 3 | service: CartService.add_item | Loads the product (ProductDAO) and the cart (CartDAO); checks availability with InventoryDAO (UC30) |
| 4 | model: Cart.add_item | Adds to an existing line or creates one with the current price |
| 5 | dao: CartDAO.update_cart | Stores the cart and its lines in cart and cart_item in one transaction |
| 6 | controller → view | Redirects to /cart, which templates.cart renders |

### 4.4.4. Services and controllers

Every use case has a home in one service, and every service receives its collaborators through its constructor, so a test can build it on an in-memory database or replace the payment gateway.

*Table 48. Service classes of the service layer*

| Service | Responsibility | Collaborators | Use cases |
|---|---|---|---|
| AuthService | Register, login, password hashing, role checks | CustomerDAO, AuditDAO | UC01-02 |
| CatalogService | Browse, product details with availability, product and promotion maintenance | ProductDAO, InventoryDAO, PromotionDAO | UC04, UC06, UC14, UC16 |
| CartService | Cart lines, availability, cart validation | CartDAO, ProductDAO, InventoryDAO | UC07, UC29-30 |
| CheckoutService | Quote, promotion, atomic order creation, payment request | CartDAO, OrderDAO, PromotionDAO, CartService, InventoryService, PaymentService | UC08, UC31-32 |
| PaymentService | Payment attempts, verified callbacks, failures | PaymentDAO, OrderDAO, InventoryService, PaymentGateway | UC09 |
| OrderService | History, tracking, cancellation, processing queue | OrderDAO, ShipmentDAO, PaymentService, InventoryService | UC10-12, UC19 |
| InventoryService | Idempotent stock events, low stock, replenishment | InventoryDAO, ProductDAO | UC20, UC24 |
| ShipmentService | Packing, delivery request, milestones | ShipmentDAO, OrderDAO, InventoryService, ShippingGateway | UC21-23 |
| ReviewService, ReportService | Eligibility and moderation / dashboard and reports | ReviewDAO, OrderDAO / OrderDAO, ProductDAO, InventoryDAO | UC13, UC18 / UC25-28 |

The controllers are the system's interface to the web browser. A route without a use case is either a missing requirement or not needed, so every route is listed with the use case it serves. Security (NFR05) is met by structure: the router loads the user of the session once, and every service method that changes data checks the caller's permission, so a new page cannot forget

the check. A customer's own data is always read with the customer id of the session, never with an id taken from the request.

*Table 49. Routes of the web application and the use cases they serve*

| Method and path | Controller | Use case | Role |
|---|---|---|---|
| GET /, GET /product/{id} | ShopController | UC04, UC06, UC33-34 | any |
| GET\|POST /search | ShopController | UC05 (text, voice, image) | any |
| GET\|POST /login, /register | AccountController | UC01, UC02 | any |
| GET /cart, POST /cart/add, /cart/update | OrderController | UC07 | customer |
| GET\|POST /checkout | OrderController | UC08, UC09, UC29-32 | customer |
| GET /orders, /orders/{id}, POST .../cancel, .../review | OrderController | UC10-13 | customer (own orders) |
| GET /staff/orders, POST .../{process\|hold\|pack\|ship} | StaffController | UC19, UC21, UC22 | sales, warehouse |
| GET\|POST /staff/inventory, /admin/reviews | StaffController | UC20, UC24, UC18 | warehouse, admin |
| GET /reports | ReportController | UC25-28 | manager |

Checkout is the request where layering matters most, so its design sequence diagram uses the real class and method names. Messages 3-4 implement UC08 A7: the idempotency key of the form is looked up first, and a second submission returns the existing order. Messages 8-10 run inside one database transaction (E1), and payment is requested only after the transaction has committed, so that a provider never receives a request for an order that does not exist.

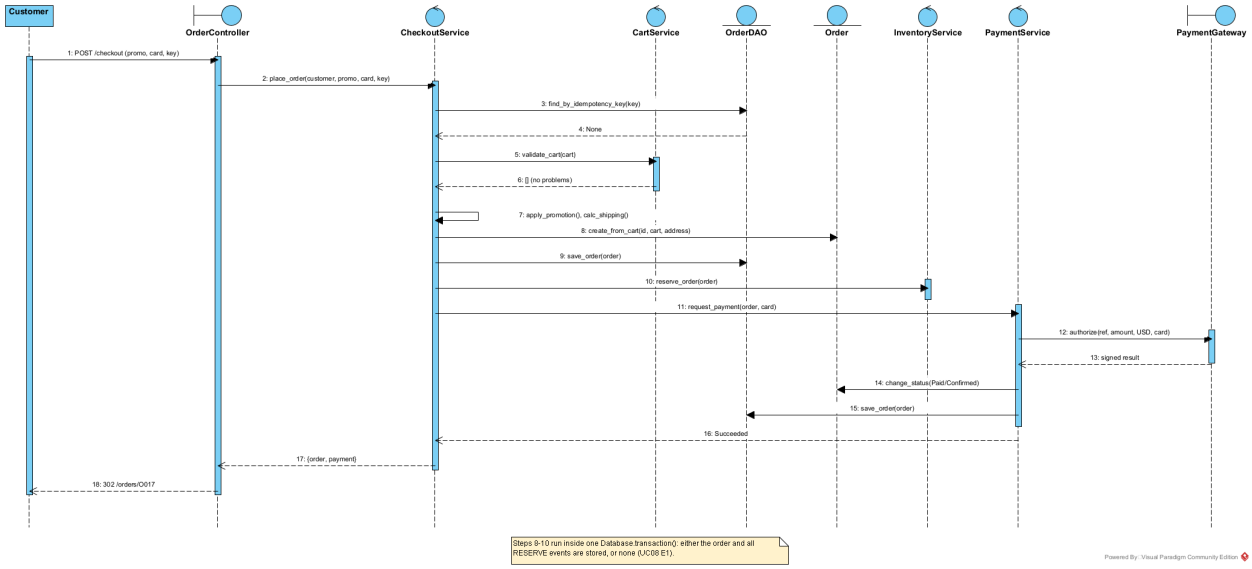



*Figure 38. Checkout through the layers - VP diagram "SD05 Checkout Through the Layers (Design)"*

## 4.5. Code Generated from the Layered Model

The last design step checks the model in the strictest way I know: turning it into code that an interpreter accepts and comparing it with the program. The Community Edition of Visual Paradigm has no code generator, and Python is not one of Visual Paradigm's target languages anyway. I therefore wrote design/generate_code.py. With --live it reads the design classes of DCL01 to DCL04 from the open project through the MCP plugin (classes, owners, attributes, operations, relationships with roles and multiplicities) and stores them in model_export.json; then it writes one Python module per model sub-package and per class of the other layers. Because the export is stored, the code can be regenerated without Visual Paradigm, and running the script again after every change of the diagrams keeps the code from drifting away from them.

### 4.5.1. From model elements to Python

The mapping rules are kept simple, so that anyone can predict the code from the diagram.

*Table 50. Mapping rules from the design model to Python*

| Model element | Python code |
|---|---|
| Layer package (view, controller, service, dao, model) | Python package (folder) of abc_shop_design |
| Sub-package of model (cart, order, ...) | Module model/cart.py, model/order.py, ... |
| Class of model | @dataclass class; every field gets a default value |
| Class of another layer | class whose __init__ receives and stores its attributes (the collaborators) |
| Class with <<interface>> (DAO) | class CartDAO(ABC) with @abstractmethod methods, collected in dao/interfaces.py |
| <<module>> (templates) | module-level functions |
| Generalization Customer → User | class Customer(User) and an import of the base class |
| Attribute +name : Type | field name: Type |
| Navigable association end with a name ending in _id / _code | field holding the key: customer_id: str # key of Customer [1] |
| Composition or association with a many end | field items: list[CartItem] = field(default_factory=list) |
| Association with a 0..1 end | field address: Optional[Address] = None |
| Operation +name(p : T) : R | def name(self, p: T) -> R: raise NotImplementedError |
| <<use>> dependency and types in signatures | import of the supplier under TYPE_CHECKING (no import cycle at run time) |

### 4.5.2. Packages and classes

The generated project has five packages with twelve modules and twenty-eight classes. The program design/check_code.py checks the result. It lists the packages and classes, compiles and imports every module, and then compares each generated class with the hand-written class of the same name in abc_shop/: every field and every method of the model must exist in the code. Finally it prints the members of one generated class.

```
> python check_code.py

[1] Packages and classes generated from the model (root abc_shop_design)
  view.templates          templates (module)
  controller.order_controller OrderController
  service.cart_service    CartService
  dao.interfaces          CartDAO, InventoryDAO, ProductDAO
  model.cart              Cart, CartItem
  model.customer          Address, User, Customer
  model.inventory         InventoryEvent, StockLevel
  model.order             Order, OrderItem, StatusChange
  model.payment           Payment
  model.product           Category, Product, ProductDetail, Promotion, ApparelDetail, BagDetail, FootwearDetail, WatchDetail
  model.review            Review
  model.shipment          Milestone, Shipment

[2] py_compile and import of 12 modules (28 classes): no errors

[3] Design model against the hand-written code in abc_shop/
  class            module                       fields  methods  difference
  OrderController  controller.order_controller   3/3     3/3     -
  CartDAO          dao.interfaces                0/0     2/2     -
  InventoryDAO     dao.interfaces                0/0     3/3     -
  ProductDAO       dao.interfaces                0/0     3/3     -
  Cart             model.cart                    3/3     7/7     -
  CartItem         model.cart                    4/4     1/1     -
  Address          model.customer                4/4     2/2     -
  Customer         model.customer                7/7     2/2     -
  User             model.customer                5/5     1/1     -
  InventoryEvent   model.inventory               7/7     0/0     -
  StockLevel       model.inventory               4/4     3/3     -
  Order            model.order                  11/11    6/6     -
  OrderItem        model.order                   4/4     1/1     -
  StatusChange     model.order                   4/4     0/0     -
  Payment          model.payment                 8/8     2/2     -
  ApparelDetail    model.product                 4/4     2/2     -
  BagDetail        model.product                 4/4     2/2     -
  Category         model.product                 3/3     0/0     -
  FootwearDetail   model.product                 5/5     2/2     -
  Product          model.product                10/10    3/3     -
  ProductDetail    model.product                 1/1     2/2     -
  Promotion        model.product                 6/6     2/2     -
  WatchDetail      model.product                 5/5     2/2     -
  Review           model.review                  8/8     1/1     -
  Milestone        model.shipment                4/4     0/0     -
  Shipment         model.shipment                5/5     2/2     -
  CartService      service.cart_service          3/3     6/6     -
  templates        view.templates                0/0     2/2     -
  28 of 28 classes: every field and method of the model exists in the code

[4] Members of the generated class model.order.Order
  field  order_id: str
  field  address: str
  field  discount: float
  field  shipping_fee: float
  field  status: str
  field  created_at: str
  field  idempotency_key: str
  field  items: list[OrderItem]
  field  history: list[StatusChange]
  field  customer_id: str
  field  promo_code: str
  method create_from_cart(self, order_id: 'str', cart: 'Cart', address: 'str', created_at: 'str', idempotency_key: 'str') -> 'Order'
  method subtotal(self) -> 'float'
  method calc_total(self) -> 'float'
  method change_status(self, new_status: 'str', actor: 'str', at: 'str', note: 'str') -> 'Order'
  method can_cancel(self) -> 'bool'
  method last_update(self) -> 'str'
```

*Figure 39. Packages and classes generated from the model, compiled, imported and compared with the hand-written application*

Block [3] is the point of the exercise: all 28 classes of the model exist in the application with every field and method the diagram promises. Getting there was the best review of the model I have done. The first comparison found a field orders in the hand-written Customer that no diagram had and no code used; I removed it from the code rather than adding it to the model. It also showed that StockLevel.available is a property in the code and a method in the model, a

Python detail that the comparison accepts. And it found that the package diagram could not be read by the generator, because the plugin's relationship export works only on class diagrams. Package ownership is therefore read from the owner of each class on the class diagrams, which is the same information.

### 4.5.3. Class code

The generated classes show the relationships of Section 4.3 in their plainest form. The module of the cart sub-package comes first. The composition with CartItem becomes the list items, the association with Customer the key field customer_id, and the <<use>> dependency on Product an import for type checking only, because add_item(product) needs the type but Cart stores no product:

```
"""model.cart - generated from the Visual Paradigm design model; do not edit by hand."""
from __future__ import annotations

from dataclasses import dataclass, field
from typing import TYPE_CHECKING

if TYPE_CHECKING:                       # <<use>> and the types in the signatures
    from model.product import Product


@dataclass
class Cart:
    cart_id: str = ""
    items: list[CartItem] = field(default_factory=list)    # composition 0..*
    customer_id: str = ""    # key of Customer [1]

    def find(self, product_id: str) -> CartItem:
        raise NotImplementedError

    def add_item(self, product: Product, quantity: int) -> CartItem:
        raise NotImplementedError

    # ... 5 more methods in the same form


@dataclass
class CartItem:
    name: str = ""
    cart_price: float = 0.0
    quantity: int = 0
    product_id: str = ""    # key of Product [1]

    def sub_total(self) -> float:
        raise NotImplementedError
```

Generalization is the base class of the generated class; the association with Address is an optional field, because the multiplicity is 0..1:

```
@dataclass
class Customer(User):
    created_at: str = ""
    address: Optional[Address] = None    # association 0..1

    def update_profile(self, name: str, address: Address) -> Customer:
        raise NotImplementedError
```

Above the model, a DAO interface becomes an abstract base class, and the service holds the three DAO interfaces of the slice as the attributes shown in its class box in DCL04:

```
class CartDAO(ABC):
```

```
    @abstractmethod
    def get_cart(self, customer_id: str) -> Cart:
        ...

    @abstractmethod
    def update_cart(self, cart: Cart) -> Cart:
        ...


class CartService:
    def __init__(self, carts: CartDAO, products: ProductDAO, inventory: InventoryDAO):
        self.carts = carts
        self.products = products
        self.inventory = inventory

    def view(self, customer_id: str) -> Cart:
        raise NotImplementedError

    # ... 5 more methods in the same form
```

### 4.5.4. What the generated code does and does not contain

The generated code contains the whole structure: packages, modules, classes, fields, signatures, inheritance and the key fields of every stored link. It does not contain behaviour; every method body raises NotImplementedError. That is the natural limit of generating code from a class diagram, which describes what exists and not what happens. The behaviour comes from the use case specifications, the activity and sequence diagrams and the state machine, and Chapter 5 describes how it was written. Two smaller gaps are intended. The DAO implementations need SQL that matches the tables of Section 4.2, which is written by hand against the generated schema. And a composition and an association look the same in Python; the comment after the field is the only trace of the difference, which shows in behaviour: deleting a cart also deletes its lines (ON DELETE CASCADE in schema.sql).

## 4.6. Conclusion

The design started from twenty-three analysis classes and ended with a database of twenty-three tables and twenty-two foreign keys, generated from the physical model, and with twenty-eight Python classes in five layers generated from the class diagrams, all of which exist in the running application. Each step could be checked against the one before it. The tables were checked against the classes through the mapping rules, the design classes against the tables through the direction of the key fields, and the layers against the use cases through the request of Section 4.4.3. The code was checked against the diagrams by check_code.py, and the database by build_db.py. The table shows how the design serves the non-functional requirements of Chapter 1.

*Table 51. Non-functional requirements and the parts of the design that serve them*

| Requirement | Part of the design |
|---|---|
| NFR09 Maintainability | Layers with downward dependencies; services depend on DAO interfaces, not on SQLite; a test enforces the layer rule |
| NFR05 Access control | Permission checks in the services; customer data read with the session's customer id |
| NFR06 Auditability | audit_log table and AuditDAO for price, stock, status and moderation changes |
| NFR07 Integrity | Idempotency key, unique attempt reference, unique stock event reference, milestone sequence; CHECK and UNIQUE rules generated from the ERD |
| NFR10 Privacy | Payment stores status and references only; the provider handles card data |
| NFR01-02 Performance | Read-only queries for browsing; the vector index is kept in memory and rebuilt only when the catalogue changes |
| AI-N1 Replaceable AI | AI components in their own package behind fixed methods: transcribe, encode, search, analyse, similar |

The table below records the main decisions in a compact form inspired by architecture decision records: the decision, the alternative I rejected and the price I accepted. The third column matters most to me, because it shows what each decision gives up.

*Table 52. Design decisions, rejected alternatives and accepted costs*

| Decision | Rejected alternative | Accepted cost |
|---|---|---|
| Layered monolith, MVC + service + DAO | Microservices | Parts cannot be scaled separately |
| Service layer between controller and DAO | Rules in controllers | One more class per area |
| Reference by key between aggregates | Object references everywhere | Extra DAO calls to load related data |
| One users table for all roles | One table per role | Role-specific columns would need nullable columns |
| Order owns its transition table | Status checks in each service | Model and diagram must change together |
| Stock as typed events | Stock as one editable number | More rows; the balance is recalculated |
| SQL and Python generated from the model | Typing them by hand | Two plugin scripts to maintain |
| Rules in database and code | Rules only in code | Rules written twice |

Testability follows from the same structure: every service depends on DAO interfaces and receives them through its constructor, so a test can build the whole shop on an in-memory database or give it a payment gateway that times out. The next step of the lecturer's plan is the evaluation of the code, and the generated classes are its starting point. Chapter 5 shows how the method bodies were filled in, and how far the running code still matches this model.

# CHAPTER 5. IMPLEMENTATION AND DEMONSTRATION

The last activity tests every earlier decision against reality. A requirement can be ambiguous and a diagram can be inconsistent without anyone noticing; a program either runs or it does not, and its tests either pass or fail. This chapter presents the application **abc_shop**, written for this essay on top of the Assignment 06 search prototype. It shows how the code follows the design, how it was run and tested, and what writing it taught about the earlier models.

## 5.1. Implementation Strategy and Environment

The application implements the Release 1 transaction path end to end (account, catalogue, cart, checkout, payment, processing, packing, delivery, tracking, cancellation, reviews, inventory, reports) and the three AI components, rather than every screen of all 32 use cases. A vertical slice through all layers is a better test of the architecture than a complete but shallow user interface: if a checkout can travel from a web form through the controller, the service and the DAO to the database and back, the remaining use cases follow the same path. Python was chosen to keep the Assignment 06 code without translation, and only the standard library plus numpy and Pillow are used, so the program runs offline on any machine with Python 3.9 or later.

*Table 53. Environment and size of the code base*

| Item | Value |
|---|---|
| Language and libraries | Python 3.11 (tested), standard library: sqlite3, wsgiref, hashlib, hmac; numpy, Pillow; pytest for tests |
| Database | SQLite file data/abc_shop.db (or in memory for tests and the demo) |
| Web server | wsgiref (standard library WSGI server); pages rendered by view/templates.py |
| Application code | 3778 lines: model 538, dao 658, service 821, ai 424, controller 452, view 278, entry points 607 |
| Tests | 498 lines in 8 modules; 45 test cases, all passing |
| Repository | github.com/qkhanhbe/ABC-Online-Shop-Information-System |

## 5.2. Project Structure

The folders are the packages of the package diagram PKG01, one to one. A reader of the package diagram can find any class in the code by its package name, and a reader of the code can find any file in the diagram.

```
abc_shop/
  main.py             entry point: seed | serve | demo | search
  shop.py             composition root (creates and wires every object)
  seed.py             catalogue, accounts, October history played through the services
  evaluate.py         evaluation of the three AI components
  model/              customer, product, cart, order, payment, shipment, inventory, review
  dao/                schema.sql, database.py, interfaces.py, sqlite_dao.py, vector_index.py, im
  service/            auth, catalog, cart, checkout, payment, order, inventory, shipment, review
  ai/                 search/ (query, speech, image, search, ranking), review_insights.py, recom
  controller/         http.py, router.py, account/shop/order/staff/report controllers
  view/               templates.py, static/style.css
  data/               seed.json, reviews_labeled.json, images/, query_images/
  tests/              8 test modules, 45 test cases
```

## 5.3. The Model

The model classes are Python dataclasses with the methods of Step 1. The order state machine of Section 2.17 is a dictionary in model/order.py; change_status is the only way to change a status, it appends to the history and rejects every transition that is not in the table:

```
def change_status(self, new_status, actor, at, note=""):
    if new_status not in TRANSITIONS.get(self.status, set()):
        allowed = ", ".join(sorted(TRANSITIONS.get(self.status, set()))) or "none"
        raise InvalidTransition(
            f"{self.order_id}: {self.status} -> {new_status} is not allowed (next: {allowed})")
    self.status = new_status
    self.history.append(StatusChange(new_status, actor, at, note))
    return self
```

The stock rule of UC20 lives in StockLevel.apply. It works on local copies and assigns the new quantities only after every check has passed, so a rejected event leaves the stock exactly as it was (minimal guarantee of UC20):

```
def apply(self, event):
    """Apply one event; raise StockError and leave the level unchanged if it is invalid."""
    q = event.quantity
    physical, reserved = self.physical, self.reserved
    if event.kind == "RECEIPT":
        physical += q
    elif event.kind == "RESERVE":
        if q > self.available:
            raise StockError(f"only {self.available} available for {self.product_id}")
        reserved += q
    elif event.kind == "RELEASE":
        if q > reserved:
            raise StockError("release exceeds reservation")
        reserved -= q
    elif event.kind == "ISSUE":
        if q > reserved:
            raise StockError("issue exceeds reservation")
        physical -= q
        reserved -= q
    elif event.kind == "ADJUST":          # signed correction with a reason code
        if not event.reason:
            raise StockError("an adjustment needs a reason")
        physical += q
    else:
        raise StockError(f"unknown event kind {event.kind}")
    if event.kind != "ADJUST" and q <= 0:
        raise StockError("quantity must be positive")
    if physical < 0 or reserved < 0 or physical - reserved < 0:
        raise StockError("resulting balance would be negative")
    self.physical, self.reserved = physical, reserved
    return self
```

## 5.4. Data Access Objects

The interfaces in dao/interfaces.py are abstract classes; the services receive them through their constructors and never import the SQLite module (an architecture test checks this). The order DAO shows how the append-only history of the analysis model reaches the database: an order is upserted, its lines are written once, and only history entries that are not stored yet are inserted.

```
def save_order(self, o):
    """Insert or update; status history rows are appended, never rewritten."""
    with self.db.transaction():
        self.db.execute(
```

```
            "INSERT INTO customer_order(order_id, customer_id, address, discount, shipping_fee, "
            "total, status, created_at, promo_code, idempotency_key) "
            "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?) ON CONFLICT(order_id) DO UPDATE SET "
            "status = excluded.status",
            (o.order_id, o.customer_id, o.address, o.discount, o.shipping_fee, o.calc_total(),
             o.status, o.created_at, o.promo_code, o.idempotency_key or None))
        if not self.db.query_one("SELECT 1 FROM order_item WHERE order_id = ?", (o.order_id,)):
            for i in o.items:
                self.db.execute("INSERT INTO order_item(order_id, product_id, name, unit_price, "
                                "quantity) VALUES (?, ?, ?, ?, ?)",
                                (o.order_id, i.product_id, i.name, i.unit_price, i.quantity))
        stored = self.db.query_one("SELECT COUNT(*) AS n FROM order_status_history "
                                   "WHERE order_id = ?", (o.order_id,))["n"]
        for h in o.history[stored:]:
            self.db.execute("INSERT INTO order_status_history(order_id, status, actor, at, note) "
                            "VALUES (?, ?, ?, ?, ?)", (o.order_id, h.status, h.actor, h.at, h.not
    return o
```

Database.transaction() is a context manager that supports nesting: the outermost call opens BEGIN IMMEDIATE and commits or rolls back, inner calls join it. This let each DAO method protect its own multi-statement writes while a service can still make several DAO calls atomic.

```
@contextmanager
def transaction(self):
    """Nested calls join the outer transaction; the outermost one commits or rolls back."""
    if self._depth == 0:
        self.conn.execute("BEGIN IMMEDIATE")
    self._depth += 1
    try:
        yield self.conn
    except Exception:
        self._depth -= 1
        if self._depth == 0:
            self.conn.execute("ROLLBACK")
        raise
    else:
        self._depth -= 1
        if self._depth == 0:
            self.conn.execute("COMMIT")
```

## 5.5. Services: Checkout and Payment

CheckoutService.place_order is UC08 in code; the comments carry the step and flow numbers of the specification in Section 2.6. The idempotency key comes from a hidden field of the checkout form, so pressing the button twice returns the first order (A7).

```
def place_order(self, customer, promo_code="", card_token="tok_visa", idempotency_key=None):
    idempotency_key = idempotency_key or uuid.uuid4().hex
    existing = self.orders.find_by_idempotency_key(idempotency_key)
    if existing:                                                    # A7 repeated submission
        return {"order": existing, "payment": existing.status, "duplicate": True}

    if customer.address is None or not customer.address.is_complete():
        raise CheckoutError(["delivery address is incomplete"])     # A3
    quote = self.quote(customer.user_id, promo_code)
    if quote["problems"]:
        raise CheckoutError(quote["problems"])                      # A1, A2
    cart = quote["cart"]

    # step 6: reserve stock and create one Payment Pending order - atomically (E1)
    with self.db.transaction():
        order = Order.create_from_cart(
            self.db.next_id("order", "O"), cart, customer.address.one_line(), self.db.now(),
            idempotency_key, quote["discount"], quote["shipping"],
```

```
            promo_code.upper() if quote["discount"] else "")
        self.orders.save_order(order)
        self.inventory.reserve_order(order)

    # steps 7-8: UC09 Make Payment
    status = self.payments.request_payment(order, card_token)
    if status == SUCCEEDED:
        cart.clear()
        self.carts.update_cart(cart)                                  # cart cleared only when pai
    return {"order": self.orders.get_order(order.order_id), "payment": status,
            "duplicate": False}
```

PaymentService.handle_result is the provider callback of UC09. The order of the checks is the order of the exception flows: unknown or unsigned result (E2), mismatching amount or currency (E1), already final (A3), then success or decline, with a late success on a closed order turned into a refund exception (A4).

```
def handle_result(self, result):
    """Provider callback. Returns the payment status after the result is applied."""
    payment = self.payments.get_by_attempt(result.get("attempt_ref", ""))
    expected = sign(result.get("attempt_ref"), result.get("status"), result.get("provider_ref"),
                    result.get("amount"), result.get("currency"))
    if payment is None or result.get("signature") != expected:
        self.audit.record("payment", "rejected_callback", "payment",
                          str(result.get("attempt_ref")), "unknown or unsigned result")
        return "Rejected"                                     # E2: no state change
    if not payment.matches(result["attempt_ref"], result["amount"], result["currency"]):
        self.audit.record("payment", "quarantined", "payment", payment.attempt_ref,
                          "amount or currency mismatch")
        return "Quarantined"                                  # E1: no state change
    if payment.is_final():
        return payment.status                                 # A3: duplicate, no second effect

    order = self.orders.get_order(payment.order_id)
    with self.db.transaction():
        payment.provider_ref = result["provider_ref"]
        if result["status"] == "SUCCEEDED":
            payment.status = SUCCEEDED
            if order.status == states.PAYMENT_PENDING:
                order.change_status(states.PAID, "payment-provider", self.db.now(),
                                    f"paid {payment.amount} {payment.currency} ref {payment.provi
            else:                                             # A4 late success on a closed order
                self.audit.record("payment", "refund_needed", "order", order.order_id,
                                  "late payment success")
        else:
            payment.status = FAILED
            if order.status == states.PAYMENT_PENDING:
                order.change_status(states.PAYMENT_FAILED, "payment-provider", self.db.now(),
                                    "payment declined")
                self.inventory.release_order(order)
        self.payments.save_payment(payment)
        self.orders.save_order(order)
    return payment.status
```

## 5.6. Controllers and Views

The web layer uses only the standard library. WebApp in controller/router.py is a WSGI application with a table of routes (Section 4.4.4). dispatch() loads the user of the session, calls the controller method and turns service errors into readable pages: NotFound becomes 404, PermissionDenied a redirect to the login page or 403, any other ServiceError a 400 page with the rule that was violated.

```
def dispatch(self, request):
    if request.path.startswith(("/static/", "/images/")):
        return self.static(request.path)
    user = self.shop.dao.customers.get_user(request.session.get("user_id", ""))
    request.user = user
    for method, pattern, handler in self.routes:
        match = re.fullmatch(pattern, request.path)
        if match and method == request.method:
            try:
                return handler(request, **match.groupdict())
            except NotFound as e:
                return Response(templates.message(request, "Not found", str(e)), 404)
            except PermissionDenied as e:
                if user is None:
                    return Response.redirect("/login", "Please log in first.", request)
                return Response(templates.message(request, "Not permitted", str(e)), 403)
            except ServiceError as e:
                return Response(templates.message(request, "Request rejected", str(e)), 400)
            except Exception:                       # pragma: no cover - last resort
                traceback.print_exc()
                return Response(templates.message(request, "Error", "Unexpected error."), 500)
    return Response(templates.message(request, "Not found", request.path), 404)
```

Controllers stay thin. OrderController.checkout reads the form, calls one service method and redirects; it touches no DAO and calculates nothing:

```
def checkout(self, request):
    customer = self._customer(request)
    try:
        result = self.checkout_service.place_order(customer, request.get("promo"),
                                                   request.get("card", "tok_visa"),
                                                   request.get("key") or None)
    except CheckoutError as e:
        return Response.redirect("/cart", "Checkout stopped: " + str(e), request)
    request.session.pop("checkout_key", None)
    order = result["order"]
    self.recommender.fit()
    return Response.redirect(f"/orders/{order.order_id}",
                             f"Order {order.order_id}: {order.status}.", request)
```

The screenshots below were taken from the running application (python main.py serve) with the seeded data. The catalogue reads price and availability from the same records as the checkout; the product page combines the catalogue with the two AI sections of Chapter 3.

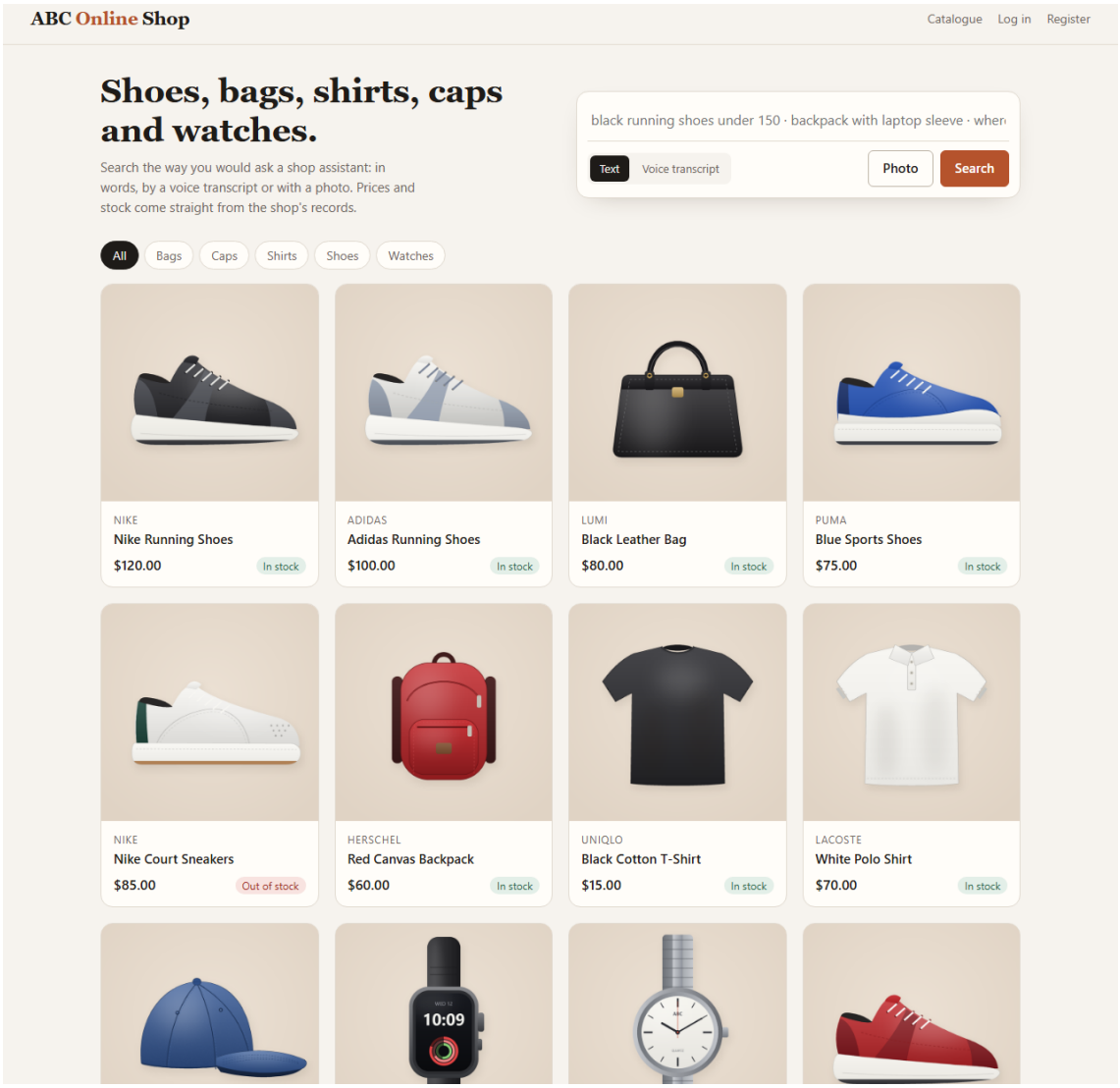



*Figure 40. Catalogue page with the multimodal search box (UC04, UC05)*

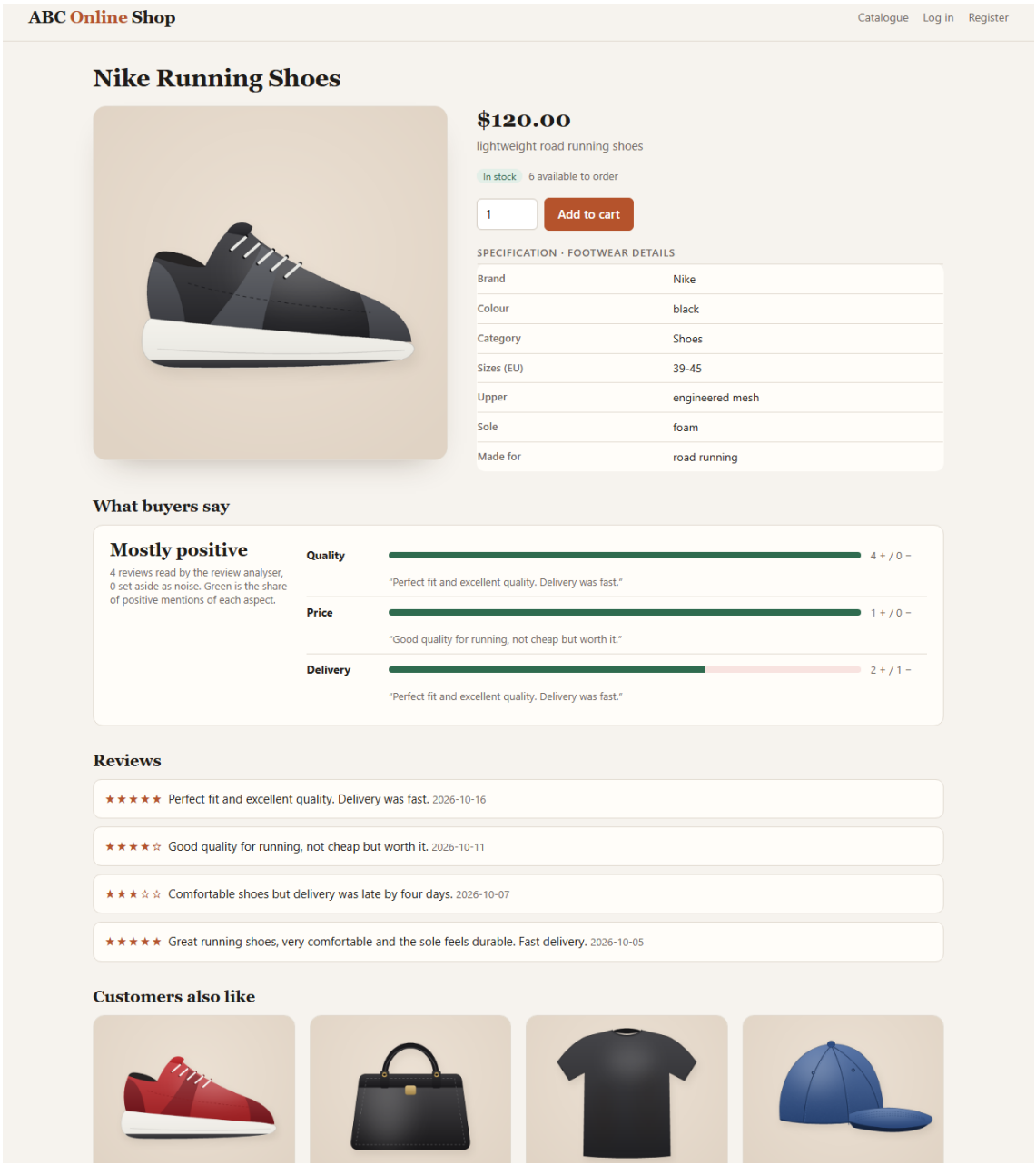



*Figure 41. Product page with AI review insights and recommendations (UC06, UC33, UC34)*

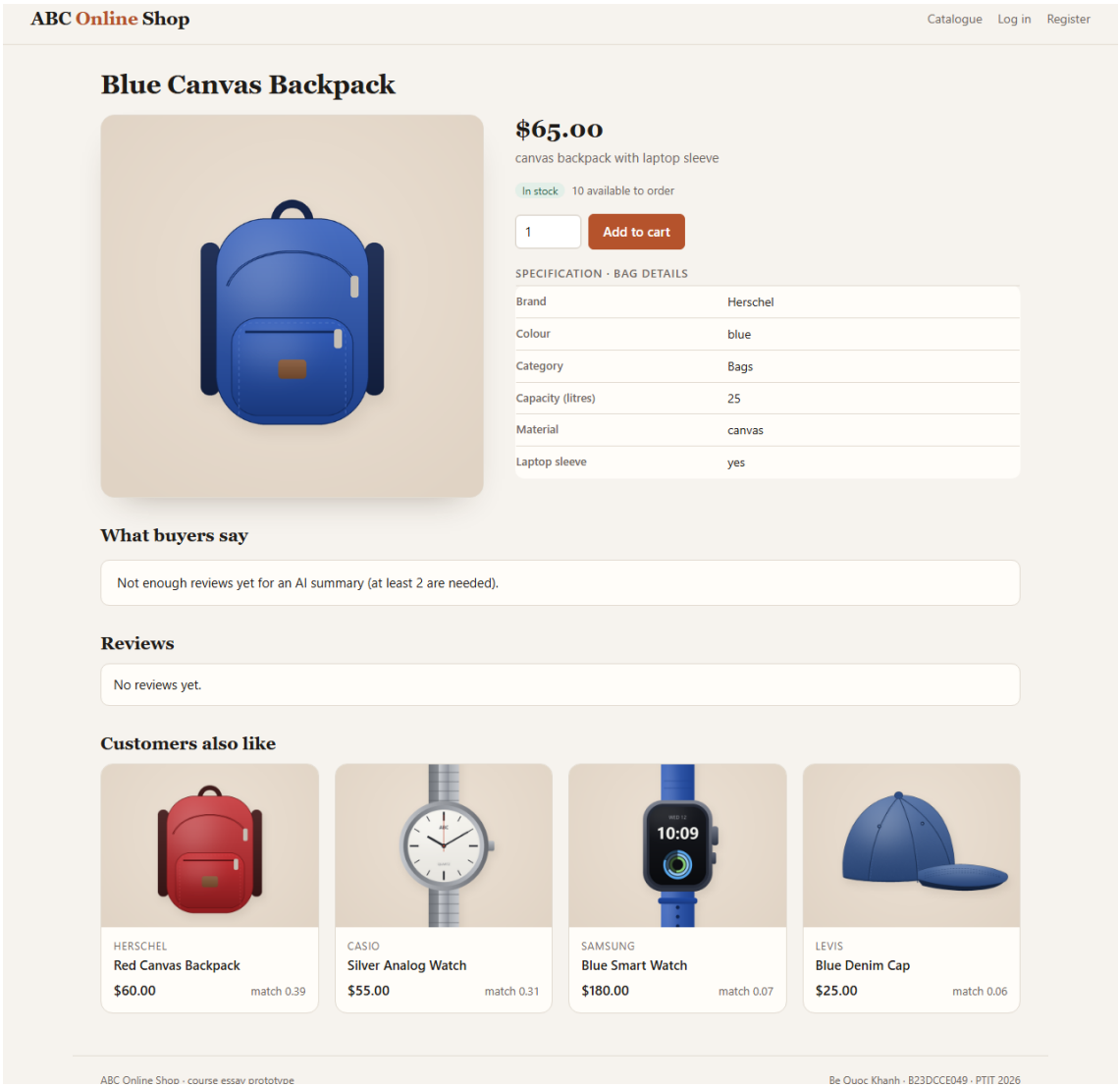



*Figure 42. Product page of a bag: the specification table comes from bag_detail, the detail table named by the category (UC06, BR-CAT-01)*

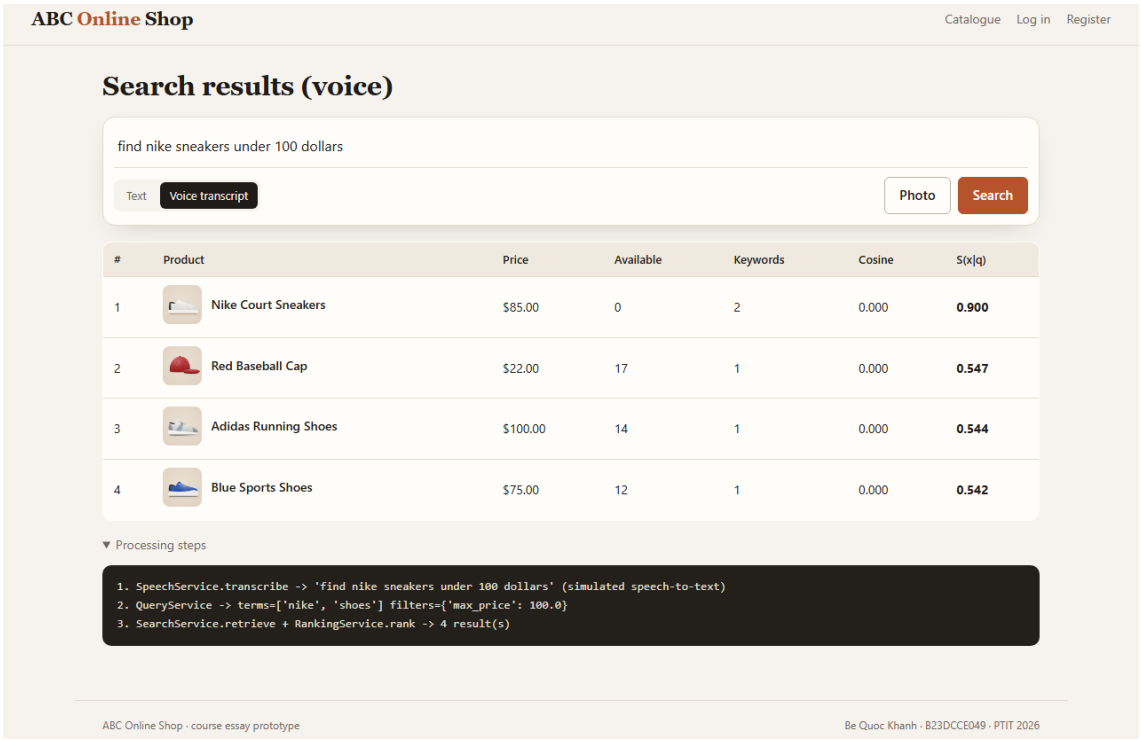



*Figure 43. Voice search result with scores and the processing trace (UC05)*

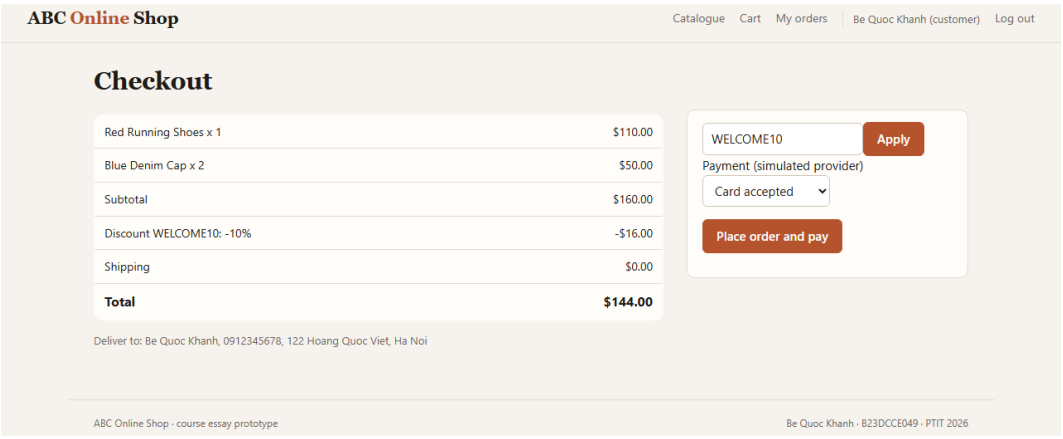



*Figure 44. Checkout with promotion WELCOME10, shipping and simulated payment options (UC08, UC31, UC32)*

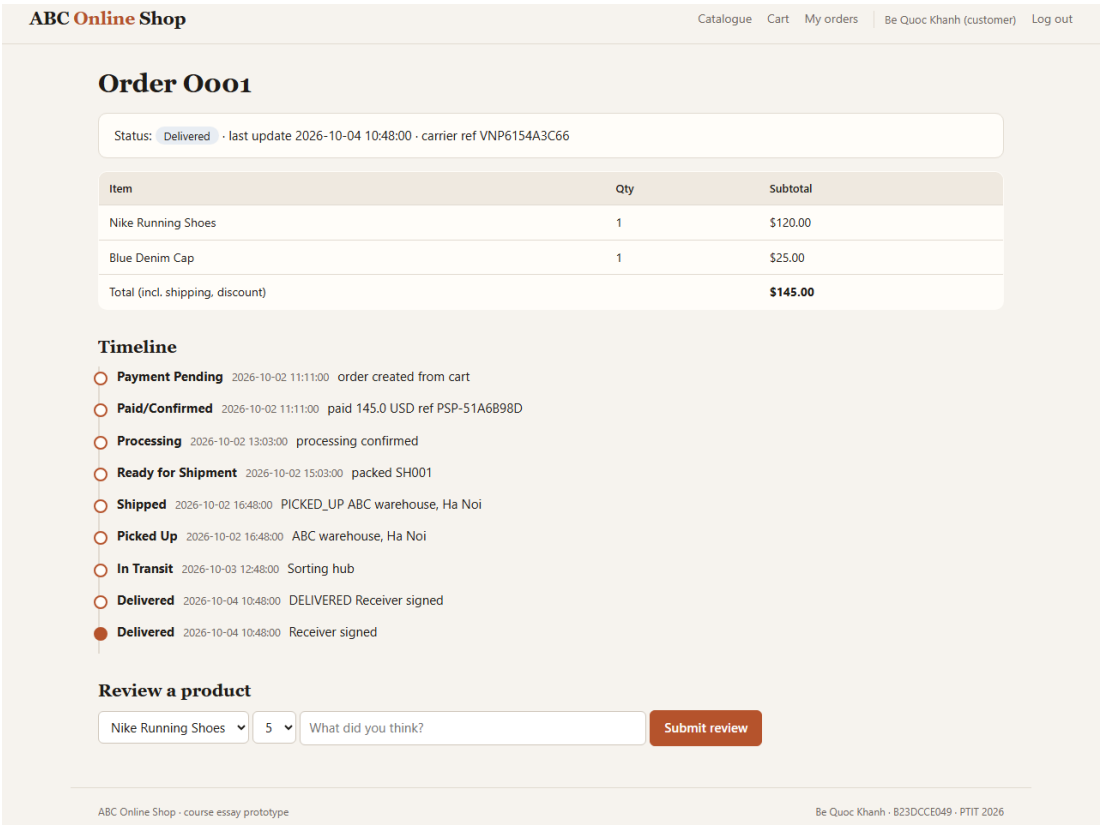



*Figure 45. Order tracking: status timeline from order history and carrier milestones (UC11)*

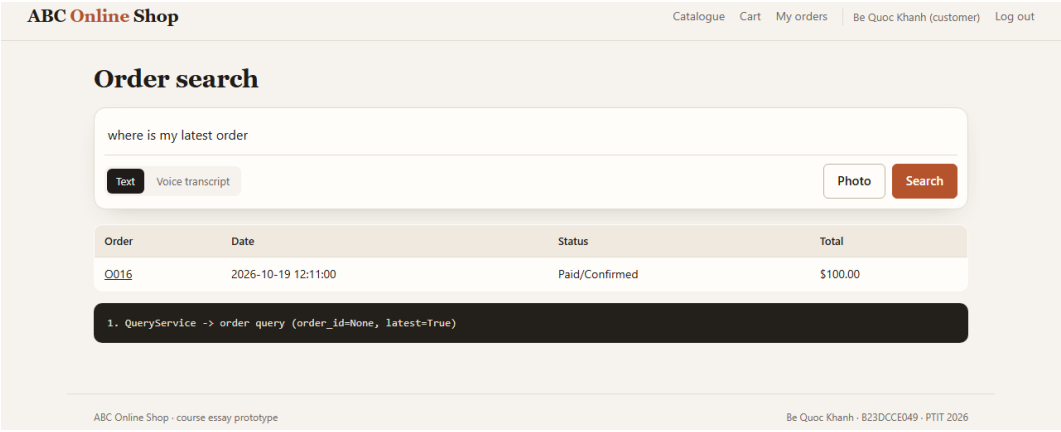



*Figure 46. Order question "where is my latest order" answered from the customer's own orders (AI-R2)*

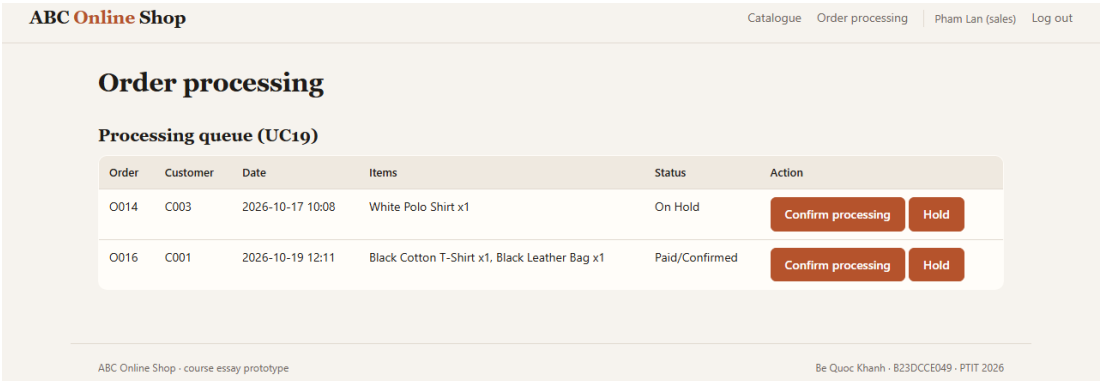



*Figure 47. Sales staff processing queue (UC19)*

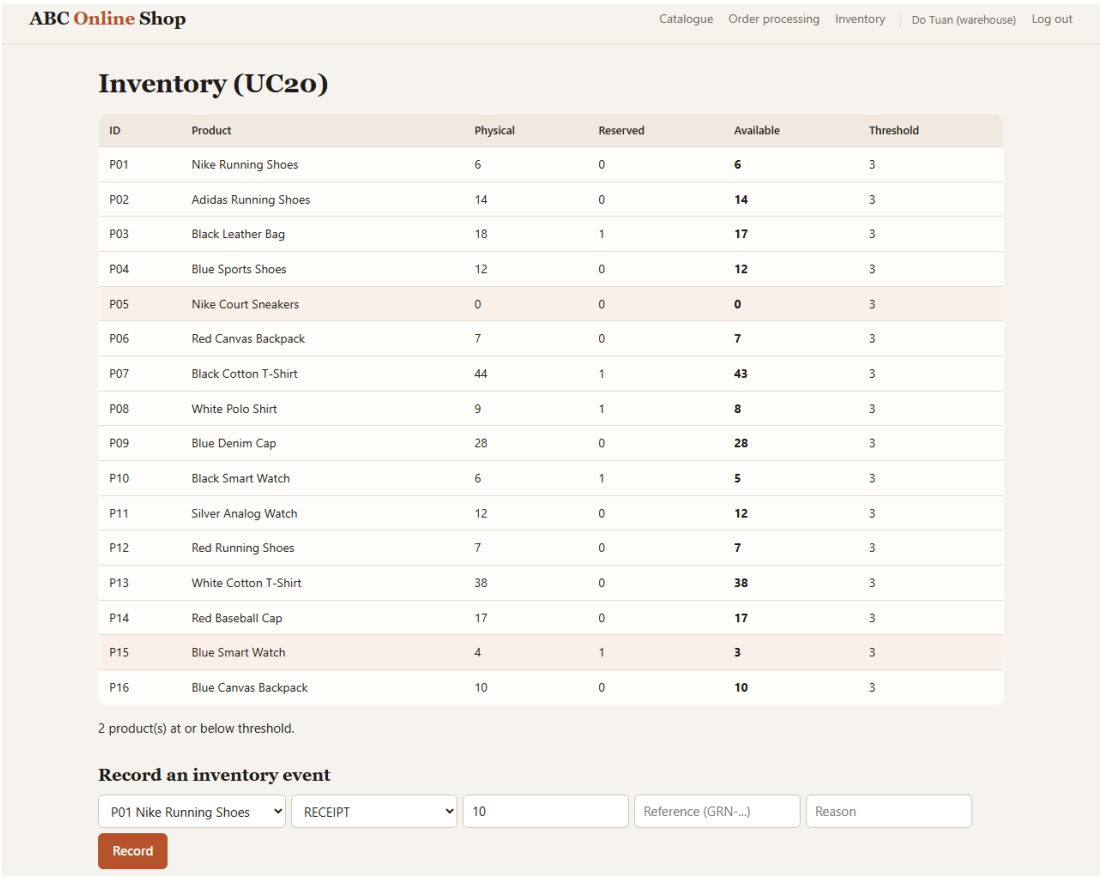



*Figure 48. Warehouse inventory with low-stock rows and event form (UC20, UC24)*

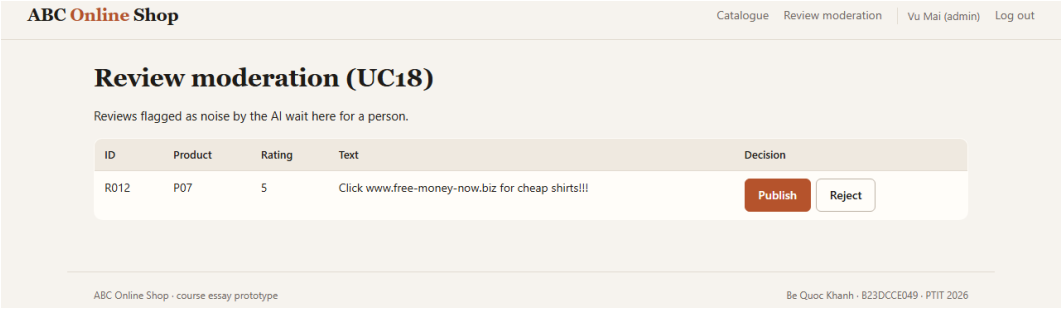



*Figure 49. Review moderation queue: the spam review held by the AI noise filter (UC18, AI-R5)*

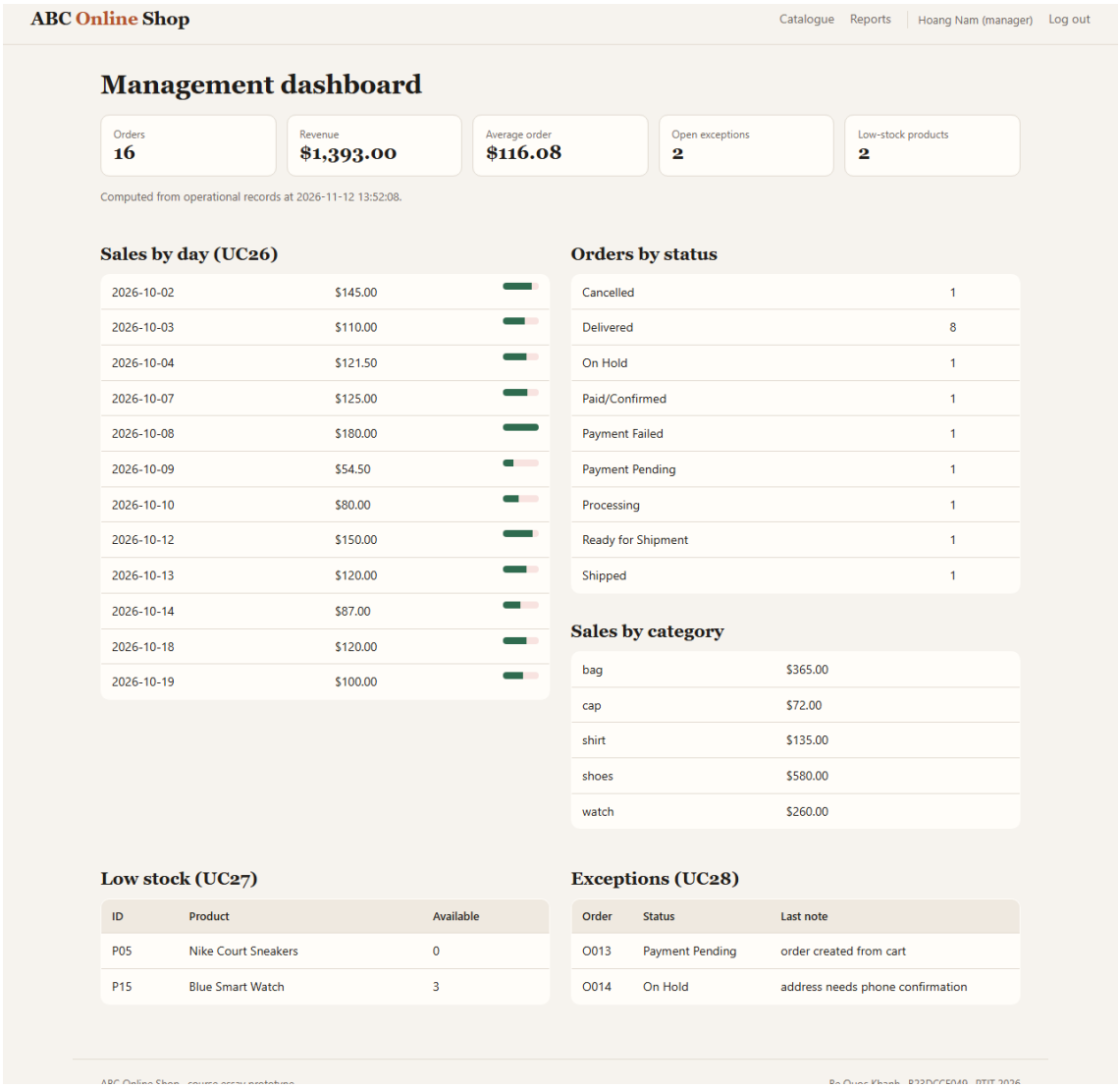



*Figure 50. Management dashboard computed from operational records (UC25-UC28)*

## 5.7. AI Components in Code

The search keeps the Assignment 06 classes; the change is in where the data comes from. SearchService now reads ProductDAO and InventoryDAO, so a result shows the same available quantity as the product page, and order questions go through OrderDAO.find_by_customer, so the privacy rule is structural:

```
def search_orders(self, query):
    """Order questions only ever see the orders of the customer in the session (FR11)."""
    own = self.orders.find_by_customer(query["customer_id"])
    if query.get("order_id"):
        return [o for o in own if o.order_id == query["order_id"]]
    if query.get("latest"):
        return own[:1]
    return own
```

The review analyser's core is the clause classifier with the negation window of Section 3.6:

```
def classify_clause(self, clause):
    """Return (aspects, polarity) for one clause; polarity is +1, -1 or 0."""
    tokens = re.findall(r"[a-z']+", clause)
    aspects = [a for a, words in ASPECTS.items()
               if any(w in tokens or (" " in w and w in clause) for w in words)]
    score = 0
```

```
        for i, t in enumerate(tokens):
            value = 1 if t in POSITIVE else -1 if t in NEGATIVE else 0
            if value and any(n in tokens[max(0, i - 3):i] for n in NEGATIONS):
                value = -value
            score += value
        return aspects, (score > 0) - (score < 0)
```

The composition root wires the AI components like every other service. The last lines show the design decision of Section 3.5: the encoder is passed into the data layer instead of being imported by it.

```
    def _rebuild_index(shop):
        """The encoder is passed into the data layer, which never imports the AI layer."""
        shop.search.vector_index = VectorIndex.build(shop.storage, shop.dao.products.list_products(),
                                                     shop.image.encode)
        return shop.search.vector_index
```

## 5.8. Running the Application

*Table 54. Commands*

| Command | What it does |
|---|---|
| pip install -r requirements.txt | numpy, Pillow, pytest |
| python main.py seed | Create data/abc_shop.db: 16 products, 4 customers, 4 staff, 16 orders, 14 reviews |
| python main.py serve --port 8000 | Run the web application on http://localhost:8000 (demo accounts on the login page) |
| python main.py demo | End-to-end scenario printed step by step (in-memory database) |
| python main.py search --voice "find running shoes" | One search; also --text, --image FILE, --text + --image |
| python evaluate.py | Search, review-insight and recommender evaluation |
| python -m pytest -q | All 45 tests |

The seed is not inserted into the tables directly. seed.py plays one month of history through the real services with a simulated clock: customers fill carts and check out, payments succeed, fail or time out, sales staff process and hold orders, the warehouse packs, the carrier sends milestones and customers review delivered products. Every row in the demo database was therefore produced by the same rules that the tests check. The demo then continues with a new checkout, a repeated submission, a declined payment and a full fulfilment path including duplicate and out-of-order carrier messages:

```
$ python main.py demo   (sections 2-8)
--- 2. UC07 + UC08 checkout with promotion WELCOME10 ---
  cart: [('P12', 1), ('P09', 2)]  subtotal 160  discount 16  shipping 0  total 144
  CheckoutService.place_order -> O017 Paid/Confirmed (payment Succeeded)
  P12 available 7 -> 6 (reserved for O017)
  same submission again -> O017, duplicate=True (no second order/charge)

--- 3. Payment declined: order fails, reservation released, cart kept ---
  O018: Payment Failed; P10 available 5; cart still has ['P10']

--- 4. UC19 -> UC21 -> UC22 -> UC23: fulfilment of O017 ---
  processed by S01, packed SH011, carrier ref VNPF21E3D57B
  milestone 1 PICKED_UP  -> accepted
  milestone 2 IN_TRANSIT -> accepted
  milestone 2 IN_TRANSIT -> ignored (duplicate/older)
  milestone 1 PICKED_UP  -> ignored (duplicate/older)
  milestone 3 DELIVERED  -> accepted
  UC11 Track Order -> Delivered, 9 timeline entries

--- 5. Order questions are limited to the customer's own orders ---
  input (text): 'where is my latest order'
  QueryService -> order query (order_id=None, latest=True, customer=C001)
  Order: O018  Status: Payment Failed  Total: 150  Date: 2026-10-19
  input (text): 'find my order O002'
  QueryService -> order query (order_id=O002, latest=False, customer=C001)
  no order of this customer matches

--- 6. AI review insights for P01 Nike Running Shoes ---
  4 reviews used, 0 skipped -> Mostly positive
  quality   +4 -0  positive share 1.00
  price     +1 -0  positive share 1.00
  delivery  +2 -1  positive share 0.67
  waiting for moderation: [('R012', 'Click www.free-money-now.biz f')]

--- 7. Recommendations ---
  similar to P01: P12 Red Running Shoes      score 0.367 (bought together 0x)
  similar to P01: P03 Black Leather Bag      score 0.338 (bought together 1x)
  similar to P01: P07 Black Cotton T-Shirt   score 0.335 (bought together 1x)
  for C001:       P02 Adidas Running Shoes   score 0.809
  for C001:       P13 White Cotton T-Shirt   score 0.692
  for C001:       P14 Red Baseball Cap       score 0.605

--- 8. UC25 management dashboard ---
  orders 18  revenue 1537  average 118.23  exceptions 2  low stock ['P05', 'P15']
```

*Figure 51. Output of main.py demo, sections 2-8: checkout, idempotency, failure, fulfilment, AI and dashboard*

## 5.9. Testing

The tests were written from the use case specifications and the design rules, so a failing test names the requirement that is broken. The architecture test turns the package diagram into code: it parses every module with Python's ast module and fails if a layer imports a layer it must not use.

```
FORBIDDEN = {
    "model": {"dao", "service", "ai", "controller", "view", "shop"},
    "dao": {"service", "ai", "controller", "view", "shop"},
    "service": {"controller", "view", "shop", "ai"},
    "ai": {"controller", "view", "shop", "service"},
    "controller": {"dao", "shop"},
    "view": {"dao", "service", "ai", "shop", "controller"},
}

@pytest.mark.parametrize("layer", FORBIDDEN)
def test_dependencies_only_point_downwards(layer):
    for module in modules_of(layer):
        illegal = imported_packages(module) & FORBIDDEN[layer]
        assert not illegal, f"{os.path.relpath(module, ROOT)} imports {illegal}"
```

The payment test is the clearest example of a requirement becoming a test. NFR07 says that a replay must not duplicate a state change, and UC09 lists forged results (E2), mismatching amounts (E1) and duplicate callbacks (A3). The test sends all of them to the same payment attempt and checks that the order became Paid/Confirmed exactly once:

```
def test_payment_callbacks_are_verified_and_idempotent(shop, users):
    shop.cart.add_item("C001", "P10", 1)
    order = shop.checkout.place_order(users["C001"], "", "tok_timeout")["order"]
    attempt = shop.dao.payments.for_order(order.order_id)[0]
    good = {"attempt_ref": attempt.attempt_ref, "status": "SUCCEEDED", "provider_ref": "PSP-X",
            "amount": attempt.amount, "currency": "USD"}
    forged = dict(good, signature="0000")
    assert shop.payment.handle_result(forged) == "Rejected"                         # E2
    wrong_amount = dict(good, amount=1.0)
    wrong_amount["signature"] = sign(attempt.attempt_ref, "SUCCEEDED", "PSP-X", 1.0, "USD")
    assert shop.payment.handle_result(wrong_amount) == "Quarantined"                # E1
    assert shop.dao.orders.get_order(order.order_id).status == "Payment Pending"
    good["signature"] = sign(attempt.attempt_ref, "SUCCEEDED", "PSP-X", attempt.amount, "USD")
    assert shop.payment.handle_result(good) == "Succeeded"
    assert shop.payment.handle_result(good) == "Succeeded"                          # A3 duplicate
    history = shop.dao.orders.get_order(order.order_id).history
    assert [h.status for h in history].count("Paid/Confirmed") == 1
```

*Table 55. Test modules*

| Module | Tests | What is checked |
|---|---|---|
| test_model.py | 5 | Cart totals, order transitions, stock rules, promotion validity, milestone ordering |
| test_checkout.py | 9 | UC08 main flow and A2, A4, A5, A6, A7, price change, address; UC09 E1, E2, A3 |
| test_operations.py | 8 | Tracking ownership, cancellation and refund, processing roles, fulfilment path, inventory idempotency, replenishment, review rules, reports |
| test_ai.py | 7 | Text/voice/image/multimodal search, synonyms, stock-aware ranking, order privacy, review negation and noise, recommender rules |
| test_web.py | 4 | Pages through WSGI: catalogue, search trace, a customer buying through the web, role enforcement |
| test_architecture.py | 7 | Layer rule for six packages (parametrised) and no storage import in services |

```
$ python -m pytest -v
test_ai.py::test_text_voice_and_price_filter PASSED                [  2%]
test_ai.py::test_synonyms_map_sneakers_to_shoes PASSED             [  4%]
test_ai.py::test_image_and_multimodal_search PASSED                [  6%]
test_ai.py::test_ranking_is_stock_aware PASSED                     [  8%]
test_ai.py::test_order_questions_only_see_own_orders PASSED        [ 11%]
test_ai.py::test_review_insights_handle_negation_and_noise PASSED  [ 13%]
test_ai.py::test_recommender_never_suggests_out_of_stock_or_bought_items PASSED [ 15%]
test_architecture.py::test_dependencies_only_point_downwards[model] PASSED [ 17%]
test_architecture.py::test_dependencies_only_point_downwards[dao] PASSED [ 20%]
test_architecture.py::test_dependencies_only_point_downwards[service] PASSED [ 22%]
test_architecture.py::test_dependencies_only_point_downwards[ai] PASSED [ 24%]
test_architecture.py::test_dependencies_only_point_downwards[controller] PASSED [ 26%]
test_architecture.py::test_dependencies_only_point_downwards[view] PASSED [ 28%]
test_architecture.py::test_services_never_import_a_storage_implementation PASSED [ 31%]
test_catalogue.py::test_each_product_loads_the_detail_kind_of_its_category PASSED [ 33%]
test_catalogue.py::test_product_page_shows_the_group_specific_attributes PASSED [ 35%]
test_catalogue.py::test_service_rejects_a_detail_of_the_wrong_group PASSED [ 37%]
test_catalogue.py::test_database_trigger_rejects_a_detail_of_the_wrong_group PASSED [ 40%]
test_catalogue.py::test_search_uses_the_detail_attributes PASSED   [ 42%]
test_checkout.py::test_successful_checkout_reserves_stock_pays_and_clears_cart PASSED [ 44%]
test_checkout.py::test_promotion_and_shipping_are_part_of_the_authoritative_total PASSED [ 46%]
test_checkout.py::test_repeated_submission_returns_the_same_order PASSED [ 48%]
test_checkout.py::test_declined_payment_releases_reservation_and_keeps_cart PASSED [ 51%]
test_checkout.py::test_timeout_keeps_order_pending_and_reserved PASSED [ 53%]
test_checkout.py::test_insufficient_stock_stops_checkout_without_side_effects PASSED [ 55%]
test_checkout.py::test_changed_price_is_reported_before_payment PASSED [ 57%]
test_checkout.py::test_incomplete_address_is_rejected PASSED       [ 60%]
test_checkout.py::test_payment_callbacks_are_verified_and_idempotent PASSED [ 62%]
test_model.py::test_cart_adds_to_an_existing_line_and_totals_the_lines PASSED [ 64%]
test_model.py::test_order_follows_only_the_allowed_transitions PASSED [ 66%]
test_model.py::test_stock_level_keeps_available_equal_physical_minus_reserved PASSED [ 68%]
test_model.py::test_promotion_checks_dates_and_minimum_total PASSED [ 71%]
test_model.py::test_shipment_ignores_duplicate_and_older_milestones PASSED [ 73%]
test_operations.py::test_customer_only_tracks_own_orders PASSED    [ 75%]
test_operations.py::test_cancel_releases_the_reservation_and_notes_a_refund PASSED [ 77%]
test_operations.py::test_processing_needs_staff_role_and_a_paid_order PASSED [ 80%]
test_operations.py::test_full_fulfilment_path_and_milestones PASSED [ 82%]
test_operations.py::test_inventory_events_are_idempotent_and_never_negative PASSED [ 84%]
test_operations.py::test_low_stock_and_replenishment PASSED        [ 86%]
test_operations.py::test_reviews_only_for_delivered_purchases_and_spam_waits_for_moderation PASSED [ 88%]
test_operations.py::test_reports_are_for_managers_and_match_the_orders PASSED [ 91%]
test_web.py::test_catalogue_and_product_page_with_ai_sections PASSED [ 93%]
test_web.py::test_search_shows_the_processing_steps PASSED         [ 95%]
test_web.py::test_customer_buys_through_the_web PASSED             [ 97%]
test_web.py::test_roles_are_enforced_on_screens PASSED             [100%]
============================== 45 passed in 18.25s ==============================
```

*Figure 52. Output of pytest: 45 tests passed*

## 5.10. Traceability from Requirements to Code and Tests

*Table 56. From requirement to design element, code and test*

| Requirement | Design element | Code | Checked by |
|---|---|---|---|
| FR08 validate before ordering | CartService.validate_cart (UC29-30) | service/cart_service.py | test_changed_price_is_reported_before_payment, test_insufficient_stock_... |
| FR09 + A7 one order per submission | idempotency key, UNIQUE constraint | checkout_service.py, schema.sql | test_repeated_submission_returns_the_same_order |
| FR10, NFR07 payment once | PaymentService.handle_result | service/payment_service.py | test_payment_callbacks_are_verified_and_idempotent |
| FR11-12, NFR05 own orders only | OrderService.own_order, search_orders | order_service.py, search_service.py | test_customer_only_tracks_own_orders, test_order_questions_only_see_own_orders |
| FR18 valid transitions | Order.change_status, TRANSITIONS | model/order.py | test_order_follows_only_the_allowed_transitions |
| FR19 stock never negative | StockLevel.apply, CHECK constraints | model/inventory.py, schema.sql | test_inventory_events_are_idempotent_and_never_negative |
| FR21 milestones, no regression | Shipment.add_milestone | model/shipment.py | test_shipment_ignores_duplicate_and_older_milestones |
| FR14, AI-R5 reviews | ReviewService.submit + insight noise filter | review_service.py | test_reviews_only_for_delivered_purchases_and_spam_... |
| NFR09 layers | Package diagram PKG01 | all packages | test_dependencies_only_point_downwards |

## 5.11. What Writing the Code Changed in the Models

Checking the code against the diagrams found four real mismatches. None produced a wrong result in the demo, which is exactly why they are dangerous: they appear only when someone later changes the system.

*Table 57. Mismatches found while implementing*

| Mismatch | How it was found | Fix |
|---|---|---|
| Two controllers read DAOs through a service attribute (order_service.orders.find_by_status) | Reading the controller imports against PKG01 | ShipmentService.ready_list and a shipment id in the redirect; no DAO access remains in controllers |
| The image index was built before the catalogue was loaded, so image search returned nothing | First run of the demo (0 candidates) | seed.py rebuilds the index after loading the catalogue; composition root offers rebuild_index() |
| The state machine did not allow Delivery Failed -> Delivered | Writing the carrier milestone handler | Transition added to ST01 and to TRANSITIONS; unexpected milestones are audited instead of crashing |
| The recommender test counted cancelled orders as purchases | Failing test on the seeded data | The test now uses the same definition as the service and the analysis: cancelled and failed orders are not purchases |

## 5.12. Development History

The code was developed in small commits over four days, in the order of the layers: model, DAO, services, AI, presentation, tests, models and report. The history is public in the repository.

*Table 58. Commit history*

| Date and time | Commit |
|---|---|
| 2026-10-18 09:42 | Initial project structure |
| 2026-10-18 10:15 | Add customer, product and cart model classes |
| 2026-10-18 14:37 | Add order state machine, payment, shipment and review models |
| 2026-10-18 16:02 | Add event-based inventory model |
| 2026-10-18 21:18 | Add SQLite schema and database helper with nested transactions |
| 2026-10-19 08:55 | Add DAO interfaces and SQLite implementations |
| 2026-10-19 10:41 | Add auth, catalogue and cart services |
| 2026-10-19 13:26 | Add inventory service and checkout with idempotency key |
| 2026-10-19 15:50 | Add payment service and simulated provider gateways |
| 2026-10-19 19:34 | Add order processing and shipment services |
| 2026-10-19 22:07 | Add review moderation and management reports |
| 2026-10-20 09:12 | Port multimodal search from assignment 06 |
| 2026-10-20 11:03 | Add review insight analyser and recommender |
| 2026-10-20 14:22 | Add composition root and seeded October history |
| 2026-10-20 16:48 | Add web controllers and router |
| 2026-10-20 20:31 | Add HTML views and stylesheet |
| 2026-10-20 22:15 | Add main entry point with end-to-end demo |
| 2026-10-21 08:47 | Add model, checkout and operation tests |
| 2026-10-21 10:12 | Add AI, web and architecture tests |
| 2026-10-21 11:39 | Add evaluation script and labelled reviews |
| 2026-10-21 14:05 | Add Visual Paradigm project and exported diagrams |
| 2026-10-21 16:26 | Add demo results and web screenshots |
| 2026-10-21 20:44 | Add essay report |
| 2026-10-21 22:31 | Update README with run instructions |
| 2026-10-21 22:52 | Relayout diagrams with orthogonal connectors and split use cases by package |

## 5.13. Chapter Summary

The design of Chapter 4 runs as 3778 lines of Python in the same six packages, with a web interface, a SQLite database and the three AI components. The use case flows of Chapter 1 appear as comments and tests, the state machine and the integrity rules of Chapter 2 as code and constraints, and the package diagram as an architecture test. All 45 tests pass, and the demo reproduces every number quoted in the essay.

# CHAPTER 6. CONCLUSION AND FUTURE WORK

## 6.1. Summary of Deliverables

This essay followed ABC Online Shop from its business problem to a running program. Requirement determination turned fragmented and delayed information into 7 problems, 14 needs, 24 functional and 12 non-functional requirements and 32 use cases, plus 9 requirements for AI extensions. Analysis refined them with activity, data-flow, sequence, class, state and data models that agree with each other. The AI chapter treated multimodal search, review insights and recommendation as ordinary requirements with data, risks, fallbacks and measurements. The design turned the updated analysis classes into a generated database and a layered MVC architecture with services and data access objects, whose class skeletons were generated from the model, and the code implements it with 45 passing tests.

## 6.2. Evaluation Against the Requirements

*Table 59. Requirement groups and the evidence that they are met*

| Requirement group | Status | Evidence |
|---|---|---|
| FR01-FR02 account, login, roles | Met | test_web, test_operations (permissions) |
| FR04-FR07 catalogue, search, product details, cart | Met | test_ai, test_catalogue, test_web; screenshots 5.x |
| FR08-FR10 checkout, order, payment | Met | test_checkout (idempotency, failure, signature), demo output |
| FR11-FR14 history, tracking, cancellation, reviews | Met | test_operations |
| FR15-FR17 catalogue, promotions, customers (administration) | Partly | Services and tests; no page for categories and customers |
| FR18-FR22 processing, inventory, shipment, replenishment | Met | test_operations; staff pages |
| FR23-FR24 reports, promotions | Met | test_operations (reports); checkout with WELCOME10 |
| NFR05-NFR07, NFR10 security, audit, integrity, privacy | Met | Permission checks, audit_log, constraints, build_db.py block [4] |
| NFR09 maintainability (layers) | Met | test_architecture; check_code.py |
| NFR01-NFR03, NFR12 performance, availability, load | Not measured | Prototype runs as one process on SQLite |
| AI-R1 to AI-R9 search, review insights, recommendation | Met on the test sets | evaluate.py: 25/26 search, F1 0.96, coverage 15/15 |

## 6.3. Limitations

- **Scope.** Screens exist for the transaction path; profile editing (UC03), category and customer management (UC15, UC17) are implemented in services but not as pages.
- **Providers.** Payment and carrier are simulated behind PaymentGateway and ShippingGateway; a real integration needs webhooks over HTTPS and a reconciliation job for Payment Pending orders.
- **AI.** Speech is simulated, the image encoder only sees colour and shape, and the review analyser is a lexicon; Section 3.12 lists the production replacements behind the same interfaces.
- **Non-functional requirements.** NFR01-03 and NFR12 (performance, availability, load) were not measured; the prototype runs on one process with SQLite.
- **Data.** 16 products and synthetic images; the AI numbers do not predict accuracy on a real catalogue.

## 6.4. Lessons and Next Steps

The main lesson is about the joints between the activities. The rules that survived all five chapters were the ones written precisely at the start: status from recorded events, unique references for every external exchange, stock that can never become negative, customers who only see their own orders. Each of them is a sentence in Chapter 1, a guard or constraint in Chapter 2, a method or a unique key in Chapter 4 and a test in Chapter 5. Where a rule was vague, as with the definition of a purchase for recommendations, the vagueness came back as a failing test. AI did not change this lesson; it confirmed it, because an AI component is only as good as the operational data and the rules around it.

Natural next steps are a real payment and carrier integration with reconciliation, measurement of the performance requirements, and replacing the prototype AI components with Whisper, CLIP and a fine-tuned PhoBERT behind the interfaces that are already in place.

## 6.5. Submitted Files

*Table 60. Submitted files*

| File / folder | Content |
|---|---|
| tieuluan_V1_04_049_KHANHBQ.pdf | This essay |
| TieuLuan_KHANHBQ.vpp | Visual Paradigm project with the 29 diagrams of the essay |
| diagrams/ | PNG export of every diagram |
| abc_shop/ | Source code, data, tests, README |
| design/ | SQL script generated from the ERD, build_db.py, generate_code.py, check_code.py and the generated code |
| demo_results/ | Demo, evaluation and test output; web screenshots |

**Project repository:** https://github.com/qkhanhbe/ABC-Online-Shop-Information-System

# REFERENCES

[1] Tran Dinh Que, *Information System Analysis and Design*, lecture slides and assignment briefs 01-06, Posts and Telecommunications Institute of Technology, 2026.

[2] Tran Dinh Que, *Integrating AI in E-Commerce: Analysis Phase*, course material, PTIT, 2026.

[3] J. W. Satzinger, R. B. Jackson and S. D. Burd, *Systems Analysis and Design in a Changing World*, 7th ed., Cengage Learning, 2016.

[4] A. Dennis, B. H. Wixom and D. Tegarden, *Systems Analysis and Design: An Object-Oriented Approach with UML*, 5th ed., Wiley, 2015.

[5] K. C. Laudon and J. P. Laudon, *Management Information Systems: Managing the Digital Firm*, 17th ed., Pearson, 2022.

[6] M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

[7] Oracle, *Core J2EE Patterns: Data Access Object*, 2002.

[8] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision (CLIP)", ICML, 2021.

[9] A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)", 2022.

[10] D. Q. Nguyen and A. T. Nguyen, "PhoBERT: Pre-trained language models for Vietnamese", Findings of EMNLP, 2020.

[11] M. Pontiki et al., "SemEval-2014 Task 4: Aspect Based Sentiment Analysis", SemEval, 2014.

[12] Visual Paradigm, *Visual Paradigm Open API (Plug-in) Developer's Guide*, version 18.

[13] Be Quoc Khanh, Assignments 01, 02, 03, 06 and Test 01 of this course, 2026.